

UNINASSAU

LoadForge

Stress and Resilience Evaluator

RECIFE - PE
2026

Planejamento da Sprint 01

LoadForge Stress and Resilience Evaluator

Este documento define o problema, o escopo do produto, os requisitos iniciais e o plano semanal de desenvolvimento do LoadForge. O MVP concentra-se na execução segura de testes de carga e inclui inteligência artificial integrada à otimização do processamento. Um modelo local, treinado pela equipe, estimará o risco de degradação da aplicação, e o controlador adaptativo usará essa previsão para regular a concorrência do motor de carga.

1 Formação da Equipe

Turma: Fábrica de Software 2026.2

Integrante	Papel inicial
Aline Bianca Arantes da Silva	Desenvolvedora Backend Python
Camile Marcele Pereira de Araújo	Product Owner e Analista de QA
Cayo Vitor Fagundes	Desenvolvedor Frontend e UI UX
Emerson Wallace Barcelos de Araújo	Scrum Master e Desenvolvedor Backend
Ricardo Cezar Ottoni Assis de Almeida	Dados infraestrutura e documentação

2 Escolha do Tema

O tema escolhido é uma plataforma web para planejamento, execução e análise de testes de carga em ambientes autorizados. O projeto pertence às áreas de Quality Assurance, confiabilidade de software, inteligência artificial e otimização de processamento. O diferencial técnico está na integração entre um motor assíncrono, um modelo preditivo local e um controlador que regula a concorrência durante o teste.

3 Definição do Problema

Equipes de desenvolvimento nem sempre conseguem medir como uma aplicação se comporta quando o volume de acessos aumenta. Ferramentas existentes podem exigir configuração especializada, e um gerador de carga mal dimensionado pode saturar a própria máquina de teste e distorcer os resultados. Além disso, controladores baseados somente em limites reagem depois que a degradação já começou. O LoadForge pretende reduzir essa dificuldade ao executar cenários de carga controlada, aprender padrões anteriores de degradação e ajustar a concorrência antes que latência e erros ultrapassem os limites definidos pelo usuário.

O problema é relevante porque falhas sob carga afetam disponibilidade, experiência do usuário e capacidade de planejamento. A solução será restrita a aplicações próprias ou expressamente autorizadas, com limites de duração, concorrência e interrupção de emergência.

4 Objetivos do Sistema

Objetivo geral: desenvolver um sistema web que permita configurar, executar e comparar testes de carga, com persistência dos resultados, previsão de degradação por inteligência artificial e otimização adaptativa do motor gerador.

Objetivo específico 1: implementar um motor de requisições HTTP concorrentes em Python com programação assíncrona e limites configuráveis.

Objetivo específico 2: construir um conjunto de dados próprio a partir de execuções controladas e treinar localmente um modelo que estime o risco de degradação nos próximos segundos.

Objetivo específico 3: integrar a previsão do modelo a um controlador que ajuste a concorrência a partir do risco previsto e das métricas observadas, sem depender de API externa.

Objetivo específico 4: comparar carga fixa, otimização por regras e otimização assistida por IA com métricas de desempenho, eficiência e qualidade preditiva.

Objetivo específico 5: oferecer autenticação, perfis de acesso, cadastro de projetos e endpoints e histórico dos testes executados.

4.1 Escopo do MVP

Escopo	Definição
Incluído no MVP	Login com perfis QA e Visualizador; projetos e endpoints; motor assíncrono; coleta de dados por janelas; treinamento e validação de modelo local; previsão de risco durante o teste; controlador híbrido com IA; comparação entre três estratégias; persistência, relatórios, testes e instruções de execução.
Fora do MVP	Chatbot ou IA generativa; consumo de API externa de IA; seleção autônoma de alvos; assinaturas e cobrança; arquitetura multi-tenant; cluster em várias máquinas; aplicativo móvel; plugins; relatórios comerciais avançados; integração com Celery.

4.2 Inteligência Artificial e Otimização

O componente avançado será formado pelo DegradationRiskModel e pelo AdaptiveLoadController. O primeiro estimará a probabilidade de degradação da aplicação; o segundo combinará essa previsão com as métricas observadas para ajustar a concorrência do motor. Todo o processamento ocorrerá localmente e será integrado ao fluxo principal do sistema.

4.2.1 Modelo de Previsão de Degradação

Durante execuções controladas, o sistema agrupará os dados em janelas de dois segundos. Cada amostra poderá incluir concorrência atual, throughput, latências p50, p95 e p99, taxa de erro, timeouts, CPU, memória e tendências recentes dessas medidas. A amostra será rotulada como degradação próxima quando, nos dez segundos seguintes, ocorrer violação do limite de latência, aumento relevante de erros ou queda de throughput mesmo com aumento de concorrência.

A equipe implementará o pipeline de preparação dos dados, criação dos rótulos, treinamento, validação, seleção e versionamento do modelo. Serão comparados ao menos um modelo linear, como regressão logística, e um modelo baseado em árvores, como Random Forest, usando Python e scikit-learn. A escolha considerará F1, recall, falsos positivos e tempo de inferência. O modelo escolhido será salvo e executado localmente; nenhuma API externa de IA será utilizada.

4.2.2 Controlador Híbrido de Concorrência

O AdaptiveLoadController receberá, a cada janela, a probabilidade de degradação produzida pelo modelo e as métricas atuais. Os limites iniciais serão configuráveis e poderão ser ajustados após os experimentos.

- Risco baixo e métricas estáveis: aumentar gradualmente a concorrência dentro do limite definido.
- Risco intermediário: manter a concorrência e aguardar a próxima janela de observação.
- Risco alto ou violação observada: reduzir a concorrência e registrar o motivo da decisão.
- Modelo indisponível ou entrada inválida: retornar automaticamente ao controlador baseado em regras.

Cada decisão armazenará horário, versão do modelo, probabilidade prevista, métricas observadas, estratégia ativa e novo valor de concorrência. Esse registro permitirá explicar a atuação da IA e reproduzir o experimento.

4.2.3 Fluxo de Integração

Etapa	Funcionamento
1 Coleta	O motor assíncrono envia requisições e agrega as métricas da janela.
2 Preparação	O sistema calcula tendências e monta o vetor de características.
3 Inferência	O modelo local retorna a probabilidade de degradação.
4 Otimização	O controlador aumenta, mantém ou reduz a concorrência.
5 Registro	Métricas, previsão, decisão e versão do modelo são persistidas.
6 Relatório	A interface compara as estratégias e apresenta a atuação da IA.

4.3 Métricas e Protocolo de Avaliação

Métricas de processamento: avaliam o motor e o controlador.

Métrica	Cálculo	Finalidade
Throughput	Requisições concluídas com sucesso por segundo	Mede a capacidade do gerador
Latência p50 p95 p99	Percentis do tempo de resposta	Mostra comportamento típico e cauda de latência
Taxa de erro	Percentual de timeouts e respostas com erro	Verifica se o ganho compromete a estabilidade
CPU e memória do gerador	Consumo médio e pico durante o teste	Detecta saturação da máquina que gera a carga
Eficiência	Throughput dividido pelo uso médio de CPU	Compara uso de recursos entre os modos
Estabilidade do controle	Quantidade e amplitude das alterações de concorrência	Identifica oscilações excessivas

Métricas de inteligência artificial: avaliam a capacidade preditiva e o custo de inferência.

Métrica	Cálculo	Finalidade
Precisão	Proporção das previsões de risco que antecedem degradação	Mede a confiabilidade dos alertas
Recall	Proporção das degradações antecipadas pelo modelo	Mede a capacidade de não perder eventos
F1	Média harmônica entre precisão e recall	Métrica principal para seleção do modelo
Falsos positivos	Alertas sem degradação posterior	Evita reduções desnecessárias de carga
Antecedência	Tempo entre o alerta e a degradação observada	Mostra utilidade operacional da previsão
Latência de inferência	Tempo para gerar uma previsão	Confirma que a IA não atrasa o controle

Os dados serão separados cronologicamente em conjuntos de treino, validação e teste para reduzir vazamento entre execuções. Cada cenário final será repetido pelo menos três vezes no mesmo ambiente e com os mesmos limites, comparando: modo fixo, controlador por regras e controlador híbrido com IA. A equipe usará a mediana dos resultados.

As metas iniciais são F1 de pelo menos 0,75, recall de pelo menos 0,80 e inferência inferior a 100 milissegundos por janela. Para a otimização, espera-se ganho de pelo menos 10 por cento em throughput ou redução de pelo menos 10 por cento em CPU, latência p95 ou taxa de erro, sem piorar outra métrica crítica em mais de 10 por cento. Resultados inferiores às metas serão documentados e usados para revisar dados, rótulos, modelo e regras de controle.

5 Público Alvo

O LoadForge será utilizado principalmente por:

- profissionais de QA e testes que precisam reproduzir cenários de carga;
- desenvolvedores backend que investigam gargalos e regressões;
- profissionais de SRE e DevOps que acompanham capacidade e resiliência;
- gestores técnicos que precisam consultar resultados consolidados.

6 Requisitos Funcionais

ID	Requisito
RF01	Permitir cadastro, login e logout com os perfis QA e Visualizador.
RF02	Permitir ao perfil QA cadastrar, editar e remover projetos e endpoints autorizados.
RF03	Permitir configurar método HTTP, duração, concorrência inicial, limite máximo, ramp-up, timeout e estratégia de controle.
RF04	Exigir confirmação de autorização do alvo antes da execução e oferecer interrupção de emergência.
RF05	Executar requisições concorrentes pelo motor assíncrono, respeitando os limites configurados.
RF06	Coletar por janelas as métricas e tendências necessárias para formar o conjunto de dados da IA.
RF07	Permitir executar o pipeline de preparação, rotulagem, treinamento e validação do modelo local.
RF08	Carregar uma versão aprovada do modelo e estimar o risco de degradação durante o teste.
RF09	No modo híbrido, ajustar a concorrência a partir do risco previsto e das métricas observadas.
RF10	Retornar ao controle por regras quando o modelo estiver indisponível ou receber dados inválidos.
RF11	Persistir configurações, janelas de métricas, previsões, decisões, versão do modelo e resultados.
RF12	Comparar os modos fixo, adaptativo por regras e adaptativo com IA em um relatório único.
RF13	Permitir ao perfil Visualizador consultar projetos, execuções e relatórios sem alterar dados.

7 Requisitos Não Funcionais

ID	Requisito
RNF01	O backend e o motor de carga devem ser desenvolvidos em Python com FastAPI e programação assíncrona.
RNF02	O pipeline de IA deve ser desenvolvido em Python, executado localmente e não pode depender de serviço ou API externa de IA.
RNF03	Dados, parâmetros, semente, versão do modelo e resultados de avaliação devem ser registrados para permitir reprodução.

ID	Requisito
RNF04	A separação de treino, validação e teste deve respeitar a ordem temporal das execuções para reduzir vazamento de dados.
RNF05	A inferência deve ocorrer em até 100 milissegundos por janela no ambiente de referência da equipe.
RNF06	O controlador deve continuar operando por regras se a inferência falhar ou o modelo não estiver disponível.
RNF07	O sistema deve limitar concorrência, conexões, duração e timeout para evitar consumo não controlado de recursos.
RNF08	As métricas devem usar relógio monotônico e unidades documentadas para permitir repetição dos testes.
RNF09	Senhas devem usar hash seguro; segredos não podem ser versionados; rotas devem aplicar autorização por perfil.
RNF10	A execução deve ocorrer apenas após declaração de autorização do alvo e deve possuir cancelamento imediato.
RNF11	O PostgreSQL deve persistir usuários, projetos, endpoints, configurações, métricas, previsões e decisões.
RNF12	A interface deve mostrar status, risco previsto, estratégia ativa, decisões e gráficos legíveis.
RNF13	O código deve incluir testes do pré-processamento, interface do modelo, controlador e fluxo integrado.
RNF14	O repositório deve conter documentação técnica, modelo do banco, descrição do conjunto de dados, ficha do modelo e instruções de execução.

8 Casos de Uso

Ator	Caso de uso	Descrição
QA e Visualizador	Autenticar-se	Acessa o sistema conforme o perfil cadastrado.
QA	Gerenciar projeto e endpoint	Mantém os dados do sistema autorizado que será avaliado.
QA	Configurar teste	Define carga, duração, limites, timeout e estratégia fixa, por regras ou com IA.
QA	Executar e interromper teste	Inicia o motor, acompanha o status e pode acionar a parada de emergência.

Ator	Caso de uso	Descrição
Equipe técnica	Treinar e validar modelo	Prepara o conjunto de dados, treina candidatos, avalia e aprova uma versão local.
QA	Executar controle com IA	Executa o teste com previsão de risco integrada ao controlador.
QA	Comparar estratégias	Compara carga fixa, controle por regras e controle assistido por IA.
QA e Visualizador	Consultar relatório	Visualiza métricas, risco previsto, decisões, versão do modelo e histórico.

9 Product Backlog

ID	Item	Prioridade	Entrega	Critério de aceite
PB01	Definir arquitetura, modelo de dados e ambiente do projeto	Alta	MVP	Diagrama, esquema inicial e projeto executando localmente
PB02	Implementar autenticação e perfis QA e Visualizador	Alta	MVP	Acesso e permissões validados por testes
PB03	Implementar projetos e endpoints	Alta	MVP	CRUD persistido com validação de URL
PB04	Implementar configuração e validação do cenário	Alta	MVP	Limites, duração, timeout e autorização registrados
PB05	Implementar motor assíncrono em modo fixo	Alta	MVP	Execução concorrente com cancelamento e métricas
PB06	Coletar janelas de métricas e formar conjunto de dados	Alta	MVP	Amostras persistidas com origem e horário
PB07	Implementar rotulagem e preparação dos dados	Alta	MVP	Pipeline reproduzível e testado
PB08	Treinar e comparar modelos candidatos	Alta	MVP	Regressão logística e Random Forest avaliadas
PB09	Versionar e carregar o modelo selecionado	Alta	MVP	Modelo e metadados disponíveis localmente
PB10	Implementar controlador adaptativo por regras	Alta	MVP	Decisões testadas e registradas por janela

ID	Item	Prioridade	Entrega	Critério de aceite
PB11	Integrar previsão de risco ao controlador	Alta	MVP	Execução híbrida de ponta a ponta
PB12	Criar fallback seguro sem IA	Alta	MVP	Falha do modelo não interrompe o teste
PB13	Criar relatório e comparação entre três estratégias	Alta	MVP	Métricas de IA e processamento apresentadas
PB14	Aplicar segurança, testes e documentação técnica	Alta	MVP	Fluxos críticos testados e documentação reproduzível
PB15	Preparar vídeos e apresentação final	Média	Final	Vídeos demonstram IA integrada ao sistema

10 Cronograma Inicial

As entregas foram distribuídas em ciclos semanais até 05/12. Em cada orientação, a equipe apresentará o repositório atualizado, as funcionalidades concluídas, as dificuldades encontradas e o plano da semana seguinte.

Ciclo	Data	Objetivo semanal	Evidência esperada
Sprint 01	05/09	Tema, problema, requisitos, casos de uso, backlog e planejamento inicial	Documento da Sprint 01 e repositório criado
Sprint 02	19/09	Revisão do escopo, arquitetura de IA, modelo de dados e protótipo	Fluxo IA e otimização validado
Sprint 03	19/09	Configuração do backend, banco, frontend e integração contínua	Ambiente reproduzível e esquema inicial
Sprint 04	26/09	Autenticação e perfis de acesso	Login e autorização por perfil
Sprint 05	03/10	Projetos, endpoints e regras de autorização do alvo	CRUD e validações funcionando
Sprint 06	17/10	Coleta por janelas, persistência, rotulagem e conjunto de dados	Dataset inicial documentado
Sprint 07	24/10	Treinamento, validação e seleção do modelo local	Modelo versionado com métricas
Sprint 08	31/10	Controlador adaptativo por regras e fallback	Controle sem IA testado
Sprint 09	07/11	Integração da inferência local com o controlador	Execução híbrida de ponta a ponta
Sprint 10	14/11	Relatório de risco, decisões e comparação das estratégias	Atuação da IA visível e explicável

Ciclo	Data	Objetivo semanal	Evidência esperada
Sprint 11	21/11	Experimentos em três modos, ajustes e testes de integração	Três repetições por cenário registradas
Sprint 12	28/11	Segurança, documentação, ficha do modelo, QA e vídeos	Versão candidata à entrega
Entrega final	05/12	Entrega e apresentação	Sistema, IA, otimização, código, banco, documentação e vídeos

11 Repositório GitHub

O repositório base da equipe está disponível em:

https://github.com/camimcl/sre-fabrica-de-software

O repositório deverá manter o código-fonte organizado, histórico de commits, documentação técnica, modelo do banco de dados, descrição do conjunto de dados, pipeline de treinamento, ficha da versão do modelo, testes e instruções de instalação e execução.

Planejamento da Sprint 02

Arquitetura, modelagem, prototipação e estrutura técnica

A Sprint 02 estabelece a base técnica do LoadForge para o desenvolvimento do MVP. A arquitetura, os modelos, o protótipo e o banco de dados descrevem o mesmo fluxo entre execução autorizada, métricas, inteligência artificial local e otimização adaptativa.

Materiais do projeto: repositório GitHub | protótipo no Figma | fontes e imagens dos diagramas

12 Arquitetura do Sistema

O LoadForge será desenvolvido como um monólito modular. A aplicação web React consumirá uma API REST FastAPI. O backend concentrará autenticação, projetos, cenários, execução da carga, métricas, relatórios, treinamento e inferência. O PostgreSQL armazenará dados transacionais e experimentais, enquanto os artefatos dos modelos serão versionados localmente. Essa arquitetura reduz a complexidade operacional do MVP e preserva separação entre os módulos por contratos explícitos.

Componente	Tecnologia	Responsabilidade e comunicação
Frontend web	React, Vite e TypeScript	Consome a API REST; apresenta configuração, monitoramento e relatórios.
API e módulos	Python 3.11 e FastAPI	Valida comandos, aplica regras de negócio e coordena persistência, IA e controlador.
Motor de carga	asyncio e httpx	Executa requisições concorrentes autorizadas, com limites, timeout e cancelamento.
Métricas	Janelas de 2 segundos	Agrega throughput, latências, erros, timeouts, CPU e memória.
IA local	scikit-learn e joblib	Treina, valida, versiona e executa o DegradationRiskModel sem API externa.
Controlador adaptativo	Python	Combina risco previsto e métricas; aumenta, mantém ou reduz a concorrência com fallback por regras.
Persistência	PostgreSQL 16	Guarda usuários, cenários, execuções, métricas, previsões, decisões e relatórios.

12.1 Integração do componente computacional avançado

• Cada MetricWindow é enviada ao modelo ativo para estimar a probabilidade de degradação nos dez segundos seguintes.

• O AdaptiveLoadController usa a previsão e as métricas observadas para decidir a nova concorrência.

• Se o modelo estiver indisponível ou a entrada for inválida, o RulesFallbackController assume a decisão.

• RiskPrediction e ControlDecision são persistidos com versão do modelo, permitindo a revisão.

12.2 Diagrama de arquitetura

O diagrama é lido em três faixas: acesso e execução autorizada, ciclo adaptativo e persistência. No ciclo central, as métricas alimentam a previsão local, a previsão orienta o controle híbrido e a nova concorrência retorna ao orquestrador. O monólito modular foi escolhido para reduzir a complexidade operacional do MVP, mantendo responsabilidades separadas e todo o ciclo crítico no mesmo processo. As ligações ortogonais deixam explícita a sequência sem cruzamentos.

Arquivos editáveis e imagens: [docs/diagrams no repositório]

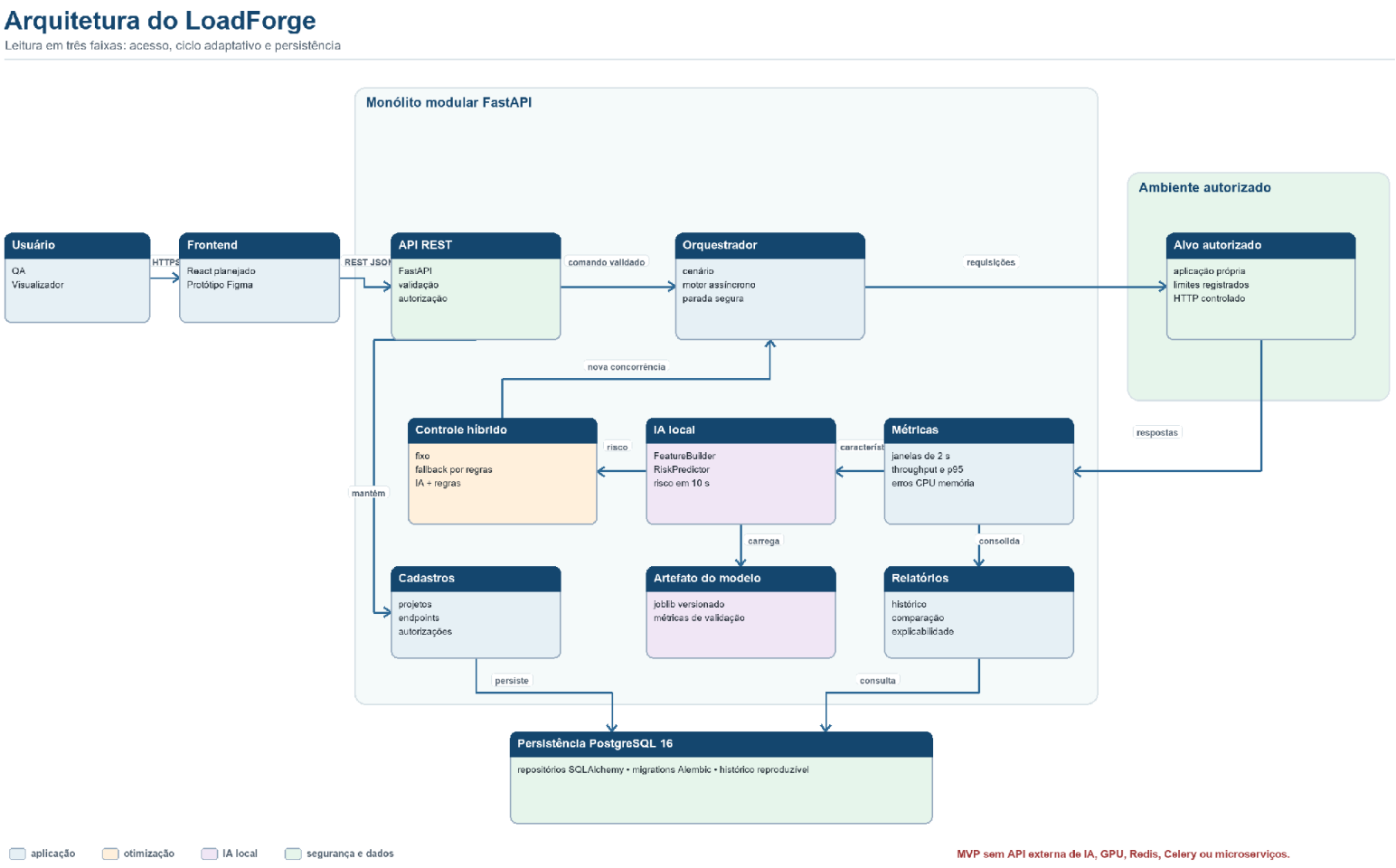


Figura 1 — Arquitetura geral e integração entre software, IA local e otimização adaptativa.

13 Diagrama de Classes

O diagrama foi dividido em dois painéis. O primeiro organiza as entidades persistidas e sua cadeia principal; o segundo apresenta serviços, contratos, IA e controle. Os contratos RiskPredictor e ConcurrencyController evitam acoplamento aos algoritmos, permitem substituir implementações e mantêm o fallback por regras testável de forma independente. Referências secundárias permanecem como atributos UUID para conservar a rastreabilidade sem criar cruzamentos visuais.

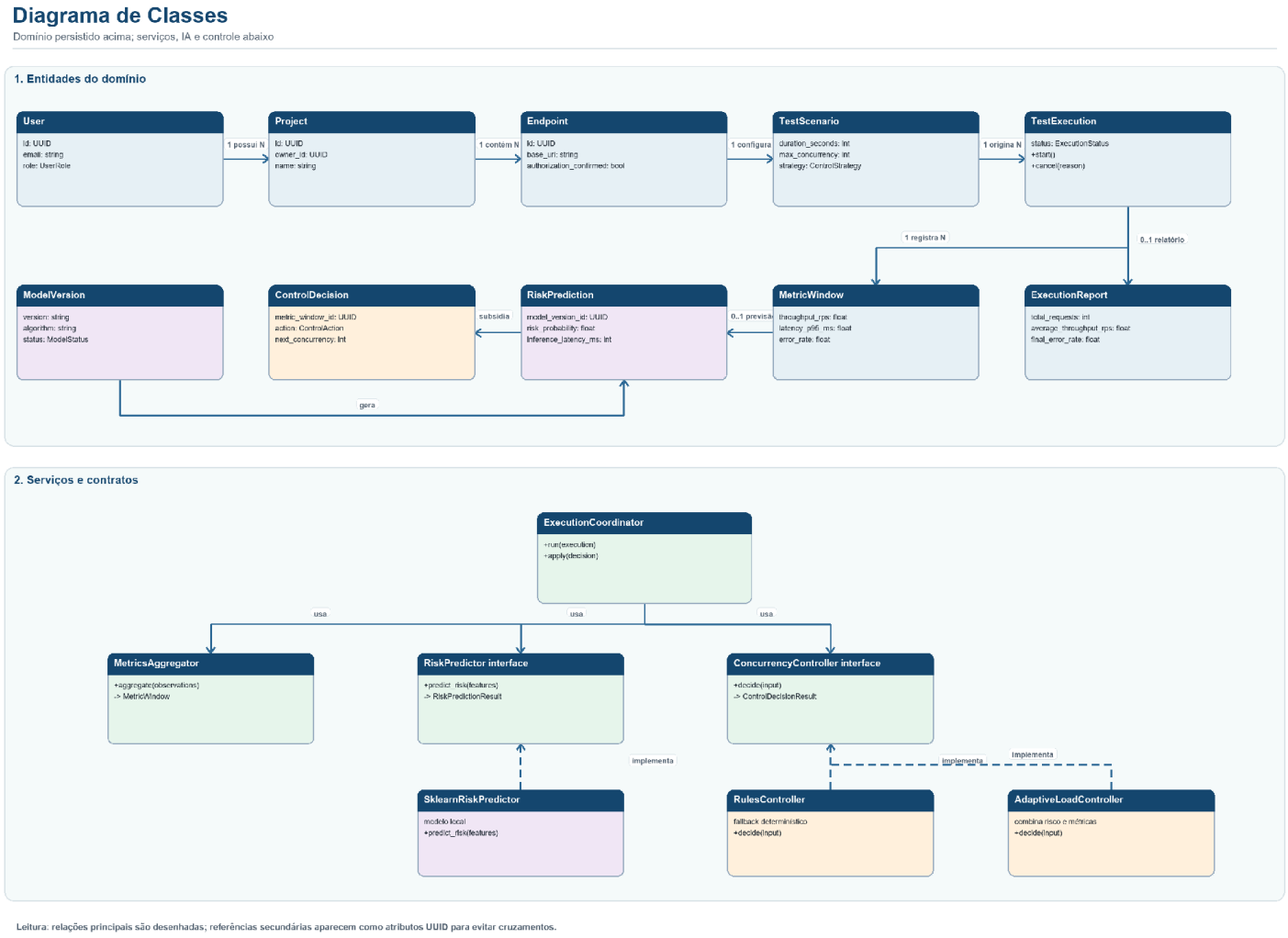


Figura 2 — Diagrama de classes com entidades, serviços e contratos do componente avançado.

14 Modelo Entidade-Relacionamento (MER)

O MER apresenta uma cadeia principal da esquerda para a direita, agrupada em cadastro e autorização, execução e métricas, e IA e controle. Relatório, decisão e versão do modelo aparecem abaixo da entidade de origem. Resultados são opcionais porque uma execução ou janela pode existir antes da conclusão do processamento. A linha tracejada identifica que uma decisão pode usar uma previsão ou ser produzida pelo fallback por regras.

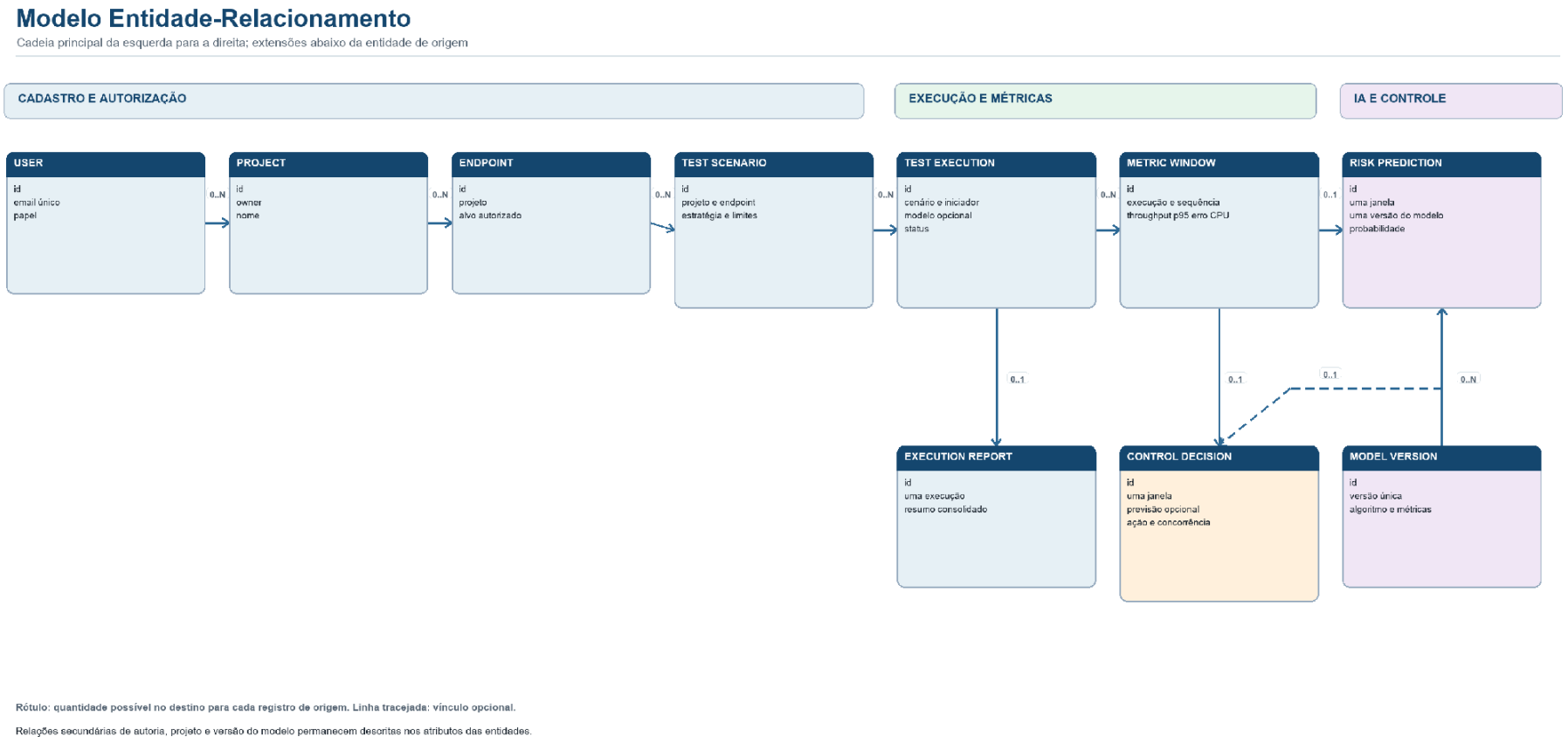


Figura 3 — MER conceitual do banco de dados.

15 Modelo Relacional

A conversão relacional usa UUIDs como chaves primárias para desacoplar a identificação dos módulos. Chaves estrangeiras preservam autoria e origem dos dados; restrições únicas limitam previsão e decisão a uma por janela e relatório a um por execução. Projetos podem existir sem endpoints, execuções sem métricas e janelas sem decisão enquanto o processamento não foi concluído. O caminho principal recebe linhas, e as demais referências aparecem dentro das tabelas como FK → tabela.id. A migration Alembic é a fonte executável do esquema e database/schema.sql apresenta a representação SQL equivalente.

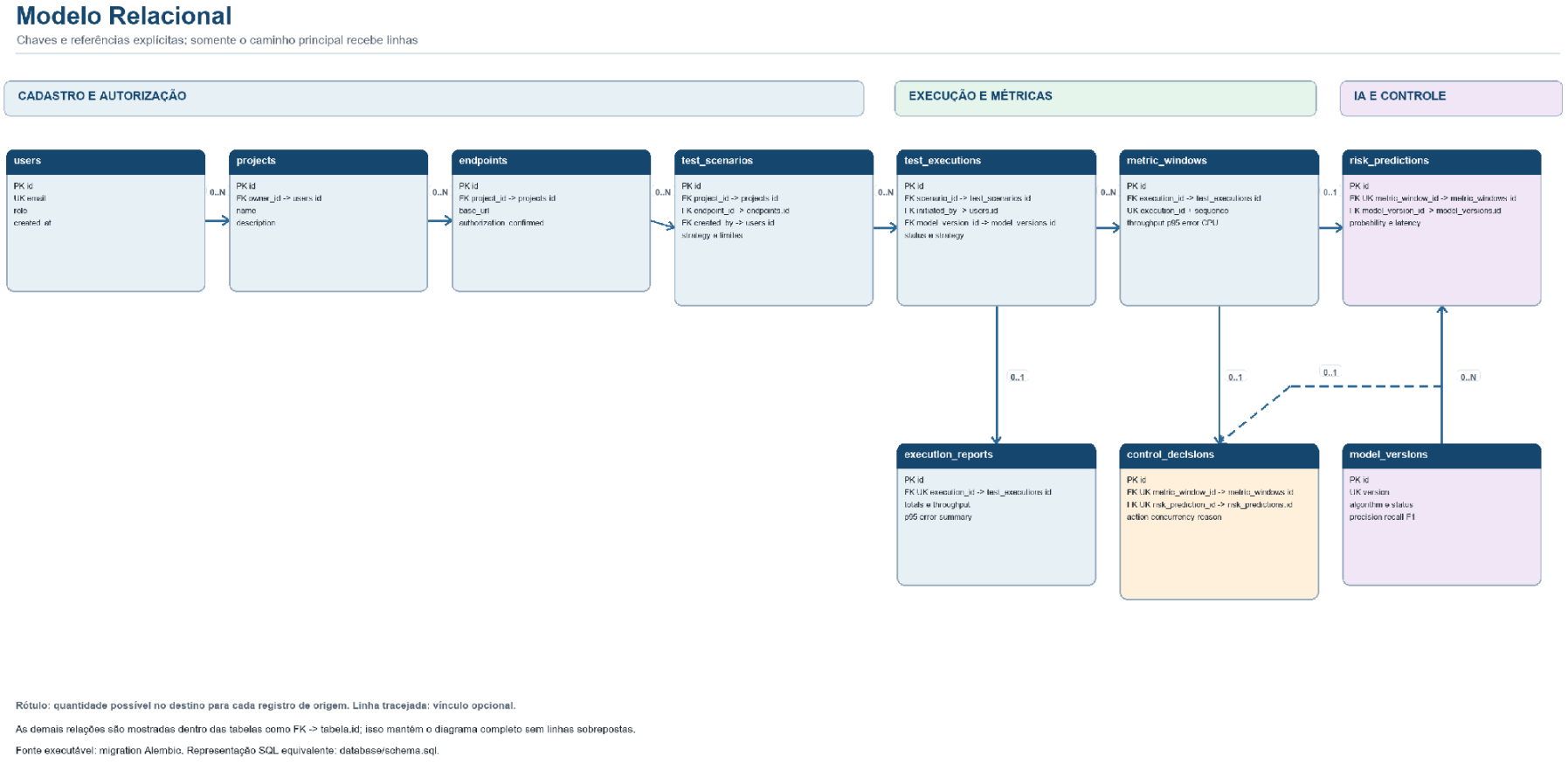


Figura 4 — Modelo relacional implementado no PostgreSQL.

16 Protótipo das Telas Principais

O protótipo inicial foi criado no Figma para validar navegação, hierarquia das informações e visibilidade da IA. Ele serve como referência temporária para a futura interface React; aparência, componentes e distribuição dos elementos poderão mudar durante a implementação e os testes de usabilidade. O fluxo atual prioriza segurança, legibilidade das métricas e compreensão das decisões do controlador.

Arquivo editável: LoadForge — Protótipo Sprint 02 no Figma

Tela	Objetivo
Login	Autenticar e comunicar que a IA é local e integrada ao produto.
Dashboard	Resumir execuções, risco atual, versão do modelo e sinais antecipados.
Projetos e endpoints	Manter alvos autorizados e seus limites operacionais.
Cadastro de endpoint	Registrar URL, método, limites e evidência de autorização.
Novo cenário	Definir carga, ramp-up, timeout, limites e estratégia de controle.
Monitoramento	Exibir métricas, risco previsto e decisões do controlador em tempo real.
Parada de emergência	Confirmar interrupção segura e preservar as métricas coletadas.
Histórico	Consultar execuções por data, projeto, estratégia e resultado.
Relatório comparativo	Comparar carga fixa, regras e IA com regras em condições equivalentes.
Modelos de IA	Apresentar versões, algoritmo, conjunto de dados e métricas de validação.

16.1 Dashboard

A tela reúne execuções recentes, risco atual e versão ativa do modelo. A síntese foi escolhida para destacar rapidamente sinais de degradação antes da abertura dos detalhes.

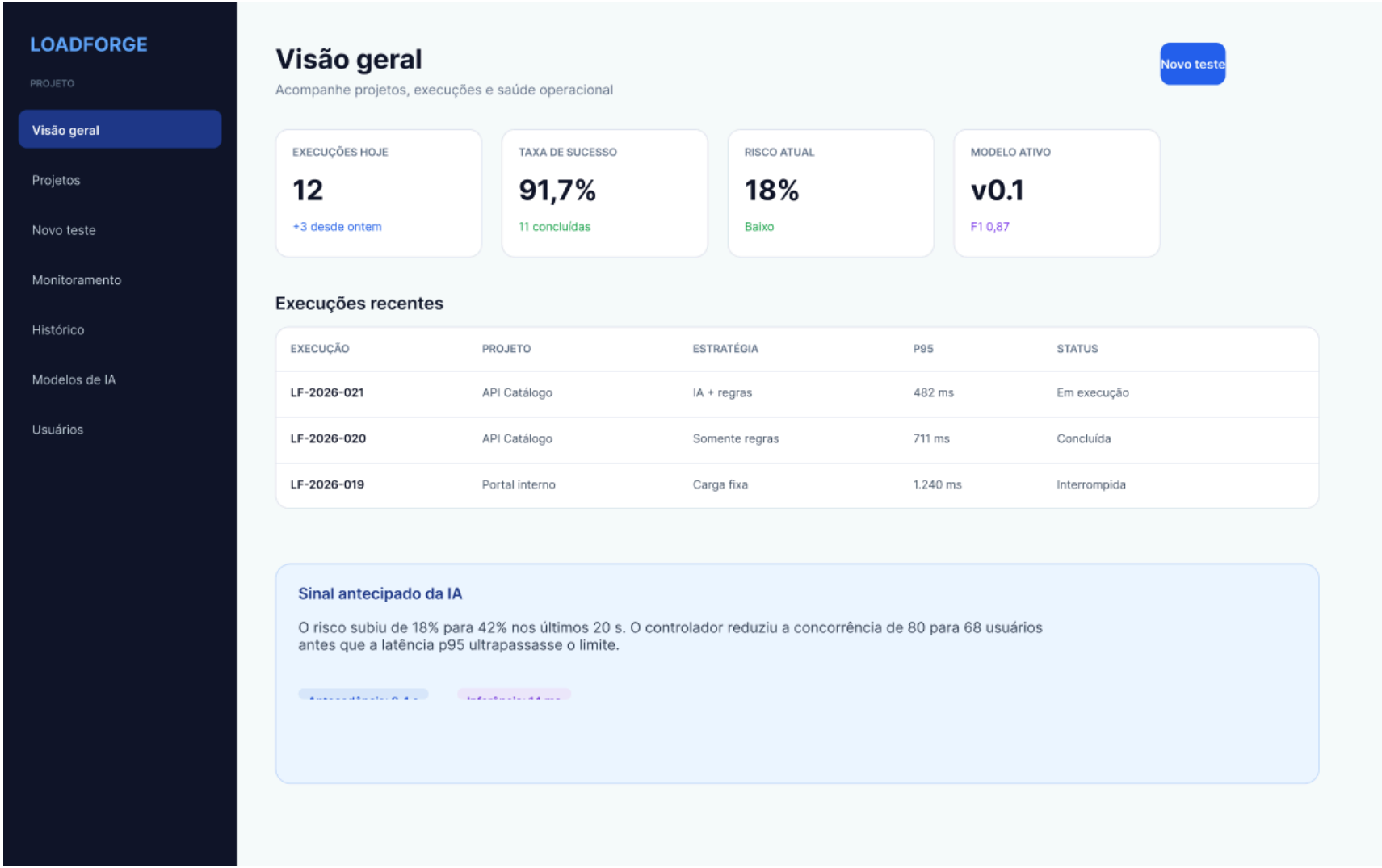


Figura 5 — Protótipo da tela Dashboard.

16.2 Novo cenário

Carga, limites de segurança e estratégia aparecem no mesmo formulário. A autorização do alvo recebe destaque porque nenhuma execução deve ocorrer sem confirmação prévia.



Figura 6 — Protótipo da tela Novo cenário.

16.3 Monitoramento

Métricas observadas, risco previsto e decisões do controlador compartilham o mesmo contexto. Essa organização relaciona cada ajuste de concorrência aos sinais que o motivaram e mantém a parada imediata acessível.

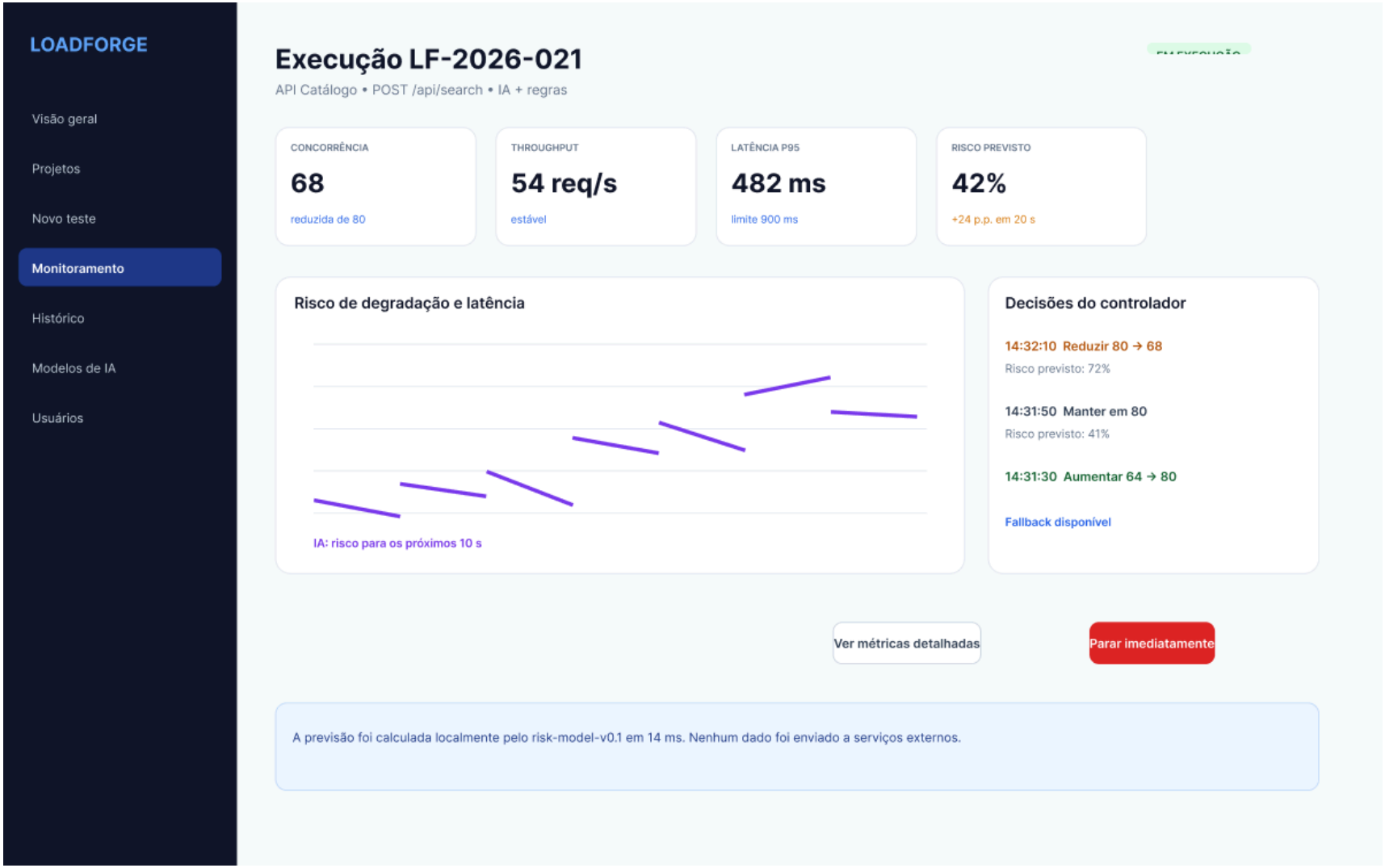


Figura 7 — Protótipo da tela Monitoramento.

16.4 Histórico

A lista permite localizar execuções por estratégia e resultado e acessar a comparação entre os modos fixo, por regras e assistido por IA.

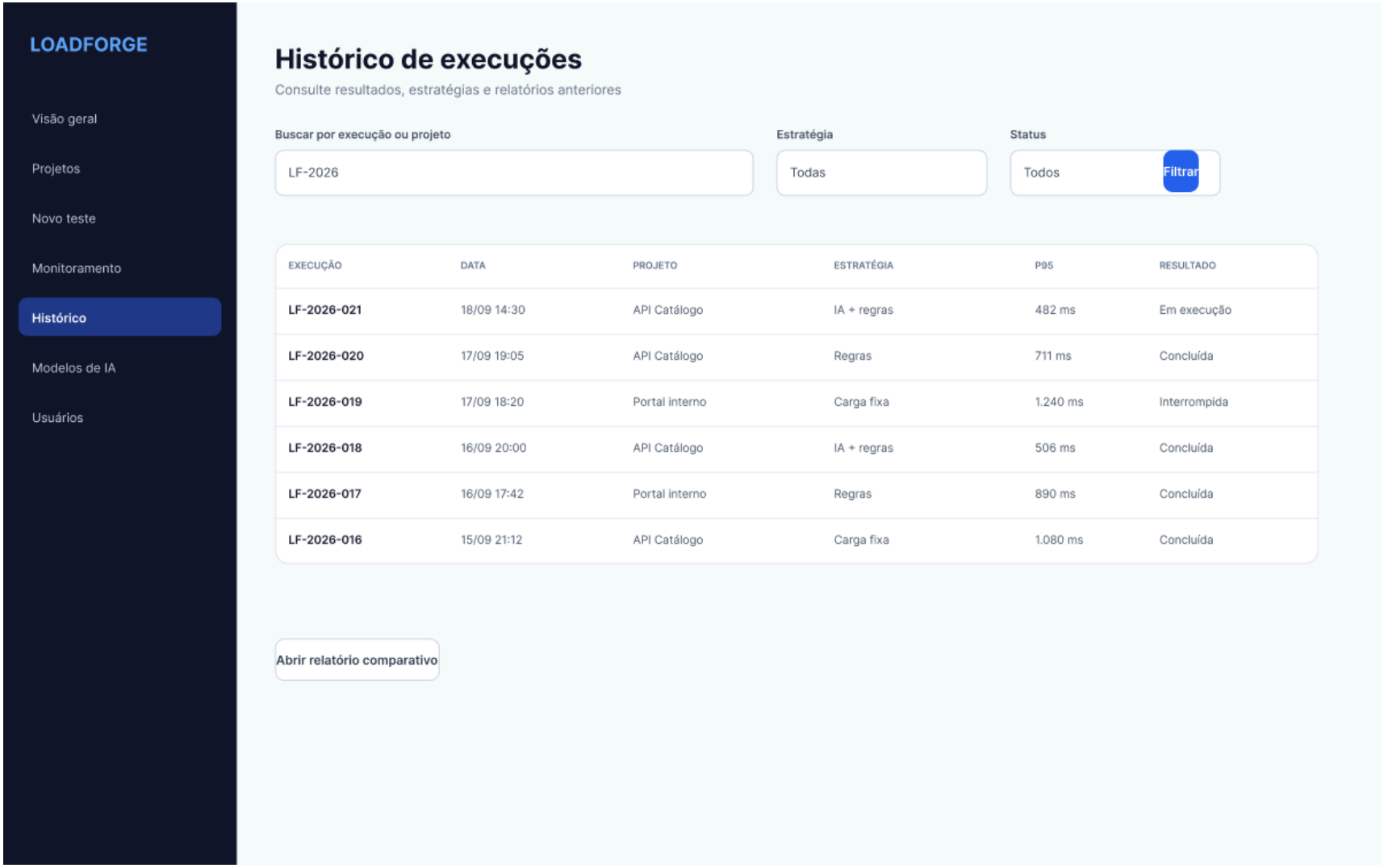


Figura 8 — Protótipo da tela Histórico.

17 Banco de Dados Criado

A estrutura inicial foi implementada para PostgreSQL 16 com SQLAlchemy e Alembic. O repositório contém o esquema SQL de referência, dados iniciais de desenvolvimento e uma migração reversível. As dez tabelas correspondem ao modelo relacional, incluindo as mesmas chaves estrangeiras, relações opcionais e restrições únicas.

Tabela	Finalidade
users	Usuários, senha derivada e papel de acesso.
projects	Projetos de teste e responsável.
endpoints	Alvos autorizados, método, limites e evidência.
test_scenarios	Configuração de carga e limites de segurança.
model_versions	Versões do modelo, algoritmo, artefato e métricas.
test_executions	Execuções, estratégia ativa, status e motivo de interrupção.
metric_windows	Métricas agregadas em janelas de dois segundos.
risk_predictions	Probabilidade, horizonte e latência de inferência.
control_decisions	Ação, concorrência anterior/nova e justificativa.
execution_reports	Resumo consolidado e comparação da execução.

18 Projeto Estruturado no GitHub

Repositório: github.com/camimcl/sre-fabrica-de-software

O repositório foi organizado como monorepo, com separação entre backend, futura interface, banco de dados, diagramas, protótipos e decisões arquiteturais. Essa divisão mantém no mesmo projeto os artefatos que evoluem juntos, enquanto cada pasta possui uma responsabilidade identificável. O README concentra a preparação do ambiente e as orientações operacionais.

Relatório técnico no repositório: docs/relatorio-sprint-02.md

Implementação da Sprint 03

Banco de dados, autenticação, perfis, CRUD e execução local

A Sprint 03 transforma a base arquitetural e o esquema criados nas etapas anteriores em um fluxo executável. O frontend React consome a API FastAPI, a API aplica autenticação e regras de negócio, e o PostgreSQL persiste usuários, projetos e endpoints. A entrega também inclui Docker Compose, migrations automáticas, documentação de execução e testes dos fluxos críticos.

19 Estrutura Implementada

A aplicação continua organizada como monólito modular, agora com três serviços executáveis e integrados. A separação entre frontend, API e banco mantém as responsabilidades claras e permite demonstrar o sistema completo em um único computador.

Camada	Implementação	Responsabilidade executável
Frontend	React, Vite, TypeScript e Nginx	Login, cadastro, perfis, usuários, projetos e endpoints.
API	Python 3.11 e FastAPI	Autenticação, autorização, validações e operações persistidas.
Persistência	PostgreSQL 16, SQLAlchemy e Alembic	Armazenamento relacional e evolução controlada do esquema.
Ambiente	Docker Compose	Inicialização coordenada de banco, API e frontend.
Testes	pytest e build TypeScript	Validação dos fluxos, permissões, configuração e compilação da interface.

19.1 Fluxo de inicialização

O serviço da API aguarda o PostgreSQL ficar saudável, executa alembic upgrade head e inicia o Uvicorn. O frontend é disponibilizado somente após a API responder ao health check. Essa ordem reduz falhas de inicialização e garante que a interface seja apresentada com o banco disponível.

19.2 Configuração e segurança

- Senhas são derivadas com PBKDF2 antes da persistência.
- Tokens de acesso são assinados, possuem expiração e são verificados nas rotas protegidas.
- Segredos permanecem no arquivo .env local, ignorado pelo Git.
- A API rejeita campos inesperados, e-mails inválidos, URLs inválidas e métodos HTTP não permitidos.
- A propriedade do projeto é validada antes de qualquer alteração em projetos e endpoints.

20 Evidências do Banco de Dados Conectado

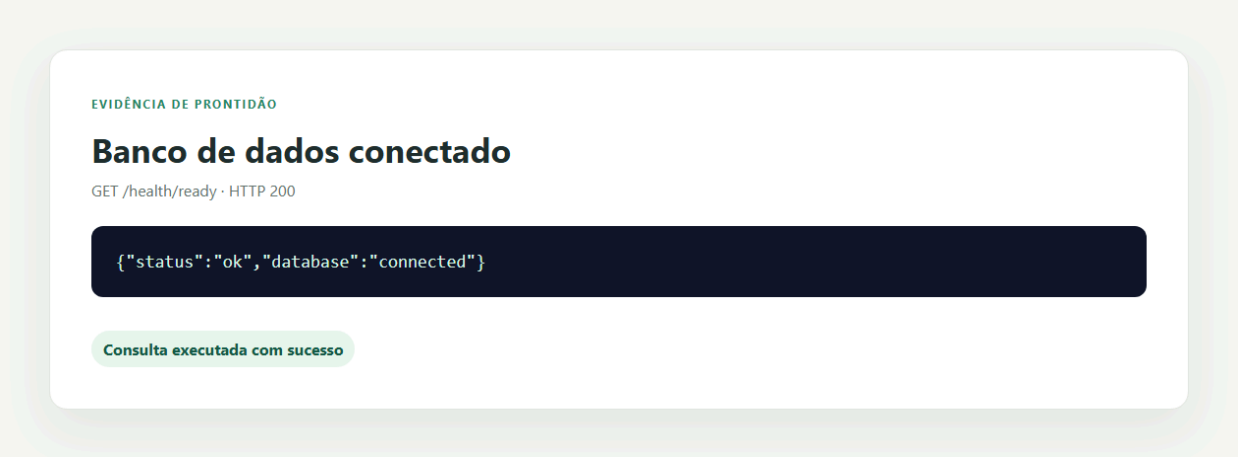
A comunicação com o banco é verificada por uma rota pública de prontidão que executa uma consulta real antes de declarar o serviço disponível. Durante a validação local, a aplicação respondeu com HTTP 200 e informou database connected.

Verificação	Evidência técnica	Resultado esperado e observado
Prontidão da API	GET /health/ready	HTTP 200 com {"status":"ok","database":"connected"}.
Ordem de inicialização	Health check do PostgreSQL no Docker Compose	API inicia após o banco aceitar conexões.
Evolução do esquema	alembic upgrade head	Migration aplicada automaticamente antes do Uvicorn.
Persistência	Volume loadforge_pgdata	Dados preservados após docker compose down e nova inicialização.
Conexão resiliente	SQLAlchemy com pool_pre_ping	Conexões são verificadas antes do uso.

20.1 Configuração do banco

Variável	Finalidade
POSTGRES_DB	Nome do banco local.
POSTGRES_USER	Usuário utilizado pela aplicação.
POSTGRES_PASSWORD	Senha local definida fora do repositório.
POSTGRES_PORT	Porta publicada somente em 127.0.0.1; pode ser alterada em caso de conflito.
DATABASE_URL	Usada quando o backend é executado fora do Docker Compose.

No ambiente de demonstração foi identificado conflito com a porta 5432. A publicação externa do PostgreSQL foi alterada para 15432 no arquivo .env, sem modificar a porta interna usada pela API.



21 Evidências do Login Cadastro e Controle de Perfis

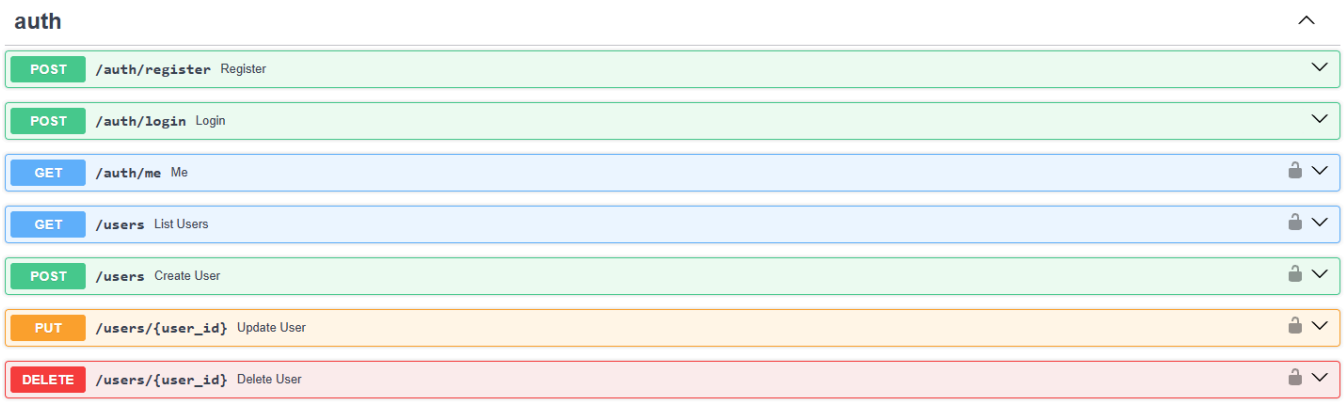
A autenticação está integrada ao PostgreSQL. O cadastro cria o usuário no banco, o login compara a senha informada com o hash persistido e o token identifica a conta nas requisições seguintes. As permissões são aplicadas pela API e refletidas na interface.

Fluxo	Evidência	Comportamento validado
Cadastro público	POST /auth/register	Cria somente contas Visualizador e rejeita e-mail duplicado.
Login	POST /auth/login	Retorna token para credenciais válidas e HTTP 401 para senha incorreta.
Sessão	GET /auth/me	Retorna nome, e-mail e perfil da conta autenticada.
Criação administrativa	POST /users	QA cria contas QA ou Visualizador.
Atualização de perfil	PUT /users/{id}	Usuário altera a própria conta; QA também altera terceiros.
Exclusão de perfil	DELETE /users/{id}	Exclui contas sem dependências e encerra a sessão na autoexclusão.

21.1 Matriz de permissões

Ação	QA	Visualizador
Consultar projetos e endpoints	Permitido	Permitido
Criar projeto	Permitido	Bloqueado
Editar projeto	Somente se for proprietário	Bloqueado
Gerenciar endpoints	Somente nos próprios projetos	Bloqueado
Listar todos os usuários	Permitido	Bloqueado
Criar usuários	Permitido	Bloqueado
Editar ou excluir a própria conta	Permitido	Permitido
Editar ou excluir terceiros	Permitido	Bloqueado
Alterar perfil de acesso	Permitido	Bloqueado

O sistema preserva ao menos uma conta QA. O último QA não pode ser excluído nem convertido para Visualizador. A exclusão também é recusada quando o usuário ainda possui projetos ou execuções vinculadas.



22 Evidências do CRUD Principal Funcionando

O CRUD principal foi implementado para projetos e endpoints, com persistência real no PostgreSQL. O gerenciamento de usuários também passou a contemplar criação, consulta, atualização e exclusão.

Entidade	Operações implementadas	Regras verificadas
Usuários	Criar, listar, atualizar e excluir	Perfil, autoatendimento, último QA, e-mail único e dependências.
Projetos	Criar, listar, consultar, atualizar e excluir	Somente QA cria; somente proprietário altera; nome único por proprietário.
Endpoints	Criar, listar, consultar, atualizar e excluir	Propriedade do projeto, URL válida, método permitido e autorização.

22.1 Evidência do CRUD de projetos

Operação	Rota	Evidência funcional
Cadastrar	POST /projects	Registro criado com owner_id do QA autenticado.
Consultar	GET /projects e GET /projects/{id}	Dados persistidos disponíveis para usuários autenticados.
Atualizar	PUT /projects/{id}	Nome e descrição alterados somente pelo proprietário.
Excluir	DELETE /projects/{id}	Exclusão concluída quando não existem dependências.

22.2 Evidência do CRUD de endpoints

Operação	Rota	Evidência funcional
Cadastrar	POST /projects/{id}/endpoints	Nome, URL, método e autorização persistidos.
Consultar	GET /projects/{id}/endpoints	Lista vinculada ao projeto selecionado.
Atualizar	PUT /projects/{id}/endpoints/{endpoint_id}	Alterações restritas ao QA proprietário.
Excluir	DELETE /projects/{id}/endpoints/{endpoint_id}	Registro removido após confirmação.

22.3 Testes automatizados

Validação	Comando	Resultado
Backend	python -m pytest -q -c backend/pyproject.toml	22 testes aprovados e 1 integração ignorada por ausência do banco _test.
Frontend	npm run build	TypeScript validado e build de produção concluído.
Formatação Git	git diff --check	Nenhum erro de espaços ou conflitos de patch.

auth

- POST `/auth/register` Register
- POST `/auth/login` Login
- GET `/auth/me` Me
- GET `/users` List Users
- POST `/users` Create User
- PUT `/users/{user_id}` Update User
- DELETE `/users/{user_id}` Delete User

projects

- GET `/projects` List Projects
- POST `/projects` Create Project
- GET `/projects/{project_id}` Get Project
- PUT `/projects/{project_id}` Update Project
- DELETE `/projects/{project_id}` Delete Project
- GET `/projects/{project_id}/endpoints` List Endpoints
- POST `/projects/{project_id}/endpoints` Create Endpoint
- GET `/projects/{project_id}/endpoints/{endpoint_id}` Get Endpoint
- PUT `/projects/{project_id}/endpoints/{endpoint_id}` Update Endpoint
- DELETE `/projects/{project_id}/endpoints/{endpoint_id}` Delete Endpoint

health

- GET `/health/live` Live
- GET `/health/ready` Ready

23 Procedimento de Execução Local

O ambiente recomendado usa Docker Desktop com Docker Compose v2. Dessa forma, não é necessário instalar PostgreSQL, Python ou Node.js separadamente.

23.1 Preparar as variáveis

No diretório do projeto, copie .env.example para .env e defina valores locais fortes para POSTGRES_PASSWORD e LOADFORGE_TOKEN_SECRET.

Etapa	Comando no PowerShell
Abrir o projeto	cd D:\Projetos\LoadForge
Criar a configuração	Copy-Item .env.example .env
Editar a configuração	notepad .env

23.2 Iniciar os serviços

Finalidade	Comando
Construir e iniciar	docker compose up --build -d
Verificar os serviços	docker compose ps
Acompanhar registros	docker compose logs -f

23.3 Criar a primeira conta QA

Execute: docker compose exec api python -m app.cli create-qa --name "Nome do QA" --email "qa@exemplo.com". A senha é solicitada de forma interativa e deve possuir pelo menos 12 caracteres.

23.4 Acessar e validar

Recurso	Endereço
Interface	http://127.0.0.1:5173
Documentação da API	http://127.0.0.1:8000/docs
Banco e API	http://127.0.0.1:8000/health/ready

Para encerrar sem apagar os dados, execute docker compose down. O volume do PostgreSQL permanece disponível para a próxima inicialização. O comando docker compose down -v deve ser usado somente quando a exclusão definitiva dos dados locais for intencional.

24 Dificuldades Encontradas e Próximos Passos

24.1 Dificuldades encontradas

Dificuldade	Tratamento adotado	Aprendizado
Ordem entre banco e API	Health check, espera ativa e migrations antes do Uvicorn.	A inicialização precisa refletir dependências reais dos serviços.
Regras de perfis	Validação centralizada no backend e controles equivalentes na interface.	Ocultar botões não substitui autorização na API.
Exclusão de usuários	Bloqueio do último QA e de contas com dados vinculados.	Integridade e continuidade administrativa devem anteceder a exclusão.
Teste PostgreSQL isolado	Teste de integração condicionado a banco cujo nome termina em _test.	A automação completa requer provisionamento seguro de banco descartável.

24.2 Próximos passos

• Implementar cadastro e validação de cenários de carga.

• Construir o motor assíncrono HTTP com duração, concorrência, ramp-up, timeout e cancelamento.

• Coletar throughput, latências, erros, timeouts, CPU e memória em janelas temporais.

• Formar e documentar o conjunto de dados das execuções controladas.

• Treinar e comparar regressão logística e Random Forest para prever degradação.

• Integrar o modelo selecionado ao controlador adaptativo com fallback por regras.

• Criar monitoramento, histórico e relatórios comparativos das estratégias.

• Executar o teste de integração PostgreSQL em pipeline com banco descartável.

Ao final da Sprint 03, a base operacional necessária para as próximas funcionalidades está concluída: ambiente reproduzível, banco acessível, autenticação persistida, perfis protegidos e CRUDs reais de usuários, projetos e endpoints.

25 Repositório GitHub Atualizado

O código da Sprint 03 foi organizado na branch Sprint-03, preservando a main para o fluxo de revisão por Pull Request.

Repositório principal: https://github.com/camimcl/sre-fabrica-de-software

Pull Request da Sprint 03: https://github.com/camimcl/sre-fabrica-de-software/pull/1

Branch Sprint-03: https://github.com/itswall/sre-fabrica-de-software/tree/Sprint-03

A branch reúne sete commits organizados por API, frontend, ambiente local, documentação e preparação da versão 0.3.0.

Implementação da Sprint 04

Cenários, execuções e motor de geração de carga

A Sprint 04 entrega o primeiro módulo completo de testes de carga do LoadForge. A API agora permite configurar cenários vinculados a endpoints autorizados, criar e controlar execuções, gerar carga HTTP concorrente e persistir métricas por janela temporal. A implementação mantém as regras de acesso da Sprint 03 e acrescenta validações, mensagens de erro tratadas e testes integrados do fluxo.

26 Módulo de Testes de Carga Implementado

O módulo load_tests foi organizado em duas partes complementares: gerenciamento persistido de cenários e execuções, e um motor assíncrono de geração de carga real. O conjunto fornece a base operacional para o monitoramento, o controle adaptativo e os relatórios previstos nas próximas sprints.

Componente	Funcionalidades entregues	Resultado
Cenários	CRUD vinculado ao projeto e ao endpoint autorizado.	Configuração reutilizável de duração, concorrência, ramp-up, timeout, estratégia e limites.
Execuções	Criação, consulta e máquina de estados persistida.	Ciclo PENDING, RUNNING, COMPLETED, CANCELLED ou FAILED.
Motor de carga	Requisições HTTP concorrentes com asyncio e httpx.	Geração real de carga sem bloquear a API.
Métricas	Agregação e gravação em janelas temporais.	Throughput, p50, p95, p99, sucessos, erros e timeouts.
Segurança	Revalidação da autorização antes de iniciar.	Carga limitada a alvos próprios ou expressamente autorizados.
Parada de emergência	Cancelamento do disparo de novas requisições.	Dados já coletados permanecem disponíveis para auditoria.

26.1 Cenários e ciclo de vida

- O cenário copia limites de segurança e parâmetros operacionais validados pela API.
- A execução registra um snapshot dos parâmetros para preservar o histórico mesmo após mudanças no cenário.
- Somente o QA proprietário do projeto pode criar, alterar ou operar execuções; usuários autenticados podem consultar.

26.2 Motor assíncrono e métricas

- O motor calcula a concorrência de cada janela sem ultrapassar o máximo configurado.
- As requisições são agregadas em janelas com contadores, latências percentis, throughput e taxa de erro.
- A persistência síncrona existente é executada fora do event loop para não bloquear a geração de carga.

27 Evidências do Módulo Funcionando

A validação integrada executou a API e o motor reais contra um servidor HTTP local autorizado e descartável. O fluxo criou cenário e execução, iniciou o motor, aguardou a conclusão e consultou as métricas persistidas.



Figura 9 — Fluxo integrado concluído com carga real e métricas coletadas.

27.1 Testes automatizados

A suíte atual foi executada sobre o código incorporado pelo Pull Request 2. Foram aprovados 44 testes e dois testes PostgreSQL foram ignorados porque a variável LOADFORGE_TEST_DATABASE_URL não estava configurada com um banco descartável terminado em _test.



Figura 10 — Resultado da suíte automatizada do código atual.

28 Evidências da Persistência de Dados

Depois da conclusão, a conexão usada pelo fluxo foi encerrada e uma nova conexão abriu o mesmo banco de validação. O cenário, a execução concluída e a janela métrica foram recuperados com os valores produzidos pelo motor.



Figura 11 — Registros recuperados após reabrir o banco de validação.

Entidade	Persistência implementada	Evidência consultável
test_scenarios	Configuração completa e vínculo com projeto, endpoint e criador.	GET /projects/{id}/scenarios/{scenario_id}
test_executions	Snapshot, estado, horários e motivo de cancelamento.	GET .../executions/{execution_id}
metric_windows	Métricas agregadas de cada janela da execução.	GET .../executions/{execution_id}/metric-windows

A evidência isolada usa SQLite para permitir repetição determinística sem alterar dados locais. Em execução pelo Docker Compose, o mesmo mapeamento SQLAlchemy persiste no PostgreSQL e no volume loadforge_pgdata. O repositório também contém um teste PostgreSQL condicional para banco descartável.

29 Validações Implementadas e Mensagens de Erro

29.1 Exemplos de validações

Regra	Validação	Resposta esperada
Nome do cenário	Obrigatório, sem espaços isolados e até 120 caracteres.	HTTP 422 com identificação do campo.
Concorrência	initial_concurrency deve ser menor ou igual a max_concurrency.	HTTP 422 com mensagem específica.
Valores mínimos	Duração, timeout, p95 e concorrências devem ser positivos; ramp-up aceita zero.	HTTP 422 quando fora do limite.
Taxa de erro	error_rate_limit deve permanecer entre 0 e 1.	HTTP 422 para valor inválido.
Autorização	A execução exige reconhecimento e o endpoint precisa continuar autorizado.	HTTP 409 quando a regra de segurança falha.
Cancelamento	A parada de emergência exige motivo não vazio.	HTTP 422 para motivo ausente.



Figura 12 — Respostas observadas para concorrência inválida e alvo sem autorização.

29.2 Mensagens de erro tratadas

Situação	Código	Mensagem principal
Credencial ausente ou expirada	401	Invalid or expired credentials
Usuário sem perfil QA	403	QA role required
Projeto de outro proprietário	403	Project owner required
Recurso inexistente	404	Mensagem específica com not found
Endpoint não autorizado	409	The target endpoint must have confirmed authorization before scenarios can target it
Transição de estado inválida	409	Invalid execution state transition
Reconhecimento de autorização ausente	409	authorization_acknowledged must be true to start an execution

Situação	Código	Mensagem principal
Cenário com execuções dependentes	409	Remove dependent executions before deleting the scenario

30 Fluxo de Navegação Entre as Telas

O fluxo previsto parte do login e da seleção de um endpoint autorizado, passa pela configuração do cenário e inicia a execução. Durante o teste, a tela de monitoramento consulta o estado e as janelas métricas por polling; ao final, os dados permanecem disponíveis no histórico.

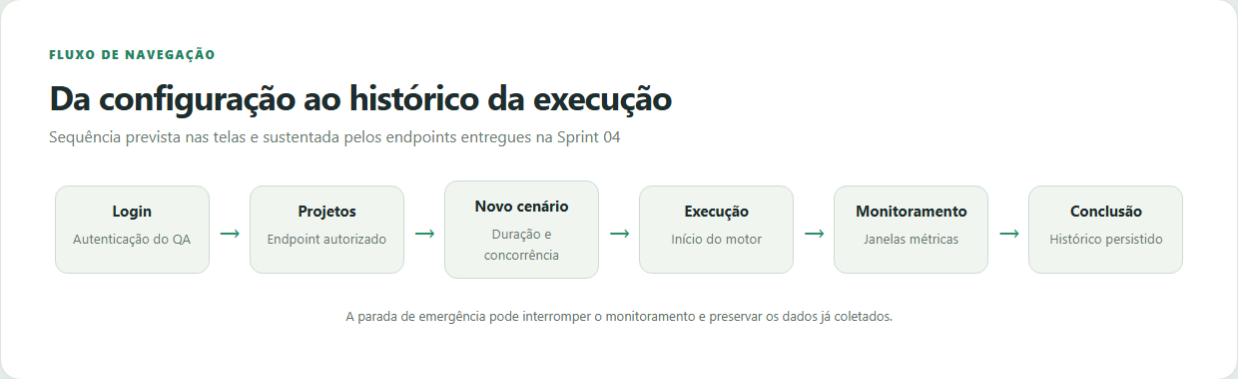


Figura 13 — Navegação prevista entre configuração execução monitoramento e histórico.

Tela	Ação principal	Suporte entregue
Login	Autenticar o QA.	POST /auth/login
Projetos e endpoints	Selecionar um alvo autorizado.	GET /projects e endpoints
Novo cenário	Definir duração, concorrência e limites.	CRUD /projects/{id}/scenarios
Execução	Confirmar autorização e iniciar.	POST executions e start
Monitoramento	Acompanhar estado e métricas.	GET execution e metric-windows
Parada de emergência	Cancelar com motivo.	POST cancel
Histórico	Consultar execuções e janelas persistidas.	GET executions e metric-windows

A Sprint 04 entrega a API necessária para esse percurso. A integração das telas React de cenário, monitoramento, parada e histórico com os novos endpoints permanece registrada como próximo passo.

31 Repositório GitHub Atualizado

O desenvolvimento da Sprint 04 foi organizado na branch sprint-4 e incorporado à main pelo Pull Request 2, com commits separados por schemas, API, motor, testes, correções e documentação.

Repositório principal: github.com/camimcl/sre-fabrica-de-software

Pull Request da Sprint 04: github.com/camimcl/sre-fabrica-de-software/pull/2

Branch da Sprint 04: github.com/camimcl/sre-fabrica-de-software/tree/sprint-4

32 Dificuldades Encontradas e Próximos Passos

32.1 Dificuldades encontradas

Dificuldade	Tratamento adotado	Resultado
Motor assíncrono com banco síncrono	Persistência executada com asyncio.to_thread.	O event loop permanece disponível para gerar carga.
Transições concorrentes	O motor passou a controlar o término das execuções que iniciou.	Transições manuais são bloqueadas enquanto o motor está ativo.
Erros e timeouts determinísticos	Servidor HTTP local descartável nos testes.	Cenários de sucesso, erro, timeout e cancelamento são reproduzíveis.
Consistência do ramp-up	API, banco e migration foram alinhados para aceitar zero.	Cenários de concorrência fixa passam pela validação e persistência.
Teste PostgreSQL seguro	Execução condicionada a banco terminado em _test.	Dados reais ficam protegidos contra limpeza acidental.

32.2 Próximos passos

• Integrar as telas React de cenário, execução, monitoramento, parada de emergência e histórico.

• Conectar o modelo local de risco e o controlador adaptativo ao intervalo entre janelas.

• Gerar relatórios comparativos entre estratégia fixa, regras e IA híbrida.

• Executar a suíte de integração em PostgreSQL descartável no pipeline.

• Aprimorar métricas de CPU e memória do alvo quando houver agente ou telemetria disponível.

Ao final da Sprint 04, o LoadForge possui um fluxo de carga executável e auditável: configura cenários autorizados, dispara requisições reais, controla o ciclo de vida, preserva métricas e comunica falhas de forma tratada. Essa base permite avançar para o controlador inteligente e a experiência completa de monitoramento no frontend.

Implementação da Sprint 05

Inteligência artificial local e controle adaptativo

A Sprint 05 integra o segundo módulo funcional do LoadForge ao motor de testes de carga: preparação de dados, treinamento e inferência de um modelo local de risco, com ajuste da concorrência e persistência das decisões. O fluxo utiliza os cenários, as execuções e as métricas construídas na Sprint 04, mantendo autenticação, autorização do alvo e consulta ao histórico.

A verificação de 03/10/2026 reuniu testes automatizados, navegação na interface e um experimento integrado em PostgreSQL. As evidências desta seção relacionam os resultados às execuções, ao modelo treinado e aos registros recuperados após reiniciar a API.

33 Segundo Módulo Implementado

O módulo intelligence concentra o aprendizado de máquina e o versionamento. O módulo control recebe o risco previsto e as métricas observadas para decidir a carga da próxima janela. Essa separação permite avaliar a IA sem acoplá-la ao transporte HTTP e mantém o controle por regras disponível quando o modelo não pode ser utilizado.

Componente	Funcionalidade implementada	Integração com o sistema
Preparação dos dados	Atributos e rótulos temporais a partir de execuções concluídas.	Leitura das janelas persistidas e dos limites da execução.
Treinamento local	Comparação de regressão logística e Random Forest.	Registro de candidato, métricas e artefato local versionado.
Aprovação e inferência	Validação do artefato e uso da versão aprovada.	Associação do modelo à execução híbrida e previsão por janela.
Controle adaptativo	Aumento, manutenção ou redução da concorrência.	Aplicação entre janelas e registro da justificativa.
Interface funcional	Cenários, início, parada, monitor e modelos locais.	Consumo das rotas reais da API, sem depender do protótipo.

33.1 Continuidade do planejamento

A Sprint 03 consolidou os cadastros e perfis, e a Sprint 04 disponibilizou a geração de carga. A antecipação da IA e do controle para esta etapa reúne essas capacidades em um fluxo demonstrável. A comparação experimental entre estratégias e a medição de ganhos permanecem como evolução do produto; não foram substituídas por resultados simulados.

33.2 Pipeline de inteligência artificial

O processamento é realizado localmente em Python com scikit-learn. A implementação própria abrange a extração de características, a rotulagem temporal, a separação dos dados, a seleção do algoritmo, a aprovação da versão e a integração da inferência ao motor. Não há chatbot nem consumo de API externa de inteligência artificial.

As características incluem concorrência, throughput, percentis de latência, erros, timeouts e tendências entre janelas. O rótulo indica violação do limite de p95 ou da taxa de erro nas cinco janelas futuras, nominalmente dez segundos. Requisições em voo podem prolongar esse intervalo: o contrato é em janelas, não dez segundos exatos. As novas versões registram a duração nominal e os extremos observados; os timestamps representam o fim de cada observação.

Etapa	Tratamento implementado	Motivo da decisão
Seleção das amostras	Execuções COMPLETED e sequências de janelas com dados utilizáveis.	Excluir coleta em andamento e sequências incompletas do treinamento.
Divisão cronológica	Treino, validação e teste em proporção inicial 60/20/20.	Reservar o teste final para avaliação, sem usá-lo na seleção.
Fronteiras temporais	Exclusão de rótulos cujo horizonte alcança a partição seguinte.	Reduzir vazamento de informação futura entre conjuntos.
Seleção do candidato	Comparação por F1, com recall e precisão como desempate.	Avaliar o equilíbrio entre identificar degradação e evitar alarmes indevidos.
Rastreabilidade	Versão, hashes, contagens e métricas do teste final.	Identificar o conjunto e o artefato usados em cada treinamento.

33.3 Integração com o processamento e a otimização

Após persistir uma janela, o motor prepara as características, obtém o risco da versão aprovada e executa o controlador. A decisão registra a concorrência anterior, a próxima concorrência e o motivo. O novo valor orienta a janela seguinte, respeitando o limite do cenário. Sem modelo válido ou em caso de falha de inferência, o fluxo utiliza regras e identifica o fallback.

A estratégia FIXED usa a progressão de carga configurada; RULES reage às métricas atuais; AI_HYBRID acrescenta o risco previsto. O processamento assíncrono e o controle da concorrência constituem a vertente de otimização. Ganhos de throughput, p95, erro e consumo de recursos deverão ser medidos em condições equivalentes, e não inferidos apenas pela existência do algoritmo.

A versão candidata registra precisão, recall, F1, acurácia e falsos positivos. Cada previsão registra também a latência de inferência. As metas de qualidade definidas no planejamento permanecem referências de avaliação, não resultados já atingidos.

34 Integração com o Banco de Dados

O segundo módulo usa as entidades relacionais previstas na modelagem, com SQLAlchemy e evolução do esquema por Alembic. O banco registra metadados e vínculos auditáveis; o arquivo do modelo permanece em armazenamento local persistente. Essa escolha evita armazenar o objeto serializado em cada execução e permite identificar a versão utilizada.

Entidade	Informações persistidas	Operação disponibilizada
model_versions	Algoritmo, estado, métricas, amostras, hashes e notas do treinamento.	Inserção de candidato, consulta, aprovação e aposentadoria.
test_executions	Snapshot dos parâmetros, estado e vínculo com o modelo.	Criação, consulta e transições do ciclo de execução.
metric_windows	Contadores, concorrência, throughput, latências e erros.	Gravação durante a carga e consulta ordenada das janelas.
risk_predictions	Probabilidade, classe prevista, modelo e tempo de inferência.	Gravação e recuperação da previsão de cada janela.
control_decisions	Estratégia, ação, concorrência anterior e posterior, justificativa.	Gravação e consulta da decisão e da previsão associada, quando houver.

34.1 Evolução do esquema e integridade

A migration 20261001_0003 acrescenta o ramp-up ao snapshot da execução e os metadados do modelo. A migration 20261001_0004 acrescenta o índice único parcial de aprovação; se houver mais de uma versão aprovada na base, conserva a mais recente e aposenta as demais. As quatro migrations foram aplicadas em PostgreSQL 16, até a revisão 20261001_0004, e os testes de integração foram aprovados.

As chaves estrangeiras vinculam previsões às janelas e aos modelos, e decisões às janelas e às previsões opcionais. Restrições únicas evitam múltiplos registros de previsão ou decisão para a mesma janela. A aprovação aposenta a versão anterior, preservando os vínculos das execuções históricas.

34.2 Consulta e recuperação das informações

As rotas da API disponibilizam modelos, treinamento e aprovação, além das previsões e decisões de cada execução. Após reiniciar a API, a consulta recuperou a mesma execução híbrida concluída, com identificadores e valores de todas as janelas iguais aos registros anteriores. A versão aprovada também continuou disponível.

Registro verificado	Identificação	Resultado observado
Execução híbrida	a45e7cef-db67-4ba4-a903-507ca824b2b5	COMPLETED antes e depois do reinício.
Versão do modelo	27d485d8-d171-4596-837a-8d40d825ad05	APPROVED; associação preservada na execução.
Janelas e vínculos	4 métricas / 4 previsões / 4 decisões	Cada previsão pertence ao modelo; cada decisão aponta para a previsão de sua janela.

35 Regras de Negócio Atualizadas

As regras anteriores de perfil, propriedade do projeto, confirmação de autorização e ciclo de vida continuam aplicáveis. As novas regras definem quando os dados podem treinar um modelo, como uma versão se torna ativa e como o risco altera a geração de carga.

Regra	Comportamento implementado	Justificativa
Base de treinamento	Mínimo de 20 amostras e presença das duas classes nas três partições após os filtros.	Recusar treinamento sem dados suficientes para aprender e avaliar.
Aprovação única	CANDIDATE passa a APPROVED; o aprovado anterior passa a RETIRED.	Evitar ambiguidade sobre a versão usada nas próximas execuções.
Integridade do artefato	Caminho restrito, SHA-256, versão e esquema de atributos verificados.	Detectar troca ou incompatibilidade do arquivo local esperado.
Redução da carga	Risco a partir de 0,70 ou limite observado atingido reduz a concorrência.	Reagir a degradação prevista ou já observada, sem ficar abaixo de 1.
Manutenção da carga	Risco a partir de 0,40 ou métrica a 80% do limite mantém a concorrência.	Evitar aumentar a pressão quando há sinal de aproximação do limite.
Aumento da carga	Métricas estáveis e baixo risco permitem aumento até o máximo configurado.	Aproveitar capacidade disponível sem ultrapassar o teto.
Continuidade sem IA	Ausência de modelo válido ou falha de inferência ativa regras com justificativa.	Não depender exclusivamente da disponibilidade do modelo.
Limites operacionais	Até 3.600 s, concorrência 500 e timeout de 60.000 ms.	Restringir o uso de recursos do gerador.

35.1 Decisões em relação ao planejamento

A rotulagem operacional adotou a violação de p95 ou da taxa de erro; throughput permanece uma característica de entrada e métrica de comparação. Essa definição torna o alvo de aprendizado explícito e verificável. A queda de throughput citada no planejamento inicial não foi implementada como condição adicional de rótulo nesta versão.

O treinamento separa validação e teste final para que a escolha do algoritmo não use os mesmos dados empregados no resultado final. Modelos permanecem globais ao sistema e podem ser treinados e aprovados por QA; não há isolamento de modelos por projeto nem arquitetura multi-tenant.

O hash confere integridade em relação ao registro esperado; não torna seguro qualquer arquivo joblib recebido de terceiros. O módulo carrega artefatos produzidos no ambiente controlado, e a política de acesso ao armazenamento continua necessária.

36 Testes das Funcionalidades

A suíte final de 03/10/2026 apresentou 64 testes aprovados, sem falhas e sem testes ignorados, em 46,15 segundos. O PostgreSQL de teste foi isolado da demonstração. Os testes incluem históricos compatíveis e rejeição preventiva dos incompatíveis. O build de produção foi concluído no Docker; a navegação utilizou serviços reais, incluindo consultas atrasadas e polling lento.

Verificação	Cobertura	Resultado registrado
Backend completo	Autenticação, cadastros, cenários, motor, persistência, IA e controle.	64 aprovados; zero falhas e zero ignorados.
Pipeline de IA	Rótulos, partições cronológicas, treino, aprovação e inferência.	Testes automatizados aprovados com dados de teste.
Integridade e validações	Artefato adulterado, dados insuficientes e coleta em andamento.	Rejeições cobertas pelos testes automatizados.
Controle integrado	Regras, caminho híbrido, decisões persistidas e fallback.	Aprovado na suíte e no experimento HTTP local.
Interface funcional	Login, cenário, autorização, execução, monitor e modelos.	Build concluído e navegação exercitada com API real.
PostgreSQL e migrations	Esquema real, vínculos e recuperação após reinício.	Revisão 20261001_0004 aplicada; dados recuperados.

36.1 Resultados do fluxo integrado

O experimento usou um alvo HTTP descartável da equipe, com respostas saudáveis, HTTP 503 e atrasos controlados. A coleta FIXED de 480 segundos produziu as métricas utilizadas para treinar o modelo; nenhuma carga foi direcionada a sistemas externos.

Fluxo de teste	Resultado observado	Registro
Execução sem modelo	Continuidade por RULES, sem previsão, com justificativa de fallback.	Aprovado.
Treino e aprovação	Dados insuficientes retornaram 409; após a coleta, candidato criado e aprovado.	Aprovado.
Execução híbrida	Previsões e decisões vinculadas ao modelo e às respectivas janelas.	Aprovado.
Erros e timeouts	HTTP 503 provocou redução; respostas lentas incrementaram timeouts.	Aprovado.
Cancelamento	Estado CANCELLED e métricas preservadas no histórico.	Aprovado.
Recuperação de dados	Mesmos IDs e métricas após reinício da API.	Aprovado.

Os testes unitários empregam dados de teste; o experimento integrado utiliza métricas coletadas pelo próprio motor. Seus resultados comprovam o funcionamento do fluxo no alvo controlado, mas não constituem estimativa de desempenho em produção nem comparação conclusiva entre estratégias.

37 Bugs Identificados e Correções

A revisão identificou problemas no motor anterior e no fluxo de aprendizado. As correções abaixo integram a versão utilizada nos testes desta Sprint; os respectivos comportamentos são cobertos pelos testes automatizados e pelas evidências de execução.

Problema identificado	Correção implementada	Efeito pretendido
Concorrência pouco efetiva	Trabalhadores de longa duração ocupam as vagas configuradas.	Manter requisições concorrentes durante a janela.
Corpo da resposta em memória	Resposta HTTP em streaming, sem reter o corpo completo.	Reduzir memória usada pelo gerador; medir status e tempo até cabeçalhos.
Ramp-up dependente do cenário	Inclusão do valor no snapshot de test_executions.	Preservar parâmetros da execução histórica.
Estratégias sem adaptação	Preditor e controlador conectados ao intervalo entre janelas.	Aplicar a próxima concorrência e registrar a causa.
Horizonte preditivo incompleto	Rótulo considera as cinco janelas futuras.	Alinhar o alvo de aprendizado ao horizonte nominal de 10 s.
Seleção usando o teste final	Partição de validação independente e exclusão nas fronteiras.	Reduzir vazamento na escolha e avaliação do modelo.
Verificação e leitura separadas	Hash e desserialização usam o mesmo conteúdo lido.	Evitar troca do arquivo entre conferência e carregamento.
Tempo de janela e pool HTTP	Tempo real registrado; pool acompanha o teto do cenário.	Evitar throughput inflado e teto implícito de 100 conexões.
Consultas antigas ou lentas	Invalidação por seleção e polling sem sobreposição.	Manter monitor e estado associados à seleção atual.
Falhas de banco e migração	Erro SQL separado da inferência; pré-verificação do histórico.	Interromper com diagnóstico sem alterar históricos incompatíveis.

37.1 Dificuldades encontradas

A principal dificuldade de ambiente foi disponibilizar o Docker para executar o banco e o build completos. O laboratório isolado permitiu aplicar as migrations e verificar a persistência sem usar dados pessoais. No aprendizado, foi necessário gerar uma sequência temporal suficientemente longa e com períodos saudáveis e degradados nas três partições. A validação independente e a exclusão de fronteiras reduziram o risco de avaliar o modelo com informação futura.

O experimento mantém carga pequena para observar o ciclo completo. Um teste adicional usou 110 vagas para verificar o pool, sem certificar o teto de 500. Avisos de depreciação de Starlette/httpx e Alembic foram registrados sem falhas nos testes.

38 Fluxo de Navegação do Segundo Módulo

O painel React reúne configuração, execução, monitoramento e modelos no contexto do projeto selecionado. Essa organização aproxima a decisão do controlador das métricas que a motivaram e mantém acessíveis o histórico e a parada emergencial.

Área da interface	Ação do usuário	Resposta da aplicação
Cenários	Selecionar alvo autorizado, limites e estratégia.	Criar cenário persistido e disponibilizá-lo para execução.
Execuções	Confirmar autorização e iniciar ou interromper.	Registrar estado, parâmetros e motivo de cancelamento.
Monitor adaptativo	Acompanhar janelas e consultar execuções anteriores.	Exibir concorrência, throughput, p95, erros, risco e decisão.
Modelos locais	Treinar candidato, consultar métricas e aprovar versão.	Atualizar estados e indicar disponibilidade de IA ou fallback.

O protótipo do Figma permanece uma referência inicial de navegação e hierarquia visual. A interface atual é uma implementação simples desse fluxo e poderá mudar após testes de usabilidade. As imagens das sprints anteriores representam o estado documentado em cada etapa, não novas evidências da Sprint 05.

38.1 Execução híbrida na aplicação

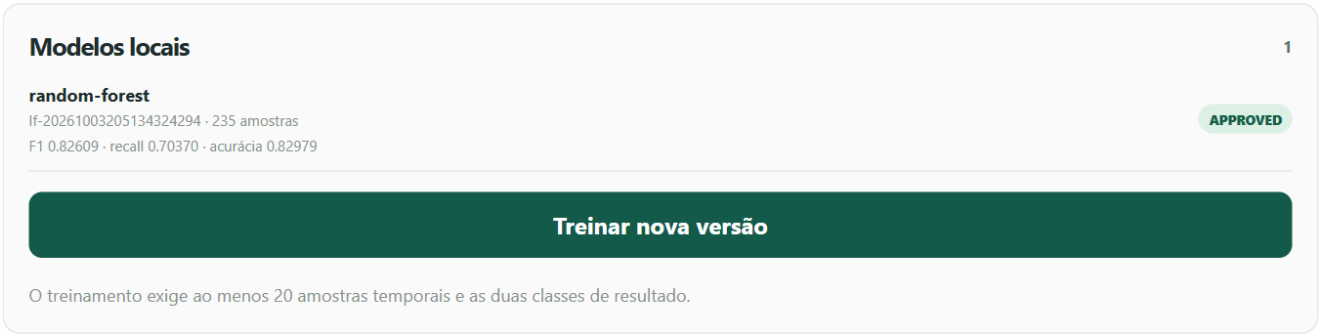
A captura mostra uma execução concluída com métricas recuperadas da API. Cada linha representa uma janela: a concorrência e o throughput descrevem a carga observada, e as colunas de risco e decisão mostram a integração entre IA e controlador.

Monitor adaptativo — 4 janelas

JANELA	CONCORRÊNCIA	THROUGHPUT	P95	ERROS	RISCO	DECISÃO
#1	2	48.0 rps	42 ms	0.0%	38%	INCREASE · 2→3
#2	3	70.5 rps	43 ms	0.0%	34%	INCREASE · 3→4
#3	4	92.0 rps	44 ms	0.0%	8%	HOLD · 4→4
#4	4	92.0 rps	44 ms	0.0%	43%	HOLD · 4→4

Evidência S5-01 — Monitor da execução híbrida com previsões e decisões persistidas, capturado em 03/10/2026.

O modelo associado à execução foi treinado a partir da coleta controlada e aprovado antes de iniciar a carga. O painel permite consultar algoritmo, versão, quantidade de amostras e métricas, mantendo visível qual versão está disponível para novas execuções.



Evidência S5-02 — Versão local aprovada e métricas apresentadas pela aplicação, em 03/10/2026.

No teste independente do candidato lf-20261003205134324294, o F1 registrado foi 0.82609, o recall foi 0.70370 e a acurácia foi 0.82979. Esses valores se referem ao alvo controlado e à divisão temporal documentada; não são extrapolados para outras aplicações.

38.2 Controle e navegação em funcionamento

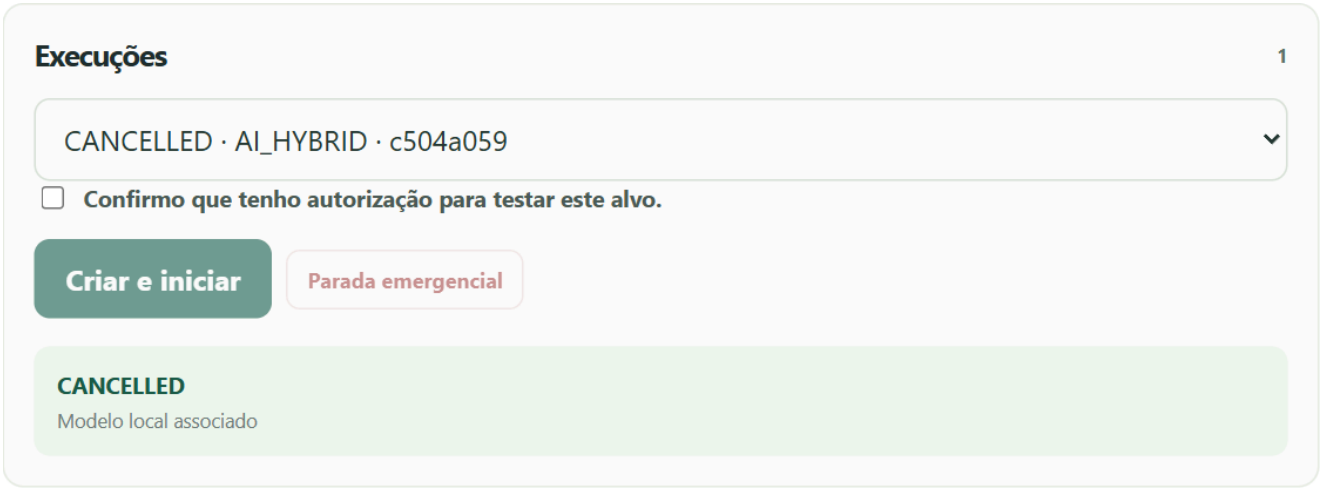
A estratégia por regras também foi exercitada com respostas HTTP 503. O monitor permite relacionar o aumento de erros à redução de concorrência, tornando a decisão explicável sem depender de um modelo de IA.

Monitor adaptativo — 3 janelas

JANELA	CONCORRÊNCIA	THROUGHPUT	P95	ERROS	RISCO	DECISÃO
#1	2	7.0 rps	304 ms	100.0%	regras	DECREASE · 2→1
#2	1	3.5 rps	303 ms	100.0%	regras	DECREASE · 1→1
#3	1	3.5 rps	303 ms	100.0%	regras	DECREASE · 1→1

Evidência S5-03 — Erros do alvo controlado e decisão DECREASE registrada na aplicação.

A navegação confirmou a seleção de cenários, o reconhecimento de autorização e a criação de uma execução pela interface. A parada emergencial foi acionada no navegador e o estado CANCELLED permaneceu consultável com as métricas coletadas.



Evidência S5-04 — Execução interrompida pela interface, preservando o histórico no banco.

39 Repositório e Referências Técnicas

Código, migrations, testes e documentação da Sprint 05 estão organizados na branch local feat/sprint-05-inteligencia-adaptativa. O endereço principal identifica o projeto; as evidências desta seção correspondem à revisão local executada em 03/10/2026.

Repositório principal: github.com/camimcl/sre-fabrica-de-software

Protótipo editável: LoadForge no Figma

Diagramas e imagens: docs/diagrams no repositório

Referências locais da implementação: módulos intelligence e control; integração em load_tests/engine.py; interface SprintFivePanel.tsx; database/schema.sql; migrations 20261001_0003 e 20261001_0004; testes de IA, controlador e motor. O relatório técnico da etapa está em docs/relatorio-sprint-05.md.

As capturas identificadas S5-01 a S5-04 foram obtidas da aplicação em execução, e não do protótipo. O registro técnico reúne as respostas das rotas, os resultados da suíte e as verificações de navegação, permitindo relacionar as imagens aos dados do experimento.

40 Próximos Passos

A próxima etapa amplia a comparação entre FIXED, RULES e AI_HYBRID com repetições equivalentes de carga. Serão consolidados throughput, p95, taxa de erro, custo de inferência e uso de recursos, além da antecedência dos alertas. Essa comparação permite avaliar ganhos de processamento e qualidade preditiva sem confundir funcionamento do módulo com superioridade de uma estratégia.

A evolução operacional inclui recuperação após interrupções inesperadas, controle da rede de saída antes de uso compartilhado e melhoria da experiência de navegação. O protótipo e a interface permanecem sujeitos a ajustes de usabilidade conforme o produto evolui.

A Sprint 05 entrega o ciclo de preparação, treinamento, aprovação, inferência e controle conectado à geração de carga. As evidências demonstram persistência e recuperação dos dados, decisões por janela, tratamento de erros, cancelamento e uso do segundo módulo pela interface.

Implementação da Sprint 06

Histórico e relatórios experimentais integrados

A Sprint 06 entrega o terceiro módulo do LoadForge: histórico de execuções e relatórios comparativos. O usuário pode consultar testes, gerar relatórios individuais, selecionar repetições equivalentes e recuperar comparações persistidas. A etapa também revisa o cronograma, os critérios de aprovação do modelo, a interface e a navegação.

A verificação final de 16/10/2026 registrou 77 testes aprovados, sem falhas e sem testes ignorados, com PostgreSQL 16 isolado. O fluxo inclui nove execuções HTTP reais locais e recuperação de relatórios após reconexão ao banco. Esses resultados comprovam integração e persistência; a demonstração de ganho da IA em uma campanha representativa permanece pendente.

41 Terceiro Módulo Implementado

O módulo reports integra as métricas coletadas pelo motor a uma visão de histórico e análise. O relatório individual consolida requisições, sucessos, timeouts, duração observada, throughput, latência, taxa de erro, concorrência máxima observada e conformidade com os limites. A comparação reúne FIXED, RULES e AI_HYBRID e calcula média, desvio-padrão populacional, mínimo, máximo e variações percentuais entre estratégias.

41.1 Regras da comparação

Regra	Comportamento implementado	Finalidade
Execuções válidas	Somente execuções concluídas e pertencentes ao mesmo projeto.	Evitar mistura de contextos.
Repetições balanceadas	Ao menos três por estratégia; quantidades iguais; até 90 execuções por comparação.	Comparar amostras repetidas.
Condições equivalentes	Mesmo alvo registrado e parâmetros de duração, concorrência, ramp-up, timeout e limites.	Reduzir diferenças de configuração.
Rastreabilidade	Persistência dos resultados, condições, hash SHA-256 e vínculos com cada execução.	Permitir consulta e auditoria.
Permissões	Usuários autenticados consultam; apenas QA proprietário gera relatórios e comparações.	Preservar as regras de acesso.

41.2 Tratamento do feedback anterior

O cronograma foi formalmente revisado em 15/10/2026 para registrar que a IA e o controlador foram antecipados para a Sprint 05. A Sprint 06 concentra relatórios e avaliação experimental; as Sprints 07 e 08 permanecem voltadas à consolidação, validação e entrega. O registro oficial está em docs/cronograma-sprints.md.

Novos treinos utilizam balanceamento de classes e calibram o limiar somente na validação, preservando a separação cronológica e o teste final. A aprovação exige recall registrado de pelo menos 0,80. O recall histórico de 0,70370 e o F1 de 0,82609 permanecem como resultados da Sprint 05, sem revisão retroativa. A calibração não modifica automaticamente os limites de risco do controlador.

42 Evidências das Funcionalidades em Funcionamento

As evidências abaixo correspondem aos testes e builds concluídos em 16/10/2026. O relatório JUnit preserva nomes, duração e resultado dos casos, permitindo conferir a execução da suíte no repositório. Os dez testes de relatórios pertencem à suíte de 77 casos; a execução adicional desse grupo confirmou as últimas asserções de permissões e entrada inválida.

Evidência	Verificação realizada	Resultado
S6-01 Suíte completa	Autenticação, cadastros, motor, IA, controle, relatórios, migrações e PostgreSQL.	77 aprovados; 0 falhas; 0 ignorados; 83,922 s.
S6-02 Fluxo HTTP real	Três repetições de cada estratégia, em ordem rotativa, contra servidor local.	Nove execuções concluídas e comparação recuperada pela API.
S6-03 Persistência	Geração sobre esquema migrado, fechamento da conexão e consulta em nova sessão PostgreSQL.	Mesmo resultado e nove vínculos recuperados.
S6-04 Validações e acesso	Requisições sem autenticação, perfil visualizador, IDs inválidos, duplicações e nome vazio.	Respostas 401, 403, 404 e 422 verificadas; consulta permitida ao visualizador.
S6-05 Evolução do banco	Aplicação das migrations até 20261016_0006 e verificação de históricos anteriores.	Dados compatíveis preservados; limites incompatíveis rejeitados.
S6-06 Compilação	TypeScript, Vite e builds Docker da API e do frontend.	Builds aprovados; instalação limpa do frontend sem vulnerabilidades reportadas.

42.1 Rastreabilidade dos testes

O caso test_repeated_real_executions_integrate_with_reports, em backend/tests/test_load_tests_integration.py, autentica o QA, executa a carga, gera a comparação e consulta o histórico. O caso test_comparison_survives_postgres_reconnection, em backend/tests/test_reports.py, verifica o resultado após reabrir a conexão. O mesmo arquivo cobre equivalência, estatísticas, snapshot do alvo, fallback e permissões.

Relatório JUnit da verificação final

Testes do terceiro módulo

Teste integrado do motor com os relatórios

42.2 Alcance das evidências

O teste HTTP integrado utiliza tráfego real apenas em localhost, sem modelo aprovado; por isso, AI_HYBRID opera em fallback por regras. A fixture de estatísticas utiliza valores controlados para verificar os cálculos. Nenhum desses resultados demonstra superioridade da IA ou cumprimento da meta de recall em dados representativos.

A suíte foi executada localmente com Python 3.12 e PostgreSQL 16 descartável. A imagem da API utiliza Python 3.11. Avisos de depreciação de TestClient/httpx e Alembic não causaram falhas. O banco e os contêineres existentes de desenvolvimento foram preservados.

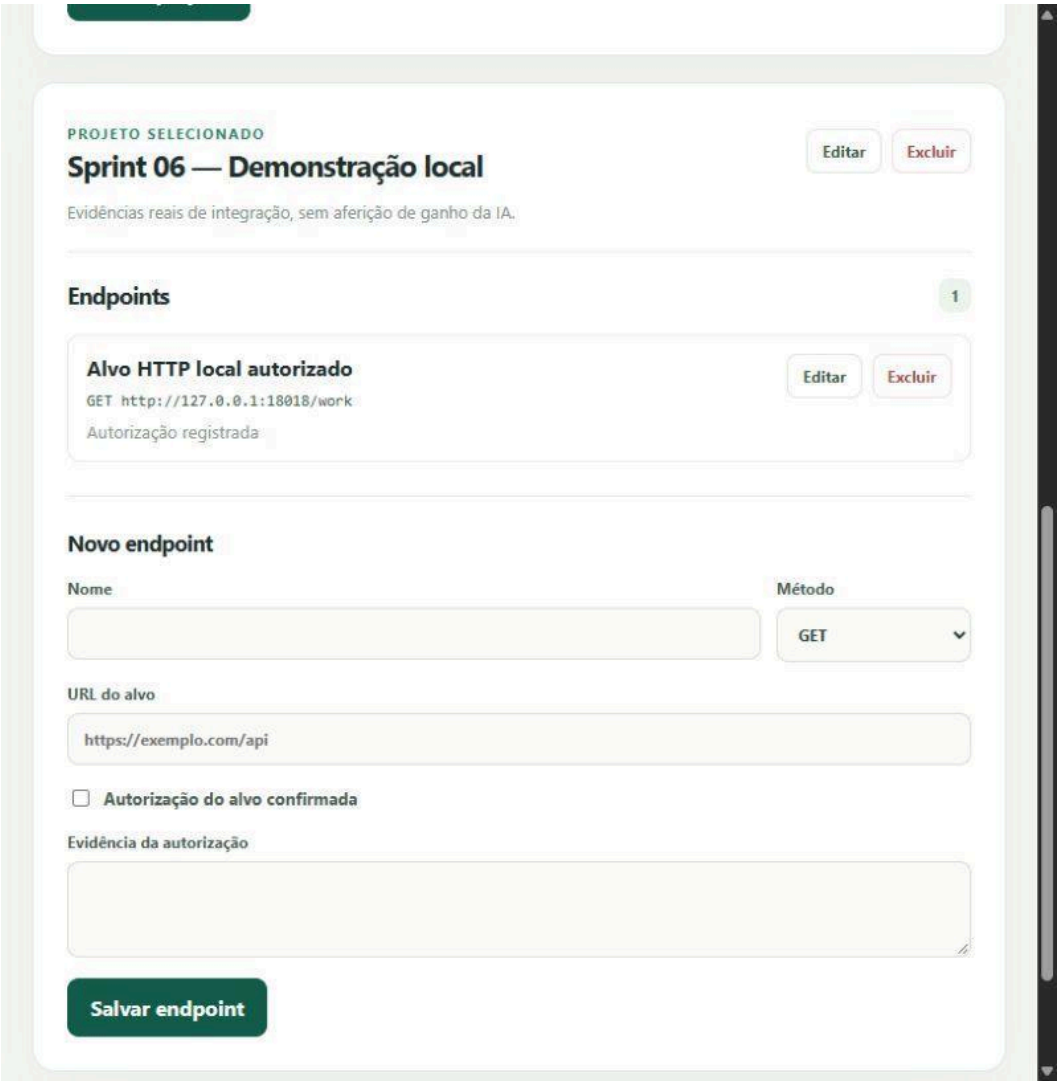
42.3 Evidências visuais da interface

As capturas S6-07 a S6-14 foram obtidas em 16/10/2026 na aplicação em execução, com o backend da revisão 6e57982 da branch da Sprint 06. Foram realizadas nove execuções HTTP locais, com quatro segundos cada, duas janelas por execução e três repetições por estratégia. As métricas foram coletadas pelo motor; não foram inseridos resultados simulados no banco.

A demonstração utilizou um schema exclusivo no PostgreSQL de testes e um alvo autorizado em localhost, sem alterar o ambiente de desenvolvimento. A comparação foi gerada pela interface e recuperada após trocar de projeto e recarregar a página. Sem modelo aprovado, as seis janelas de AI_HYBRID utilizaram fallback por regras; os números comprovam o funcionamento do fluxo, não ganho da IA.

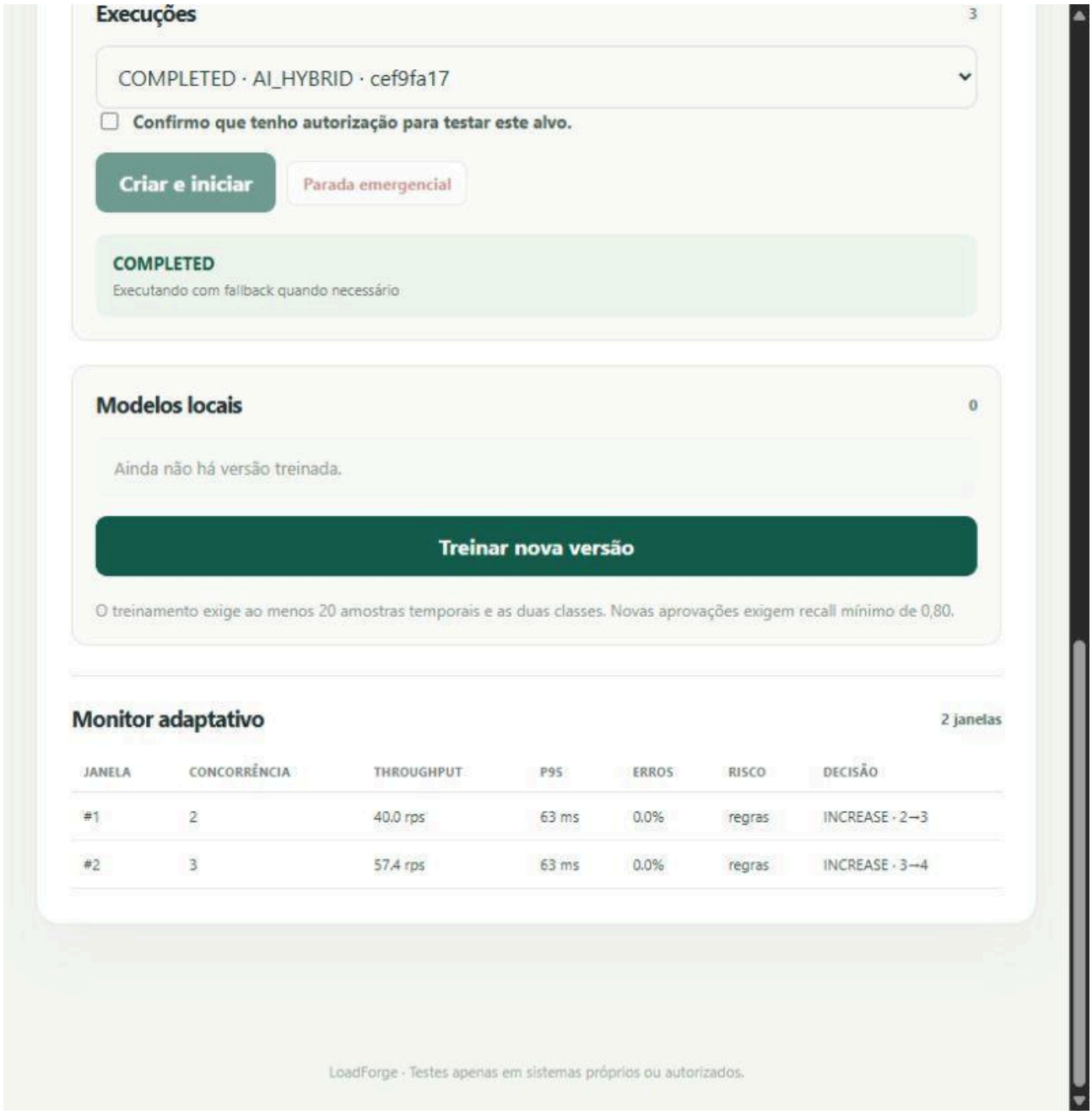
Estas imagens complementam o registro anterior: as menções à ausência de inspeção visual e à captura ainda pendente nas seções seguintes descrevem a revisão histórica. Na revisão atual, o snapshot do alvo já é salvo na criação da execução, e a leitura do plano não grava dados.

Projeto e alvo autorizado



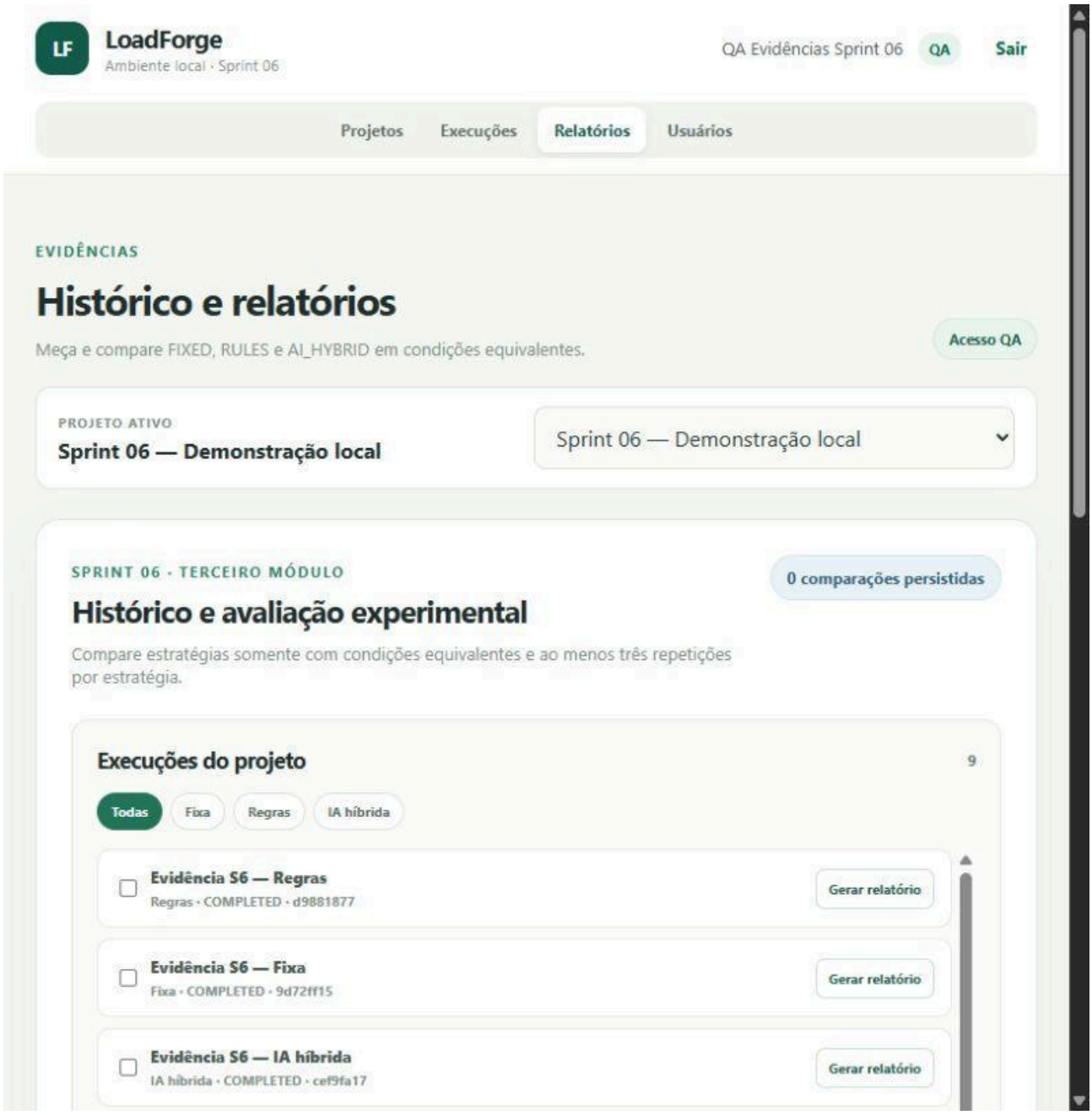
Evidência S6-07 — Projeto de demonstração e endpoint GET em localhost com autorização registrada. O mesmo alvo aparece nas condições persistidas da comparação.

Métricas e decisões integradas



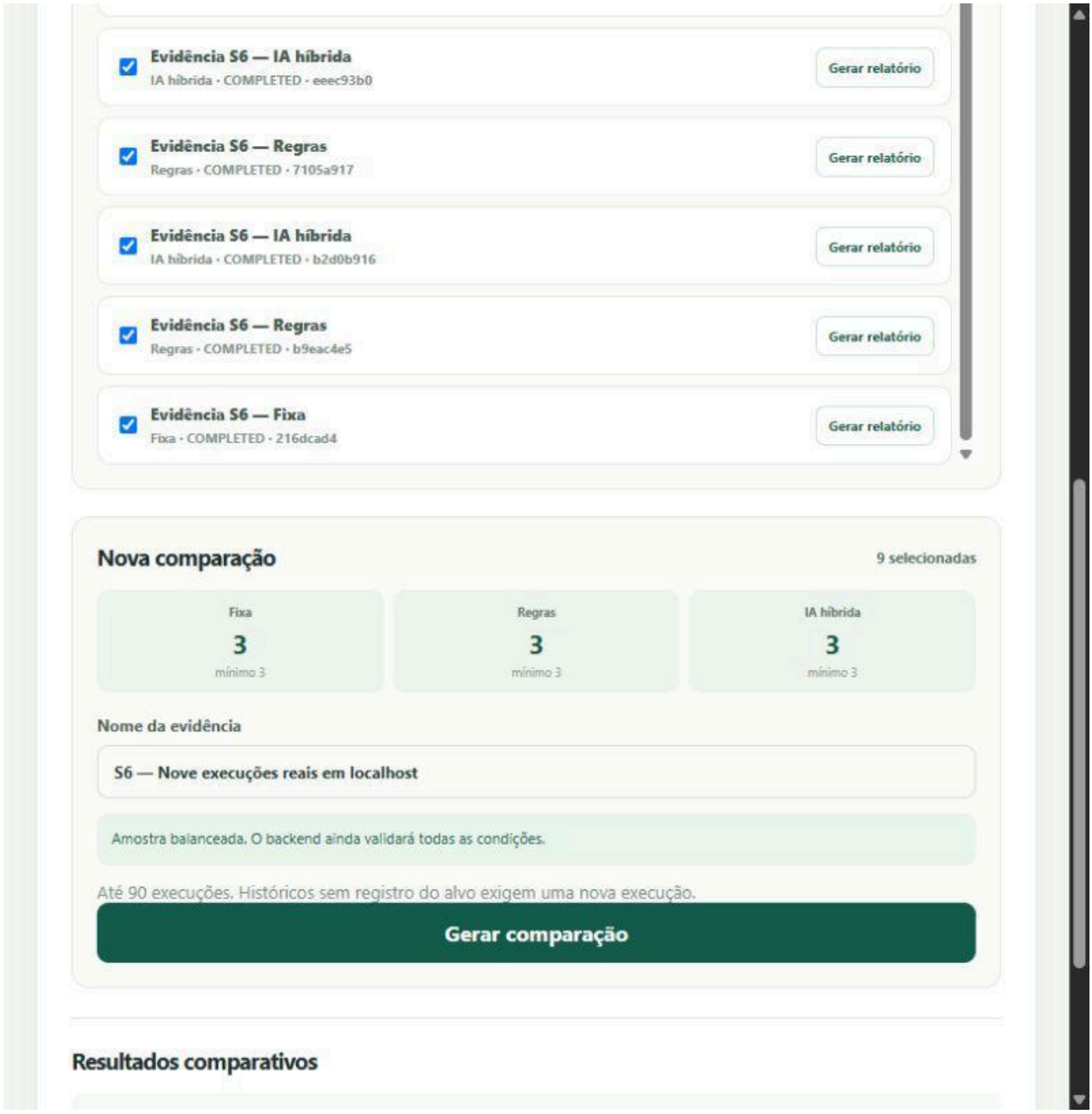
Evidência S6-08 — Execução AI_HYBRID concluída, com duas janelas reais e decisões INCREASE de 2 para 3 e de 3 para 4. A tela informa ausência de modelo e uso de fallback por regras.

Histórico do terceiro módulo



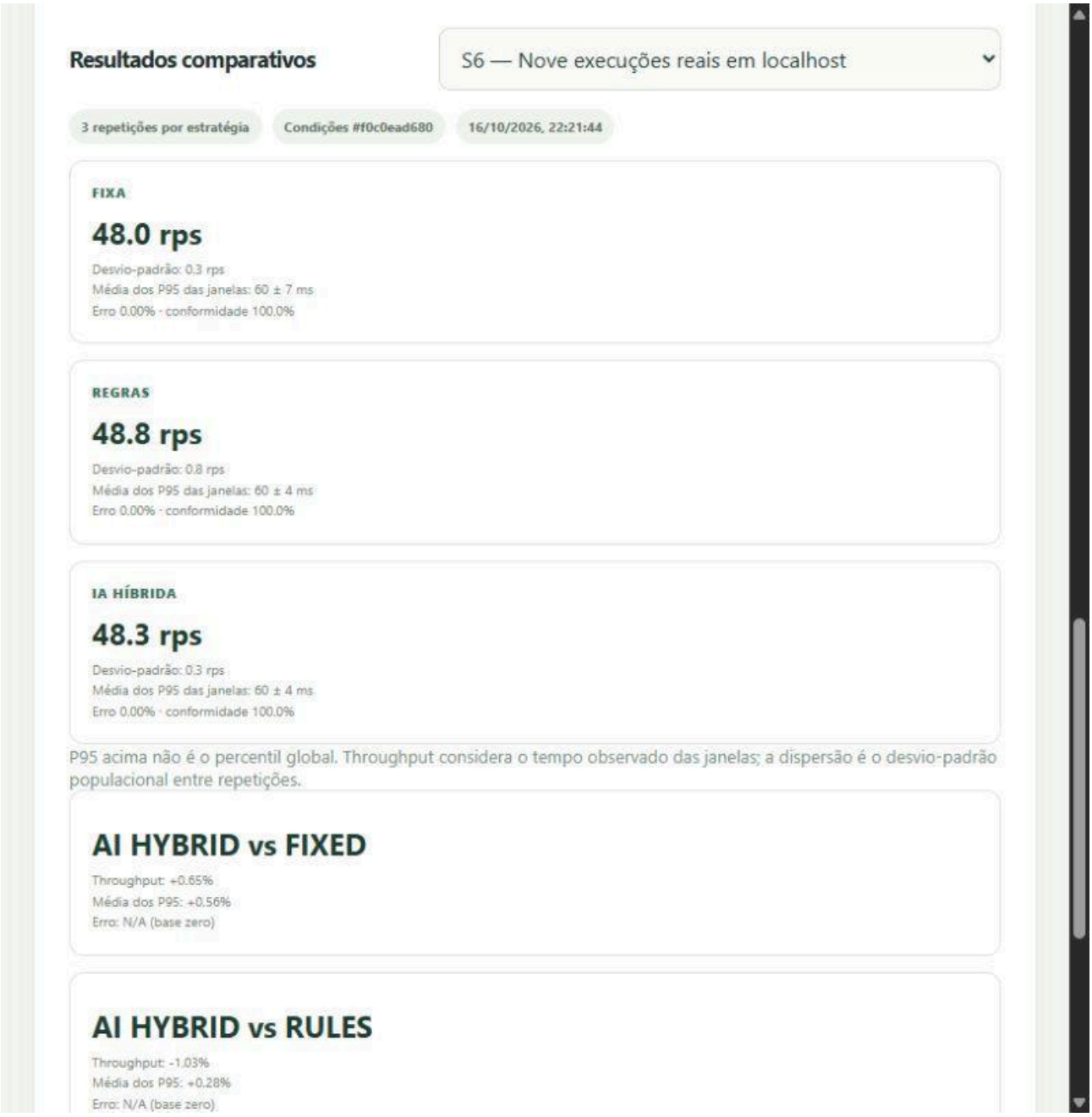
Evidência S6-09 — A área Relatórios recebe as nove execuções concluídas do projeto. A navegação principal, o seletor de contexto e os filtros por estratégia estão disponíveis na mesma interface.

Seleção balanceada



Evidência S6-10 — Nove execuções selecionadas, sendo três de cada estratégia. A interface reconhece a amostra balanceada e habilita a geração; o backend valida as condições antes de persistir o resultado.

Comparação gerada



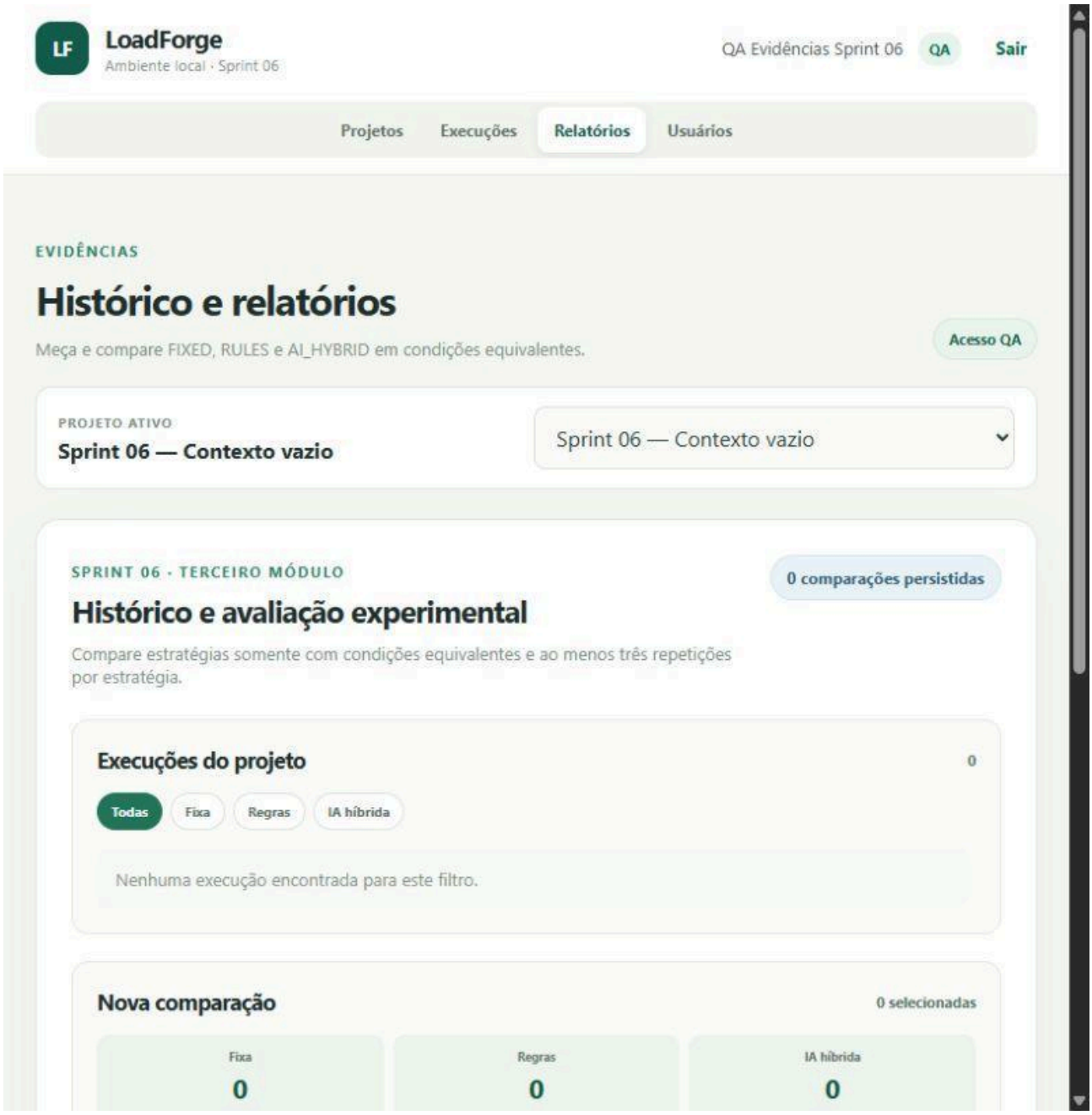
Evidência S6-11 — Resultado gerado pela interface com três repetições por estratégia, impressão das condições f0c0ead680, throughput, dispersão, latência e deltas. Os valores pertencem à demonstração local e não demonstram superioridade da IA.

Auditoria e condições registradas



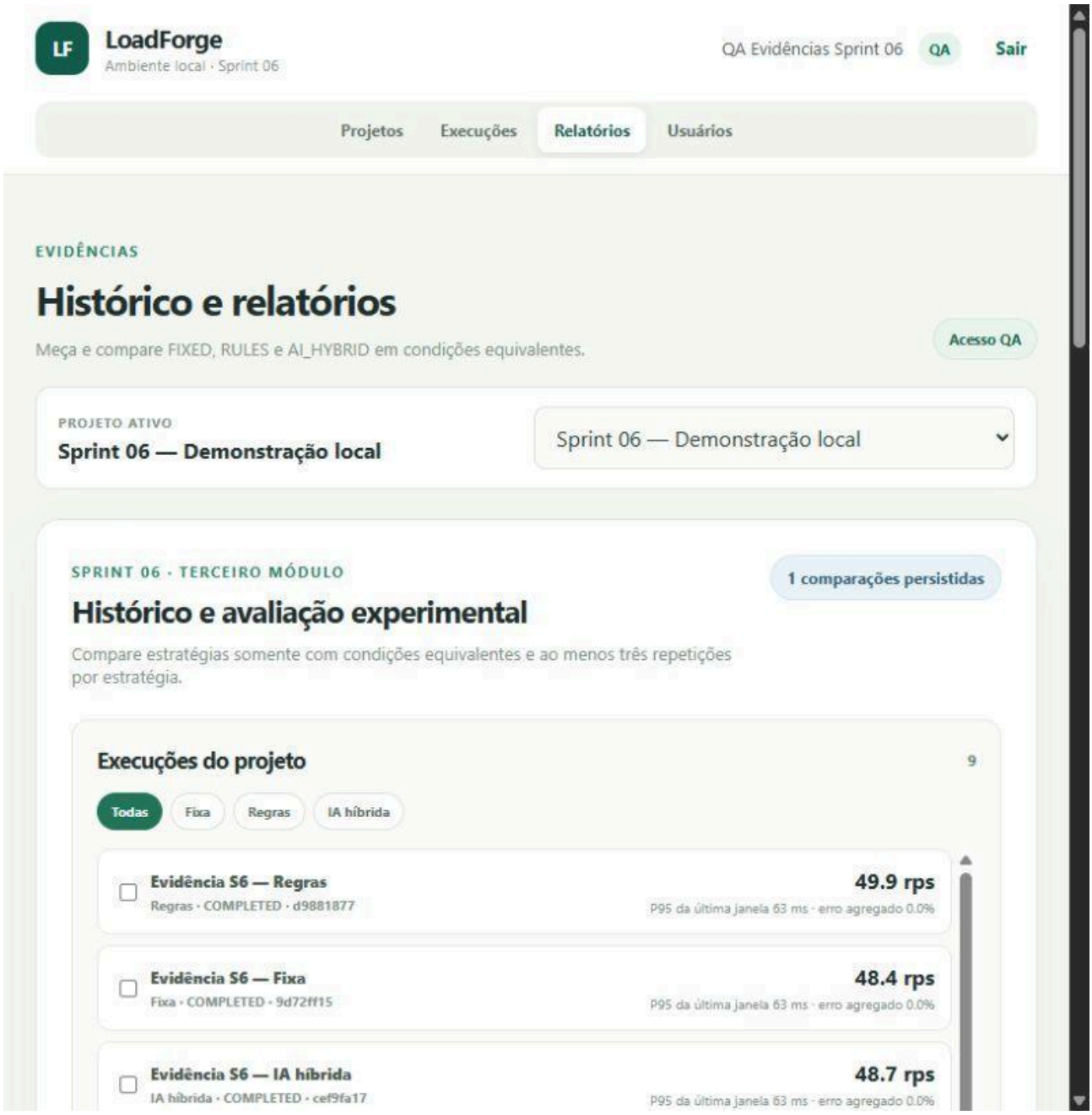
Evidência S6-12 — Detalhes expandidos com parâmetros de carga, snapshot do alvo e os nove identificadores das execuções. A auditoria registra 0 de 6 janelas com inferência e 6 em fallback, preservando a ressalva na conclusão.

Navegação entre projetos



Evidência S6-13 — Após trocar para o projeto Contexto vazio, o histórico e as comparações ficam vazios. O painel não reaproveita resultados ou seleções do projeto anterior.

Recuperação após recarregar



Evidência S6-14 — Após retornar ao projeto de demonstração, recarregar a aplicação e abrir Relatórios novamente, a interface recupera uma comparação persistida e nove execuções com seus relatórios. A comparação manteve o mesmo nome, impressão das condições e valores.

43 Evidências da Integração entre os Módulos

O terceiro módulo utiliza os registros produzidos pelas etapas anteriores. A comparação parte das execuções do projeto, agrega suas janelas de métricas e relaciona o resultado às decisões e versões de modelo. A API disponibiliza histórico, geração de relatórios individuais e criação, listagem e consulta de comparações.

Origem	Integração com o terceiro módulo	Evidência
Projetos e autorização	O projeto delimita as execuções; perfil e propriedade controlam a geração.	Testes HTTP de autenticação e permissões.
Cenários e motor	As execuções preservam configurações de carga e o alvo carregado antes do tráfego.	Fluxo real local e testes de snapshot.
Métricas	MetricWindow alimenta agregações e estatísticas; não há resultado manual na aplicação.	Testes de agregação e relatórios individuais.
Inteligência e controle	A auditoria distingue inferência, fallback e janelas sem comprovação, com IDs de modelos.	Teste de fallback e histórico de decisões.
Banco e interface	Comparações salvas podem ser listadas e recuperadas junto às execuções utilizadas.	Consulta posterior pela API e reconexão PostgreSQL.

43.1 Persistência e integridade das condições

A migration 20261015_0005 cria comparison_reports e comparison_report_executions. A revisão 20261016_0006 adiciona target_snapshot às execuções. O motor registra endpoint, URL, método HTTP e duração nominal da janela antes de iniciar a carga, evitando que uma edição posterior do cadastro altere o significado de uma comparação.

Históricos sem esse snapshot continuam disponíveis para consulta e relatório individual, mas não podem formar novas comparações verificáveis. O sistema solicita novas execuções em vez de preencher retroativamente um alvo que não foi registrado. O teste de mudança de endpoint confirma que as condições originais permanecem identificáveis.

43.2 Interpretação das métricas e do uso de IA

A latência comparativa é calculada a partir da média dos P95 das janelas por execução, e não do percentil global de todas as requisições. O throughput considera o tempo observado nas janelas, excluindo intervalos de persistência entre elas. No relatório individual, o P95 final corresponde à última janela, enquanto a taxa de erro é agregada.

O resultado informa quantas janelas tiveram inferência, quantas usaram regras e quantas não têm comprovação, além das versões de modelo registradas. Fallback, informação incompleta ou modelos diferentes impedem atribuir os deltas exclusivamente à IA. A conclusão permanece descritiva, sem afirmar significância estatística ou generalização para produção.

Serviço de agregação e comparação

44 Melhorias Realizadas na Interface

A interface organiza o uso em Projetos, Testes, Relatórios e Usuários, respeitando as permissões. Nas áreas dependentes de contexto, o seletor de projeto permanece visível. O painel de relatórios reúne histórico, montagem da comparação e resultados, reduzindo a necessidade de retornar ao cadastro para alternar o conjunto de execuções.

Área	Melhoria implementada	Efeito no uso
Histórico	Filtro por estratégia, identificação de cenário, estado e execução; geração individual.	Localizar e selecionar testes.
Comparação	Contadores por estratégia, balanceamento, limite de 90 e bloqueio durante operações.	Orientar a seleção e evitar solicitações concorrentes.
Resultados	Deltas, dispersão, condições expansíveis, IDs de execuções e auditoria de fallback.	Explicar a origem e os limites dos números.
Modelo e mensagens	Meta de recall, limiar versionado e detalhamento dos erros de validação 422.	Mostrar critérios e motivos de rejeição.
Troca de contexto	Reinicialização dos painéis e descarte de respostas atrasadas do contexto anterior.	Evitar mistura visual de projetos.

45 Demonstração dos Ajustes de Navegação

O roteiro a seguir relaciona os caminhos implementados e o resultado esperado na demonstração da Sprint 06. O build e os fluxos HTTP foram verificados; a revisão final não incluiu nova inspeção visual no navegador. As capturas S5-01 a S5-04 permanecem evidências históricas da Sprint 05 e não representam a nova tela de relatórios.

Etapa	Caminho e ação	Resultado esperado
1 Contexto	Entrar como QA proprietário; abrir Projetos e confirmar o endpoint autorizado.	Projeto e alvo corretos.
2 Execução	Abrir Testes, selecionar o projeto e concluir três repetições por estratégia.	Métricas e snapshots persistidos.
3 Histórico	Abrir Relatórios, conferir o projeto e filtrar as execuções.	Histórico do contexto selecionado.
4 Seleção	Selecionar execuções concluídas em quantidades iguais e informar o nome.	Contadores balanceados; validação final pelo backend.
5 Resultado	Gerar a comparação e expandir condições e execuções utilizadas.	Estatísticas, deltas e ressalvas consultáveis.
6 Retorno	Alternar área ou projeto e recuperar a comparação; repetir a consulta após recarregar.	Dados persistidos, sem reaproveitar seleção do projeto anterior.

Painel de relatórios e navegação implementada

46 Repositório GitHub Atualizado

As alterações da Sprint 06 foram publicadas na branch sprint-06-avaliacao-experimental-relatorios, sobre a main que incorporou o PR 4 de integração contínua. O workflow existente foi preservado. A referência abaixo fixa a revisão documentada em 16/10/2026; a branch pode receber novas alterações posteriormente.

Repositório principal github.com/camimcl/sre-fabrica-de-software

Branch publicada da Sprint 06

Revisão documentada 85575e7

Cronograma formal revisado

Relatório técnico e evidências da Sprint 06

Os commits finais foram 1bd0aed, com preservação do alvo, auditoria de fallback e testes, e 85575e7, com a documentação da verificação e a descrição preparada para o pull request. Nenhum PR da Sprint 06 foi criado nessa revisão; os links de branch e commit identificam o conteúdo disponível para análise.

47 Dificuldades Encontradas e Próximos Passos

47.1 Dificuldades e tratamento adotado

A equivalência experimental exigiu corrigir a dependência do endpoint atual do cenário. O snapshot passou a representar o alvo utilizado no teste. Também foi necessário diferenciar o funcionamento em fallback da atuação efetiva do modelo e esclarecer a média dos P95, evitando interpretações indevidas dos relatórios.

A integração com o CI exigiu alinhar o uv.lock à versão 0.6.0. A validação do PostgreSQL foi realizada em contêiner descartável separado, sem alterar o banco de desenvolvimento. A vulnerabilidade transitiva identificada no frontend foi tratada com a atualização de source-map-js para 1.2.2. O envio por SSH encontrou falta de chave autorizada e foi concluído pela conexão autenticada do GitHub, com correspondência conferida entre os conteúdos local e remoto.

47.2 Próximos passos

• Coletar dados representativos, treinar uma nova versão e verificar recall de pelo menos 0,80 no teste reservado, registrando também F1, precisão, falsos positivos e custo de inferência.

• Executar uma campanha com modelo aprovado e ao menos três repetições equivalentes de FIXED, RULES e AI_HYBRID, registrando versão do alvo, ordem, ambiente, aquecimento e eventuais descartes.

• Avaliar ganhos e perdas com dispersão e limites de interpretação. A implementação atual usa médias, desvios-padrão e extremos; a mediana prevista inicialmente não foi acrescentada ao relatório nesta versão.

• Demonstrar o roteiro no navegador e registrar novas capturas da interface, incluindo troca de projetos, retorno à comparação e mensagens de validação.

• Submeter a branch à revisão da equipe e executar o workflow remoto quando o PR for autorizado; avançar em segurança operacional, recuperação após interrupções e documentação final.

48. Entrega da Sprint 07 e escopo das evidências

A Sprint 07 amplia o LoadForge com um painel de acompanhamento, consultas com pesquisa e filtros, exportação e histórico de operações. Os recursos utilizam os projetos, cenários, execuções, métricas e relatórios persistidos pelos módulos anteriores.

Este complemento foi acrescentado após as 59 páginas do documento recebido. As seções e evidências das Sprints anteriores permanecem preservadas. As novas capturas foram obtidas em 24/10/2026, executando a revisão final do PR nº 6 em ambiente local isolado, com PostgreSQL e um alvo HTTP local autorizado.

Referência	Versão utilizada
Repositório	github.com/camimcl/sre-fabrica-de-software
Entrega Sprint 07	PR nº 6, integrado ao repositório em 23/10/2026
Revisão do PR	d3170ee62cc95c21a70e7285f36e9db46921380b
Integração em main	597677fe9612402557ca292e0557f824303fad42

Protocolo da demonstração. Foram criados um projeto, um endpoint e três cenários equivalentes: FIXED, RULES e AI_HYBRID. Cada estratégia foi executada três vezes, alternando a ordem, com duração de quatro segundos e duas janelas por execução. Uma décima execução foi cancelada antes de produzir tráfego.

Os nove testes concluídos geraram 1.714 requisições reais ao alvo local, nove relatórios individuais e uma comparação persistida. O painel apresentou 10 execuções, três cenários, um endpoint, uma comparação e 90% de conclusão. A auditoria registrou 26 operações no contexto consultado.

Limite de interpretação. Os ensaios curtos comprovam o funcionamento e a integração dos recursos da Sprint 07. Não substituem uma campanha de desempenho em diferentes cargas. Sem modelo aprovado neste ambiente, as seis janelas híbridas usaram fallback; não houve inferência de IA comprovada.

49. Dashboard implementado e funcionamento

O dashboard reúne contadores, taxa de conclusão, throughput médio, P95 médio e erro médio, além da distribuição das execuções por situação e estratégia. Os valores são consultados no sistema e permitem acompanhar o projeto selecionado.

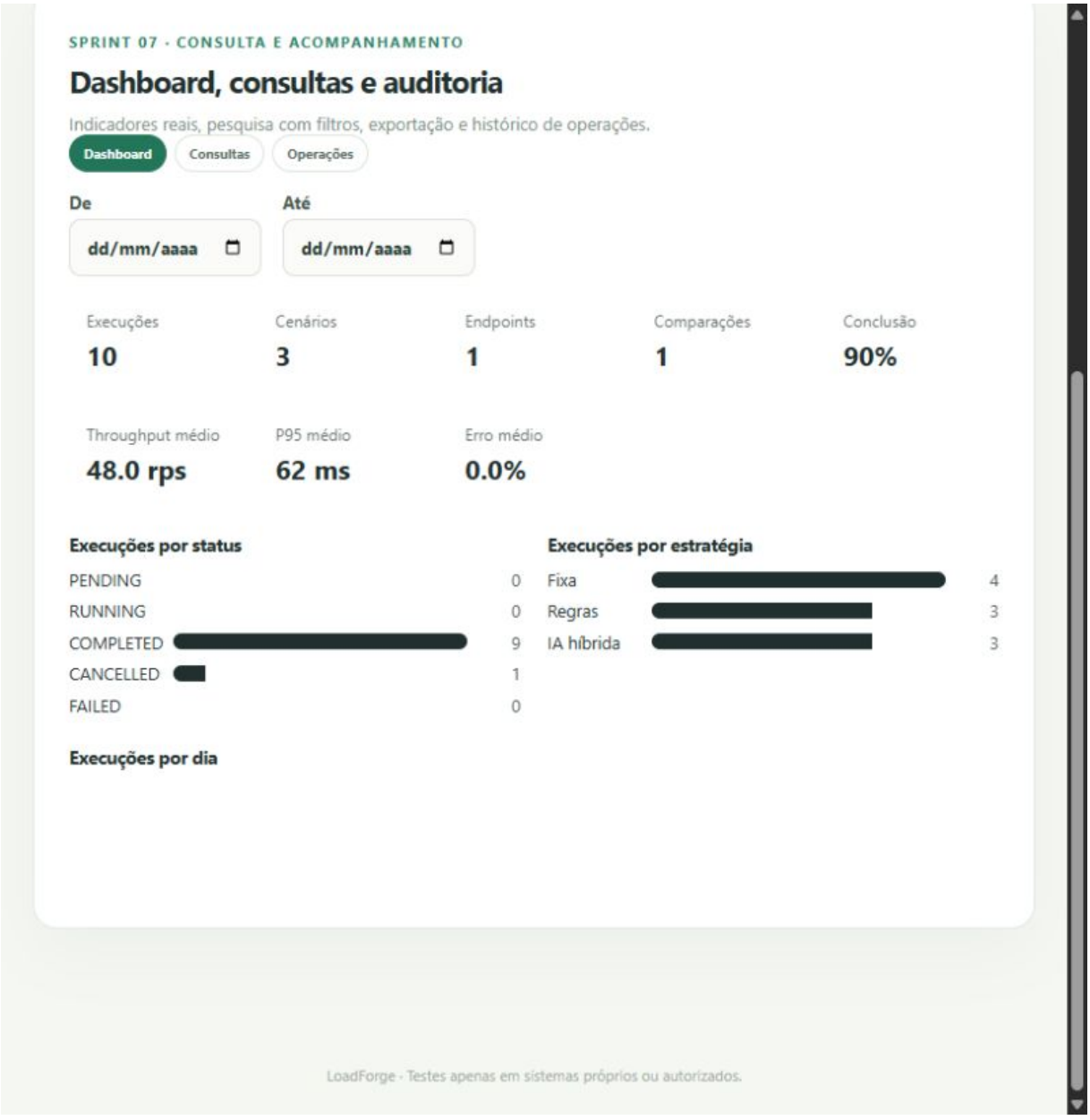


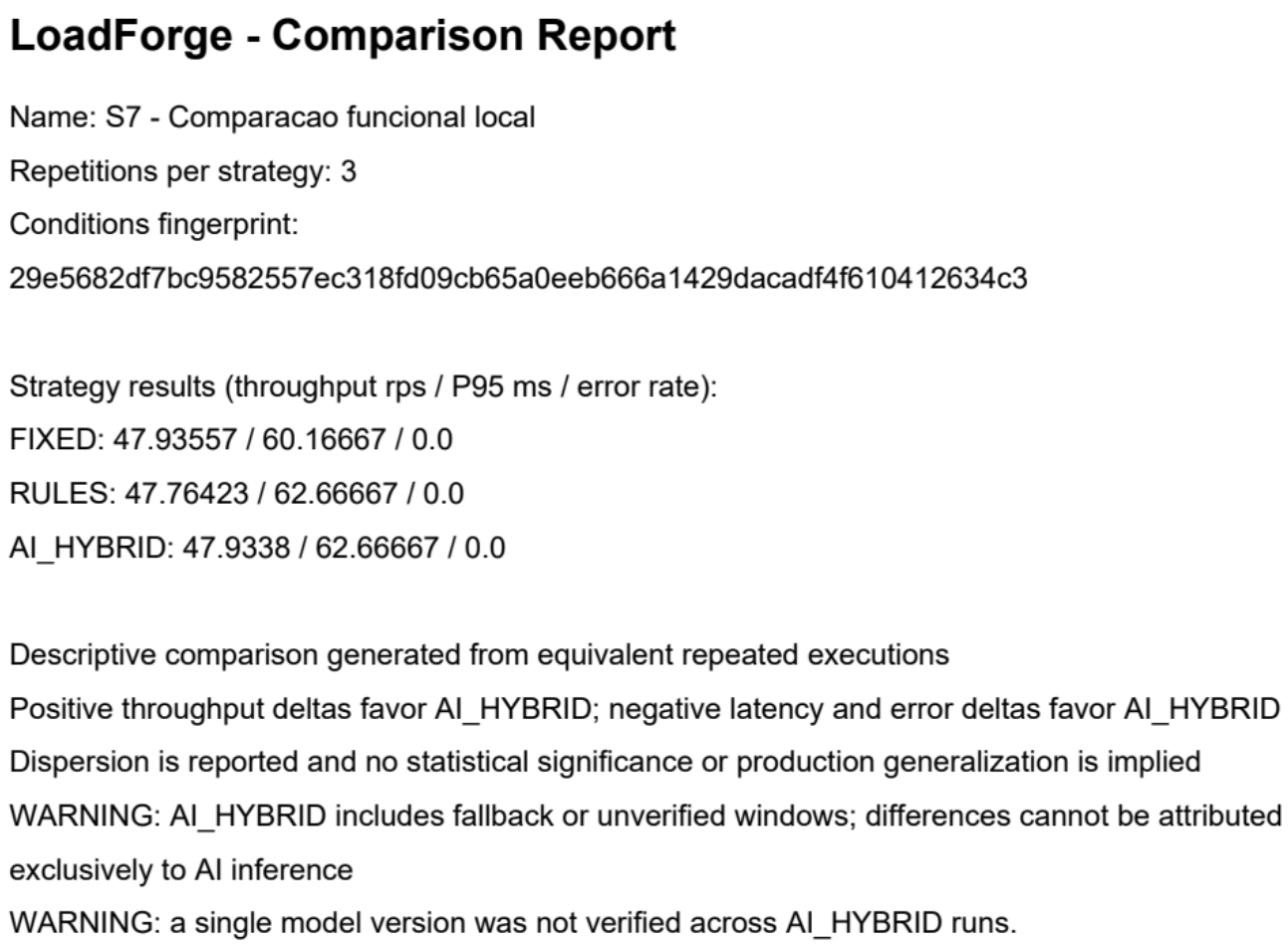
Figura S7-01. Print real do dashboard: 10 execuções, 9 concluídas, 1 cancelada, 48,0 rps e erro médio de 0,0%. Dados da demonstração local de 24/10/2026.

O período refina as métricas e as distribuições de execuções. Os totais de cenários, endpoints e comparações representam o cadastro do projeto. A série por dia possui apenas uma data nesta amostra; o print não demonstra tendência em vários dias.

50. Relatórios individuais e comparação de estratégias

Os relatórios individuais consolidam requisições, sucessos, throughput, latência P95 e taxa de erro a partir das janelas da execução. A comparação reutiliza esses relatórios e registra condições equivalentes, repetições, dispersão e diferenças entre FIXED, RULES e AI_HYBRID.

Na demonstração foram gerados nove relatórios e a comparação “S7 - Comparacao funcional local”, com três repetições por estratégia. A exportação abaixo foi retornada pelo próprio sistema e apresenta os resultados consolidados.



LoadForge - Comparison Report

Name: S7 - Comparacao funcional local

Repetitions per strategy: 3

Conditions fingerprint:

29e5682df7bc9582557ec318fd09cb65a0eeb666a1429dacadf4f610412634c3

Strategy results (throughput rps / P95 ms / error rate):

FIXED: 47.93557 / 60.16667 / 0.0

RULES: 47.76423 / 62.66667 / 0.0

AI_HYBRID: 47.9338 / 62.66667 / 0.0

Descriptive comparison generated from equivalent repeated executions

Positive throughput deltas favor AI_HYBRID; negative latency and error deltas favor AI_HYBRID

Dispersion is reported and no statistical significance or production generalization is implied

WARNING: AI_HYBRID includes fallback or unverified windows; differences cannot be attributed exclusively to AI inference

WARNING: a single model version was not verified across AI_HYBRID runs.

Figura S7-02. Reprodução da primeira página do PDF comparativo efetivamente exportado pela aplicação. Não é uma tela simulada.

O relatório também preserva a rastreabilidade das condições e da participação da IA. Nesta amostra, AI_HYBRID teve zero janelas com inferência e seis com fallback. Resultados próximos entre estratégias, neste alvo e duração, não demonstram superioridade do modelo.

51. Pesquisas e filtros integrados aos dados

A área Consultas permite localizar execuções por texto e refinar o resultado por período, situação e estratégia. A paginação informa a quantidade de registros e os resultados mantêm acesso à exportação. Na captura, os critérios retornam exatamente as três execuções híbridas concluídas.

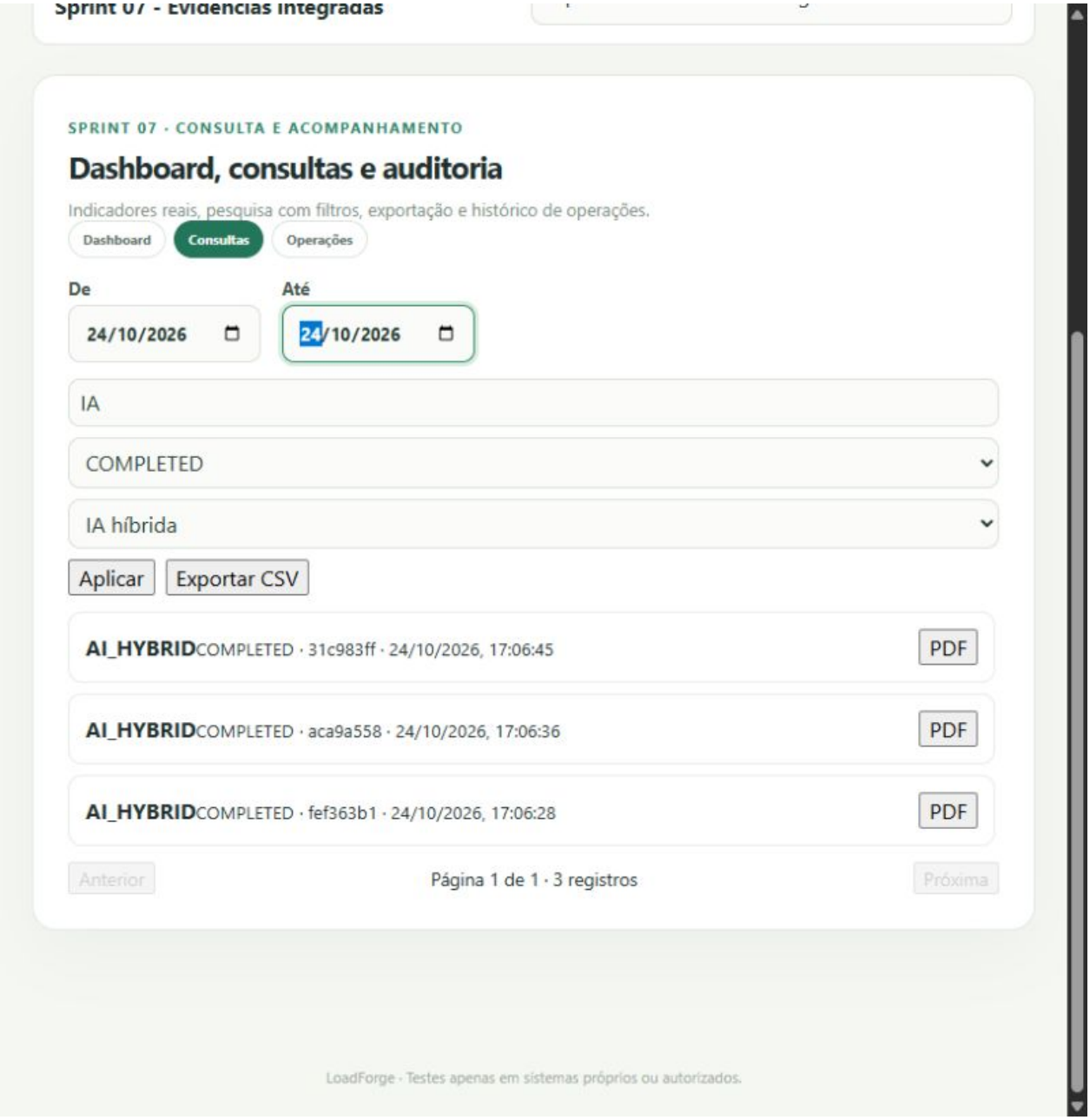


Figura S7-03. Print real com pesquisa “IA”, período de 24/10/2026, situação COMPLETED e estratégia IA híbrida: três registros, uma página e ações de exportação CSV/PDF.

As execuções identificadas por 31c983ff, aca9a558 e fef363b1 também aparecem no CSV exportado com os mesmos critérios, demonstrando a ligação entre pesquisa, filtro e saída de dados.

52. Exportação de dados e relatórios

A exportação é aplicável ao LoadForge porque permite anexar resultados às entregas, comparar ensaios e analisar os dados fora do sistema. A consulta oferece CSV e PDF de execução; a API também disponibiliza CSV e PDF de comparação. A captura da seção anterior mostra os comandos disponíveis na interface.

Arquivo validado	Conteúdo observado
CSV de execuções filtradas	3 registros, todos AI_HYBRID e COMPLETED; os mesmos IDs da consulta.
PDF de execução	Relatório individual com métricas da execução persistida.
CSV de comparação	3 linhas de resultado, uma por estratégia.
PDF de comparação	Consolidação da comparação gerada com três repetições por estratégia.

Conferência do CSV retornado. O arquivo contém as colunas execution_id, scenario_name, strategy, status, started_at, ended_at e created_at. As linhas abaixo reproduzem os dados do arquivo, com os identificadores abreviados apenas para leitura neste documento.

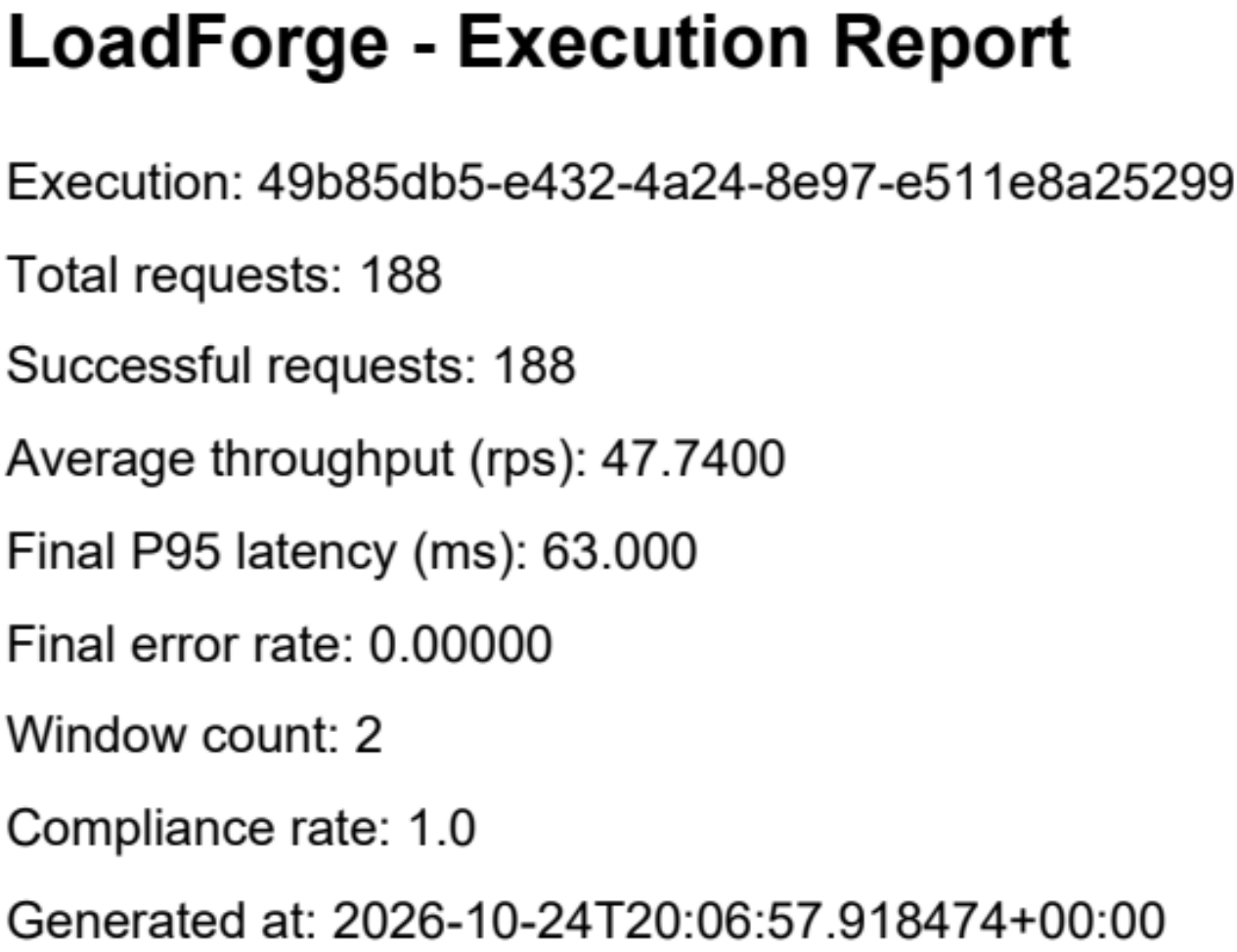
Execução	Cenário	Estratégia / situação
31c983ff	Evidencia S7 - IA hibrida	AI_HYBRID / COMPLETED
aca9a558	Evidencia S7 - IA hibrida	AI_HYBRID / COMPLETED
fef363b1	Evidencia S7 - IA hibrida	AI_HYBRID / COMPLETED

As quatro respostas de exportação foram recebidas com sucesso (HTTP 200), e os arquivos foram abertos e conferidos. Os horários do CSV incluem o fuso UTC; a interface apresenta os horários locais. A próxima página reproduz o conteúdo de um PDF individual exportado.

O CSV de comparação e o PDF comparativo foram verificados pela API; esta evidência não pressupõe a existência de botões de comparação na interface atual.

52.1. Evidência do PDF individual exportado

O documento abaixo foi gerado pelo recurso de exportação do LoadForge a partir de uma execução concluída. As métricas pertencem ao mesmo conjunto utilizado pelo dashboard e pela comparação.


LoadForge - Execution Report

Execution: 49b85db5-e432-4a24-8e97-e511e8a25299

Total requests: 188

Successful requests: 188

Average throughput (rps): 47.7400

Final P95 latency (ms): 63.000

Final error rate: 0.00000

Window count: 2

Compliance rate: 1.0

Generated at: 2026-10-24T20:06:57.918474+00:00

Figura S7-04. Reprodução do PDF individual retornado pelo sistema. O identificador e os indicadores permitem relacionar o arquivo à execução armazenada.

A reprodução mostra o arquivo real exportado. A disponibilização do PDF permite consultar e compartilhar uma apresentação organizada dos resultados sem depender da tela de acompanhamento.

53. Logs e histórico de operações

A aba Operações exibe o histórico persistido com ação, entidade, data, identificador e detalhes das alterações. A API mantém o vínculo com o ator; os detalhes permitem distinguir ações do usuário e transições realizadas pelo motor de execução.

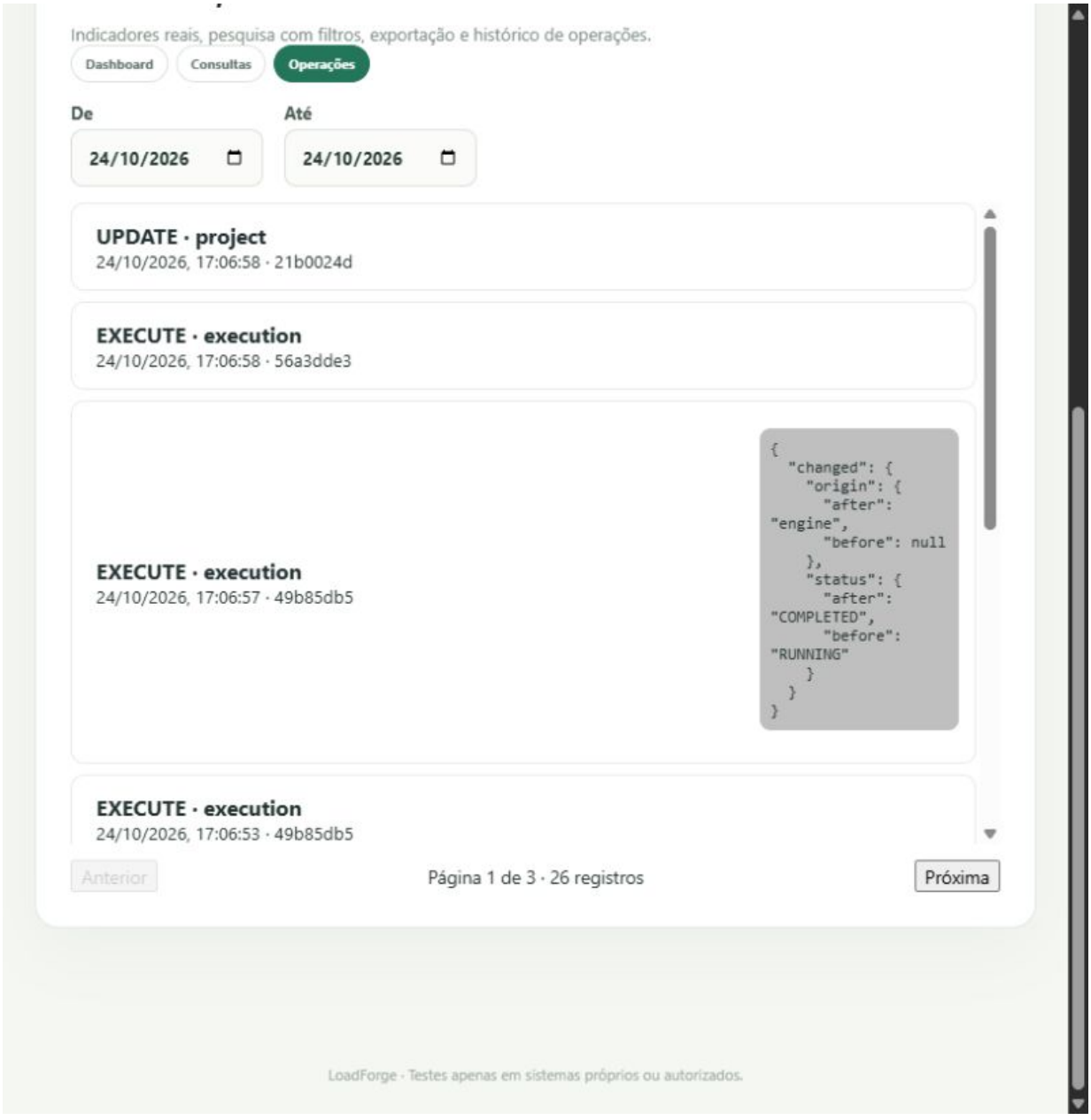


Figura S7-05. Print real do histórico com 26 registros. A alteração expandida registra a origem “engine” e a transição de RUNNING para COMPLETED; também aparecem a atualização do projeto e o cancelamento de uma execução.

O histórico permite reconstituir o ciclo de uma execução. O período é integrado à consulta de operações e a paginação evita carregar toda a lista de uma só vez. A visualização respeita o escopo de acesso da conta; não se deve interpretar o histórico global da conta como restrito apenas ao projeto selecionado.

53.1. Evidência do filtro de período no histórico

Após consultar as operações de 24/10/2026, foi selecionado o dia seguinte, sem operações nesse ambiente. O sistema apresentou zero registros e a paginação voltou à primeira página, mantendo a interface disponível para uma nova consulta.

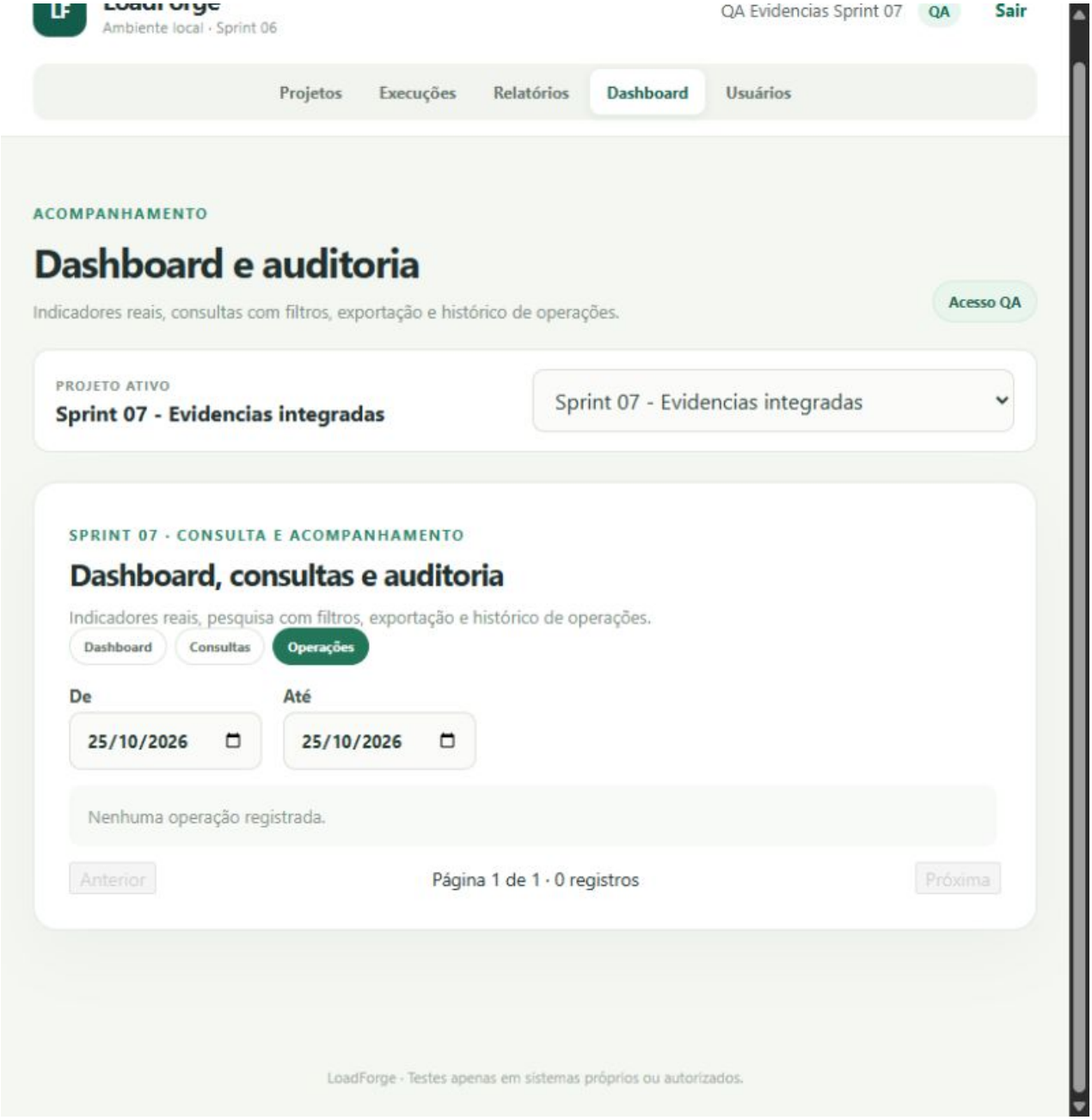


Figura S7-06. Print real do histórico filtrado para 25/10/2026: nenhum registro e página 1 de 1. A combinação com a Figura S7-05 demonstra que o período altera a consulta efetivamente.

54. Integração com os módulos anteriores

O fluxo demonstrado percorreu cadastro de projeto e endpoint autorizado, configuração dos cenários, execução pelo motor, persistência das janelas, geração dos relatórios, comparação e consulta no dashboard. As pesquisas e exportações reutilizaram essas mesmas execuções; a auditoria registrou as ações e mudanças de situação.

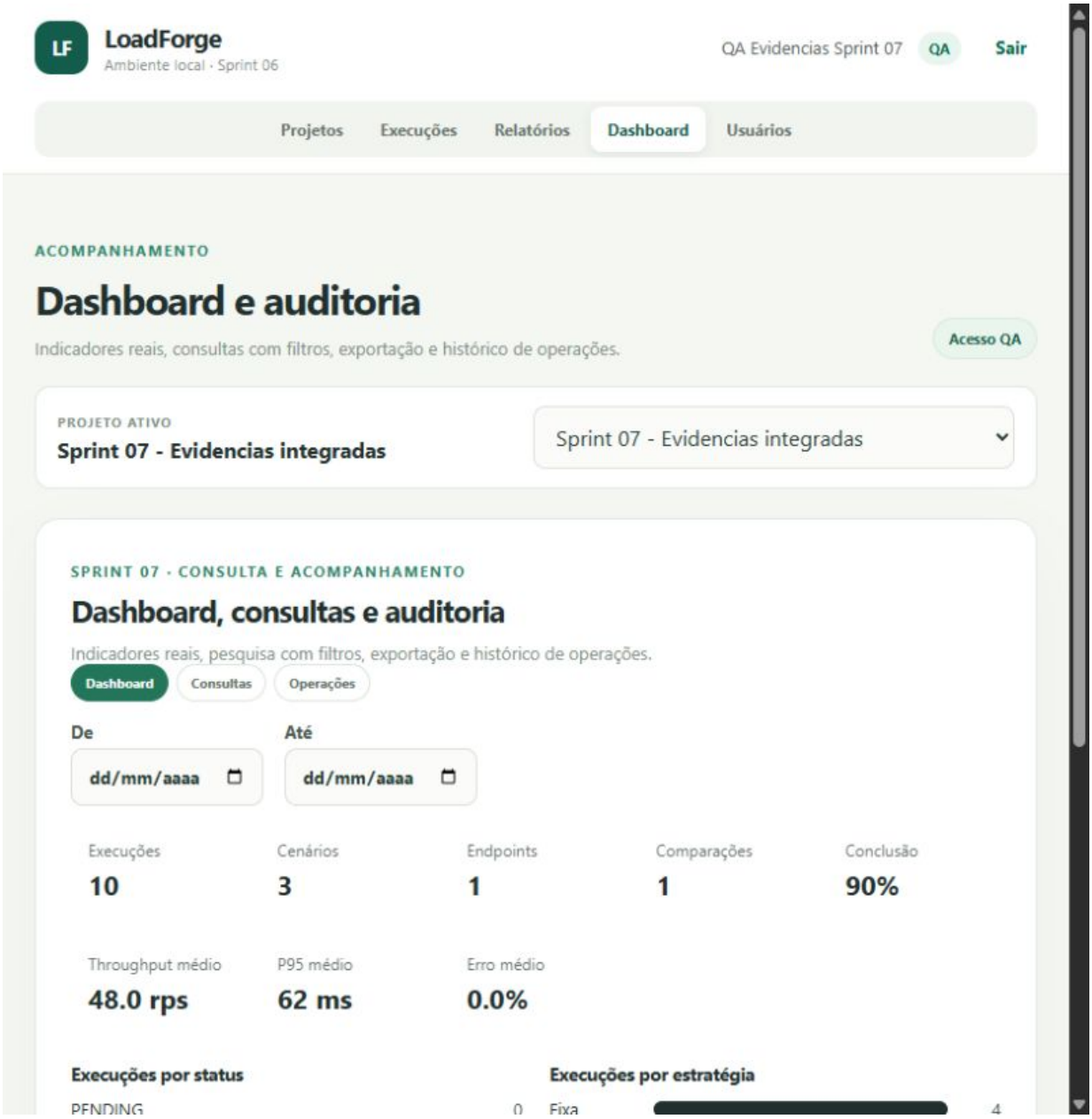


Figura S7-07. Print real do contexto do projeto e dos indicadores calculados a partir das execuções do motor. A navegação mantém acesso a Projetos, Execuções, Relatórios e Dashboard.

A interface ainda contém alguns rótulos herdados “Sprint 06”. A versão demonstrada corresponde à revisão final da Sprint 07 identificada na seção 48. A integração foi verificada com dados persistidos no PostgreSQL do ambiente isolado.

55. Verificação, dificuldades e próximos passos

Verificações realizadas. A versão final do PR apresentou sucesso nos trabalhos de integração contínua de backend com PostgreSQL, compilação do frontend e construção das imagens Docker. Também foi compilado o frontend usado nas capturas e exercitado o fluxo real de criação, execução, cancelamento, relatórios, comparação, consulta, auditoria e quatro exportações.

Dificuldades e cuidados registrados. A revisão da Sprint 07 exigiu alinhar filtros de datas e fusos, rotas de exportação, seleção de registros e rastreamento das transições do motor. Os ajustes constam da versão final do PR. A preparação das evidências exigiu um ambiente isolado com dados de demonstração, evitando misturar o histórico anterior com os ensaios desta entrega.

A amostra foi concentrada em um único dia e em um alvo local estável. Isso limita a evidência de evolução temporal e de diferenças de desempenho. A ausência de um modelo aprovado acionou o fallback da estratégia híbrida; o documento registra esse comportamento sem atribuir ganho à IA.

Próximos passos. Ampliar a campanha experimental com diferentes intensidades de carga e repetições mais longas; usar um modelo aprovado e auditar sua participação; acompanhar recall e a meta de 0,80 com teste reservado; produzir séries em vários dias; padronizar os rótulos de Sprint na interface e melhorar a apresentação do contexto e das ações de exportação.

Repositório atualizado
https://github.com/camimcl/sre-fabrica-de-software

Entrega e histórico de alterações
https://github.com/camimcl/sre-fabrica-de-software/pull/6

Verificações automatizadas da revisão
Execução de CI nº 39995799535

Registro preparado em 24/10/2026. Os prints desta seção representam a aplicação em funcionamento e as reproduções dos relatórios foram extraídas dos arquivos produzidos por ela.

56. Sprint 08: funcionalidades concluídas

A Sprint 08 consolida o LoadForge na versão 0.8.0, completa o gerenciamento de cenários pela interface e integra as exportações de comparação. A entrega também revisa permissões, navegação, validações e auditoria, utilizando os módulos persistidos das Sprints anteriores.

Área	Funcionalidade disponível
Autenticação e contas	Cadastro como Visualizador, login, perfil próprio e administração por QA.
Projetos e alvos	Cadastro e manutenção de projetos e endpoints com autorização registrada.
Cenários e execução	Criar, consultar, editar e excluir cenários; ajustar timeout; iniciar carga real e interromper pelo controle emergencial.
Inteligência e controle	Preparação temporal, treino e aprovação de modelos, inferência local e fallback por regras.
Análise e acompanhamento	Métricas, dashboard, pesquisa, filtros, histórico, relatórios individuais e comparação de estratégias.
Exportação e rastreabilidade	CSV/PDF integrados à interface e histórico transacional das principais operações.

Versão documentada. Branch **sprint-08-consolidacao-permissoes-usabilidade**, com 17 commits publicados; revisão **9114be3**. As capturas foram produzidas em 02/11/2026 com essa revisão, em banco e alvo HTTP locais separados dos serviços de desenvolvimento.

As 69 páginas recebidas foram mantidas integralmente. Este complemento dá continuidade à numeração do documento e reúne prints reais, resultados observados, regras e limites da demonstração.

Roteiro demonstrado: projeto e alvo autorizado → cenário → execução e métricas → parada → relatório/comparação → exportação → dashboard/consulta → auditoria. O mesmo projeto foi acessado como QA proprietário, Visualizador e QA não proprietário.

57. Evidência do fluxo integrado: projeto e alvo

O projeto “Sprint 08 - Fluxo integrado” reúne um endpoint com autorização registrada e os cenários utilizados na demonstração. O QA proprietário acessa os controles de edição do projeto e do endpoint; a navegação superior permite seguir para Execuções, Relatórios e Dashboard.

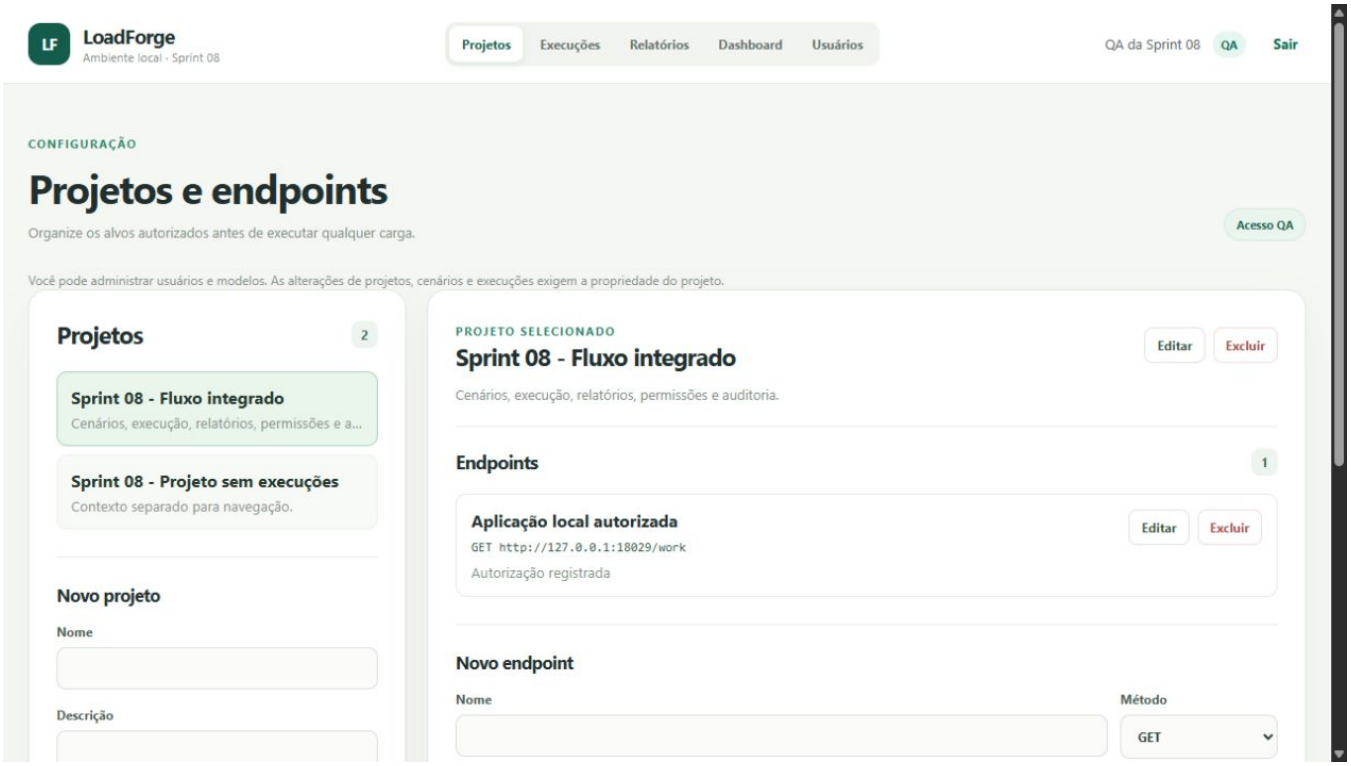


Figura S8-01. Print real do QA proprietário: projeto selecionado, aplicação local autorizada e controles de manutenção disponíveis.

Etapa	Resultado observado
Preparação	Um endpoint local autorizado, três cenários comparáveis e cenários adicionais de edição/monitoramento.
Carga comparativa	Nove execuções concluídas: três por FIXED, RULES e AI_HYBRID, com 1.772 requisições reais.
Integração	Relatórios e comparação gerados com as métricas persistidas pelo motor; consulta, dashboard e exportações reutilizam os mesmos dados.

O alvo responde em ambiente local e foi criado para esta demonstração. Os ensaios curtos comprovam integração funcional; seus resultados não representam capacidade de produção nem uma campanha experimental completa.

58. Gestão de cenários e usabilidade do formulário

A interface passou a oferecer edição, exclusão com confirmação e cancelamento da edição, além do campo de timeout. Na demonstração, o cenário foi renomeado e seu timeout alterado de 1.000 para 2.500 ms; após salvar, o formulário foi reaberto com os valores persistidos.



Figura S8-02. Recorte do print real do cenário reaberto: nome “S8 - Cenário revisado” e timeout de 2.500 ms. A explicação informa que a edição vale para novas execuções.

Campo	Antes	Após salvar e reabrir
Nome	S8 - Cenário para edição	S8 - Cenário revisado
Timeout	1.000 ms	2.500 ms

Os testes verificam que editar um cenário não altera o snapshot de execuções existentes. A exclusão permanece condicionada à ausência de execuções dependentes, preservando o histórico.

59. Execução real, métricas e parada emergencial

O cenário “S8 - Monitor e parada” foi iniciado pela interface após marcar a confirmação de autorização. O motor produziu seis janelas de métricas e decisões de controle. Ao acionar a parada emergencial, a execução **d545daa3** terminou com situação **CANCELLED**.

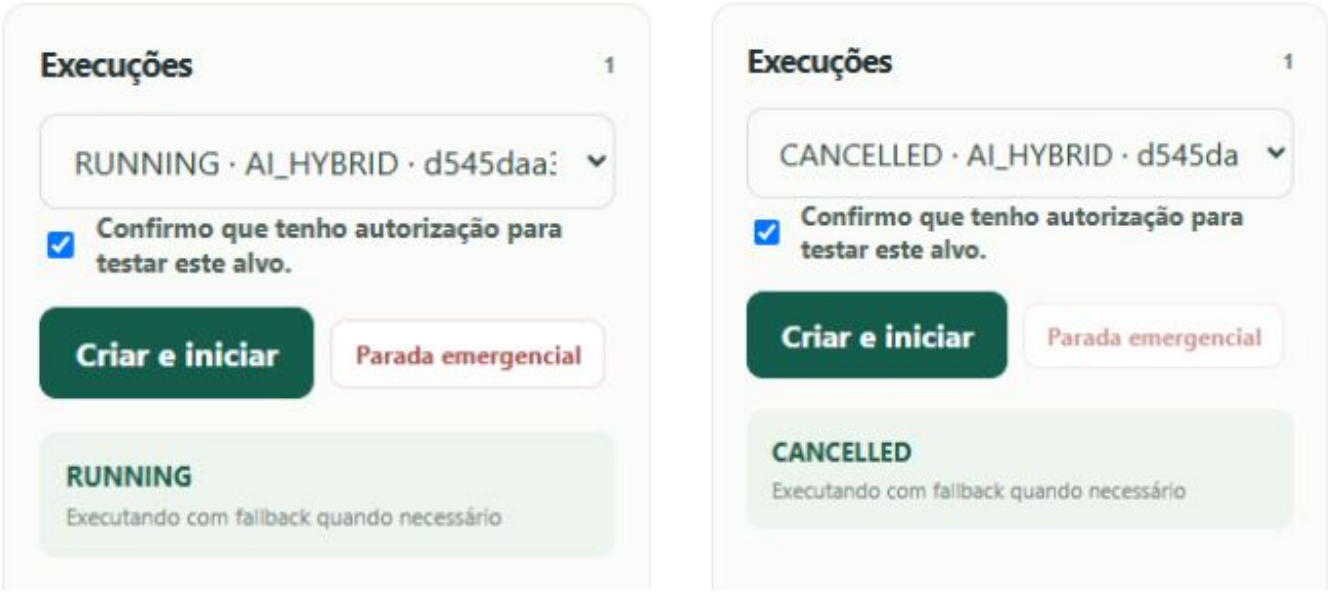


Figura S8-03. Recortes reais do mesmo fluxo: à esquerda, RUNNING e parada disponível; à direita, CANCELLED e parada desabilitada após o encerramento.

Janela	Concorrência	Throughput	P95	Decisão
1	2	39.9 rps	63 ms	INCREASE
2	3	57.6 rps	63 ms	INCREASE
3	4	73.9 rps	63 ms	HOLD
4	4	70.9 rps	63 ms	HOLD
5	4	75.0 rps	63 ms	HOLD
6	4	71.1 rps	63 ms	HOLD

Valores recuperados da API após a parada. A concorrência evoluiu de 2 para 3 e 4, mantendo os registros disponíveis para consulta. Sem modelo aprovado neste ambiente, o controle híbrido usou fallback por regras; isso não comprova ganho da IA.

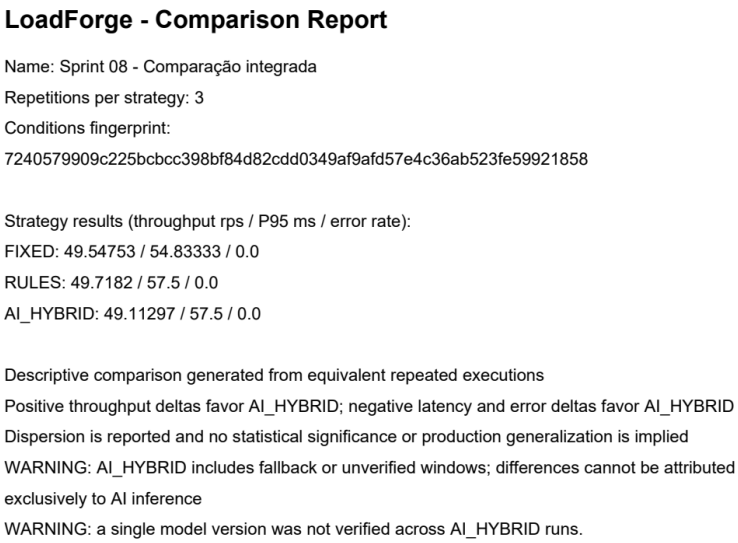
60. Relatórios e exportações integradas

A comparação “Sprint 08 - Comparação integrada” reúne três repetições de cada estratégia, sob condições equivalentes. A interface apresenta médias, dispersão, conformidade e botões para exportar o resultado em CSV e PDF.



Figura S8-04. Recorte real dos resultados comparativos e dos novos botões de exportação junto à comparação selecionada.

O botão “Exportar comparação PDF” foi acionado e o arquivo baixado foi aberto. Seu conteúdo textual foi conferido com a exportação da API. CSV de comparação, PDF de comparação e CSV de execuções também retornaram HTTP 200.


LoadForge - Comparison Report

Name: Sprint 08 - Comparação integrada
Repetitions per strategy: 3
Conditions fingerprint:
7240579909c225bcbcc398bf84d82cdd0349af9afd57e4c36ab523fe59921858

Strategy results (throughput rps / P95 ms / error rate):
FIXED: 49.54753 / 54.83333 / 0.0
RULES: 49.7182 / 57.5 / 0.0
AI_HYBRID: 49.11297 / 57.5 / 0.0

Descriptive comparison generated from equivalent repeated executions
Positive throughput deltas favor AI_HYBRID; negative latency and error deltas favor AI_HYBRID
Dispersion is reported and no statistical significance or production generalization is implied
WARNING: AI_HYBRID includes fallback or unverified windows; differences cannot be attributed exclusively to AI inference
WARNING: a single model version was not verified across AI_HYBRID runs.

Figura S8-05. Reprodução do PDF efetivamente baixado pela interface, com os resultados das três estratégias.

A amostra apresentou aproximadamente 49,5 rps em FIXED, 49,7 rps em RULES e 49,1 rps em AI_HYBRID. As seis janelas híbridas da comparação usaram fallback. As diferenças observadas não podem ser atribuídas a inferência de IA.

61. Pesquisas, filtros e consultas organizadas

A pesquisa por cenário foi combinada com período, situação e estratégia. No exemplo, o texto “IA”, o dia 02/11/2026, a situação COMPLETED e a estratégia IA híbrida retornaram as três execuções correspondentes. A exportação utiliza os critérios da consulta.

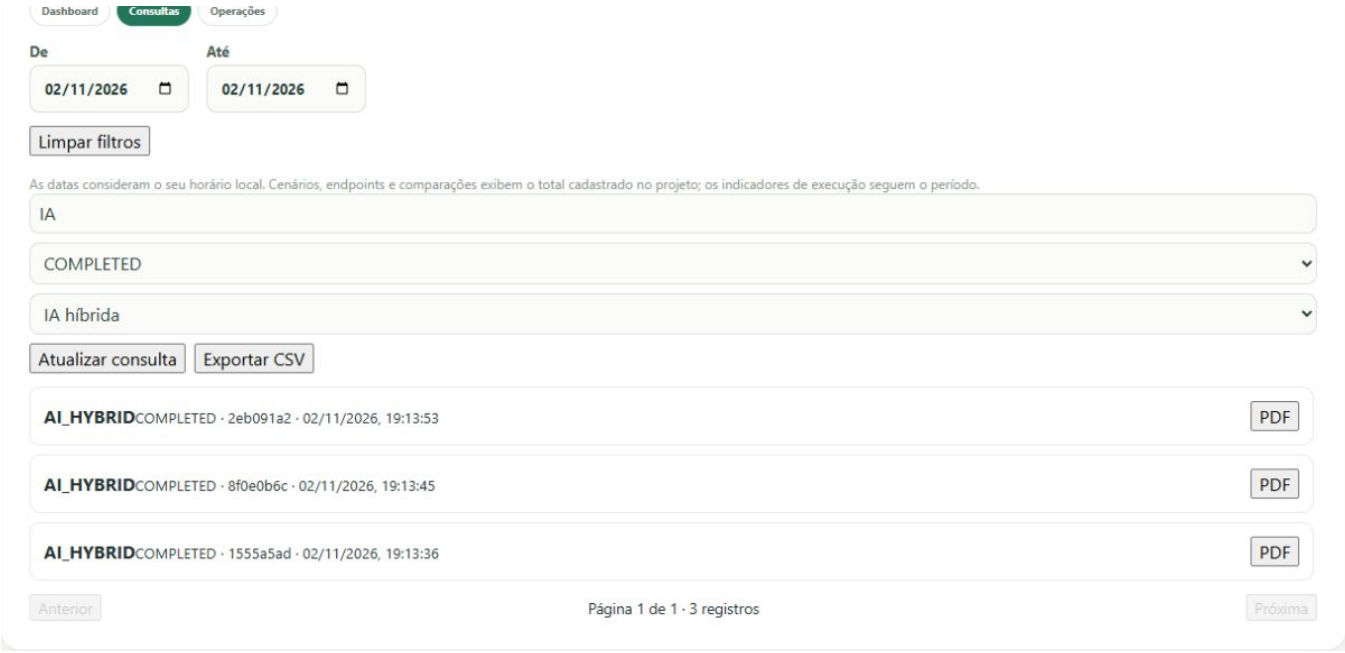


Figura S8-06. Print real dos critérios aplicados: três registros e uma página, com exportação CSV e acesso ao PDF individual.

Melhoria	Comportamento
Limpar filtros	Reinicia período, pesquisa, situação, estratégia e paginação.
Datas locais	Converte o início e fim do dia local para UTC; inclui o último milissegundo do dia.
Navegação	Descarta respostas antigas quando o contexto muda e limpa formulários ao trocar de projeto.
Downloads individuais	Execuções pendentes/em andamento não oferecem PDF ativo. Quando falta um relatório, a mensagem orienta a solicitar sua geração ao QA proprietário.

A paginação informa a quantidade de registros. As datas da interface são locais; o agrupamento diário do gráfico é identificado como UTC. Esse esclarecimento evita interpretar horários e agrupamentos como se fossem a mesma referência.

62. Controle de permissões: Visualizador e proprietário

O ambiente do MVP é compartilhado: usuários autenticados consultam projetos e resultados. O papel e a propriedade determinam quem pode alterar dados e executar carga. O Visualizador mantém acesso à própria conta, sem poder promover seu perfil.

Operação	Visualizador	QA não proprietário	QA proprietário
Consultar dados e exportar resultados existentes	Permitido	Permitido	Permitido
Alterar projeto, endpoint ou cenário	Negado	Negado	Permitido
Executar carga e gerar relatórios/comparações	Negado	Negado	Permitido, com regras
Administrar usuários/modelos e consultar auditoria	Negado	Permitido	Permitido

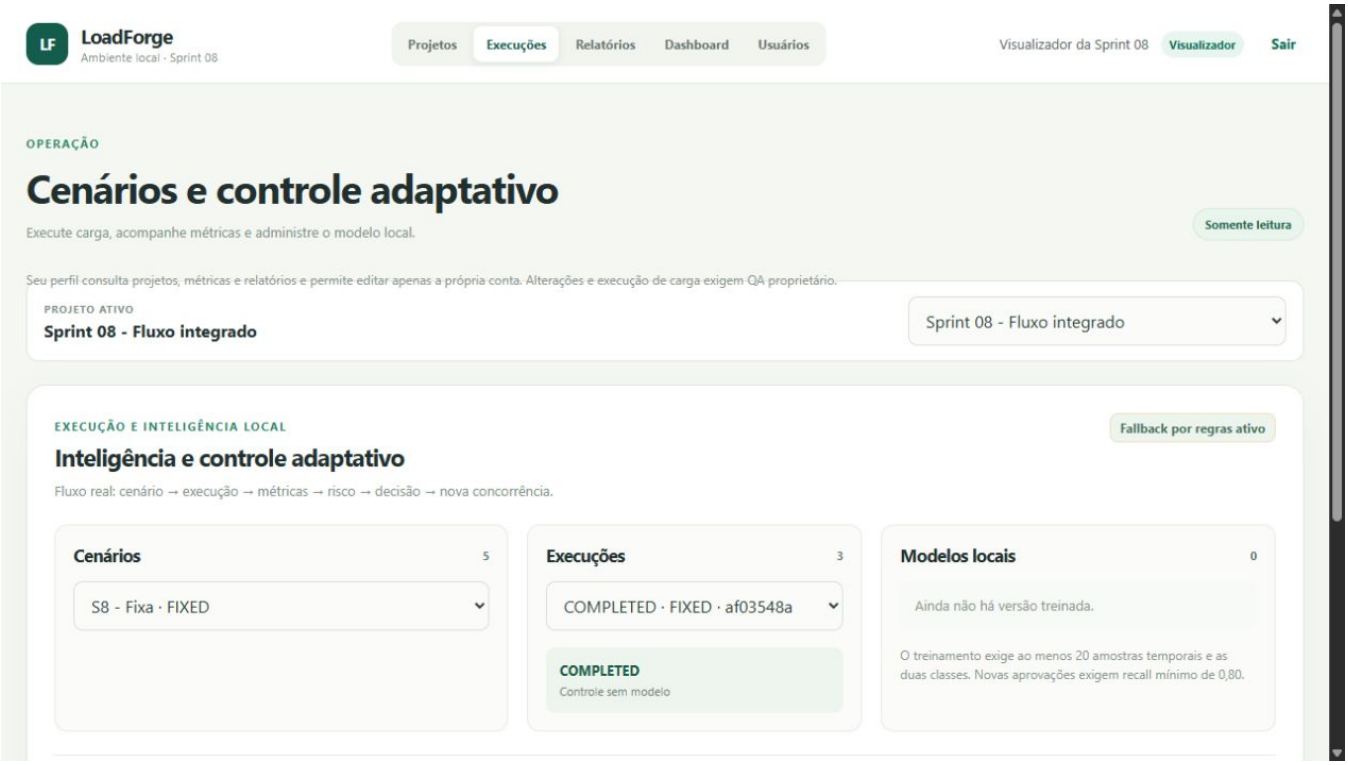


Figura S8-07. Print real autenticado como Visualizador: consulta dos cenários e resultados, com aviso “Somente leitura” e sem os controles de criação, edição, carga ou treinamento.

Em chamadas diretas à API, o Visualizador recebeu HTTP 200 ao consultar o dashboard e HTTP 403 ao tentar alterar o projeto ou consultar a auditoria. A restrição é aplicada no backend, além da apresentação da interface.

62.1. QA não proprietário e administração do ambiente

O QA de consulta acessou o mesmo projeto, pertencente a outro usuário. A interface explica que alterações e execuções são exclusivas do proprietário e oculta os controles de manutenção do projeto e do endpoint. Criar um projeto próprio continua permitido.

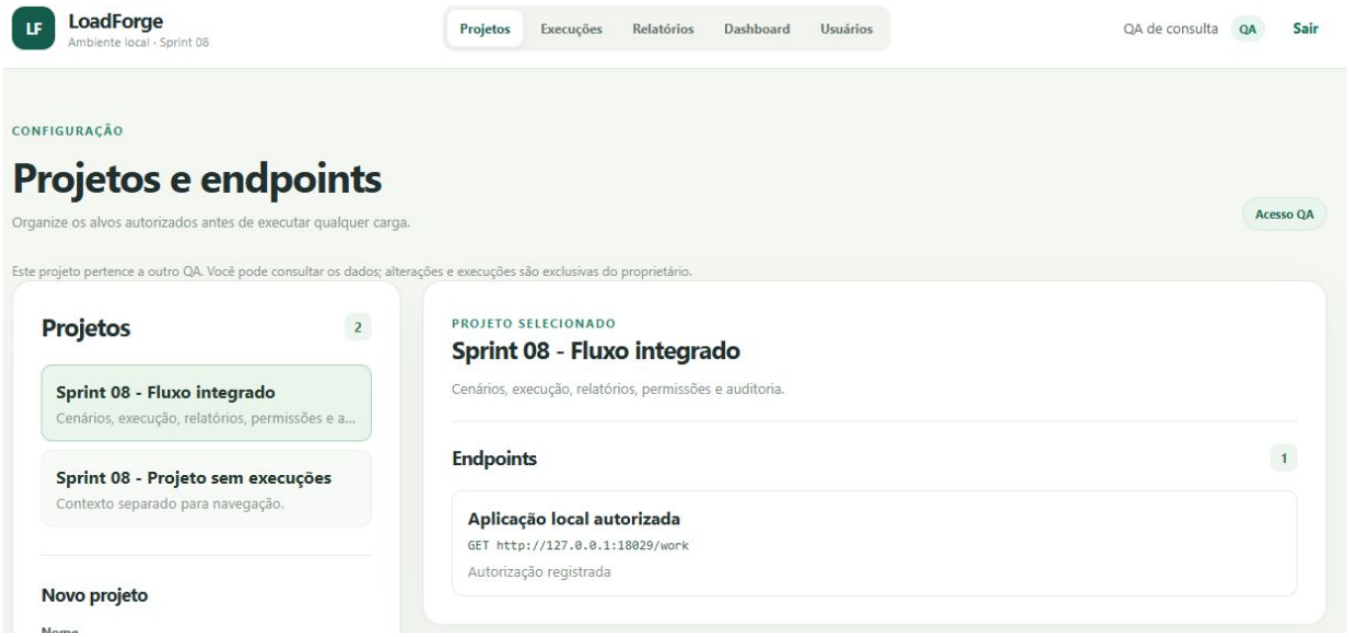


Figura S8-08. Print real do QA não proprietário: consulta permitida e aviso de propriedade, sem ações para alterar o projeto selecionado.

Esse QA continua autorizado a administrar usuários e modelos globais, conforme a regra do ambiente compartilhado. O Visualizador, na mesma área, acessa somente “Meu perfil”.



Figura S8-09. Recorte real da administração QA, com três contas de demonstração e formulário de criação de usuário.

A tentativa direta desse QA de alterar o projeto de outro proprietário recebeu HTTP 403. Os testes também verificam que uma mudança de perfil passa a valer nas requisições seguintes de uma sessão já existente.

63. Dashboard e melhorias de compreensão

O dashboard consolida os dados persistidos do projeto. Após os ensaios e a parada, foram observadas 10 execuções, sendo nove concluídas e uma cancelada, cinco cenários, um endpoint e uma comparação. A taxa de conclusão corresponde a 90%.



Figura S8-10. Print real do dashboard: indicadores, distribuições, ponto visível para um único dia e valor diário também apresentado em texto.

Ajuste de usabilidade	Contribuição
Ponto para uma data	O gráfico deixa de parecer vazio quando há um único dia na série.
Valores em texto	Permitem consultar a contagem sem depender apenas da representação gráfica.
Escopo dos indicadores	A interface informa que totais cadastrais do projeto não seguem o período das execuções.
Orientação de acesso	Mensagens explicam as atribuições do perfil e a necessidade de ser proprietário.

A demonstração concentra-se em um único dia. A série não constitui evidência de tendência temporal em vários dias nem de desempenho em produção.

64. Auditoria e rastreabilidade das alterações

Edição/exclusão de cenários e criação/alteração/exclusão de contas passam a compor o histórico na mesma transação da operação. As transições do motor continuam registradas. A lista é global ao ambiente compartilhado e restrita a QA.

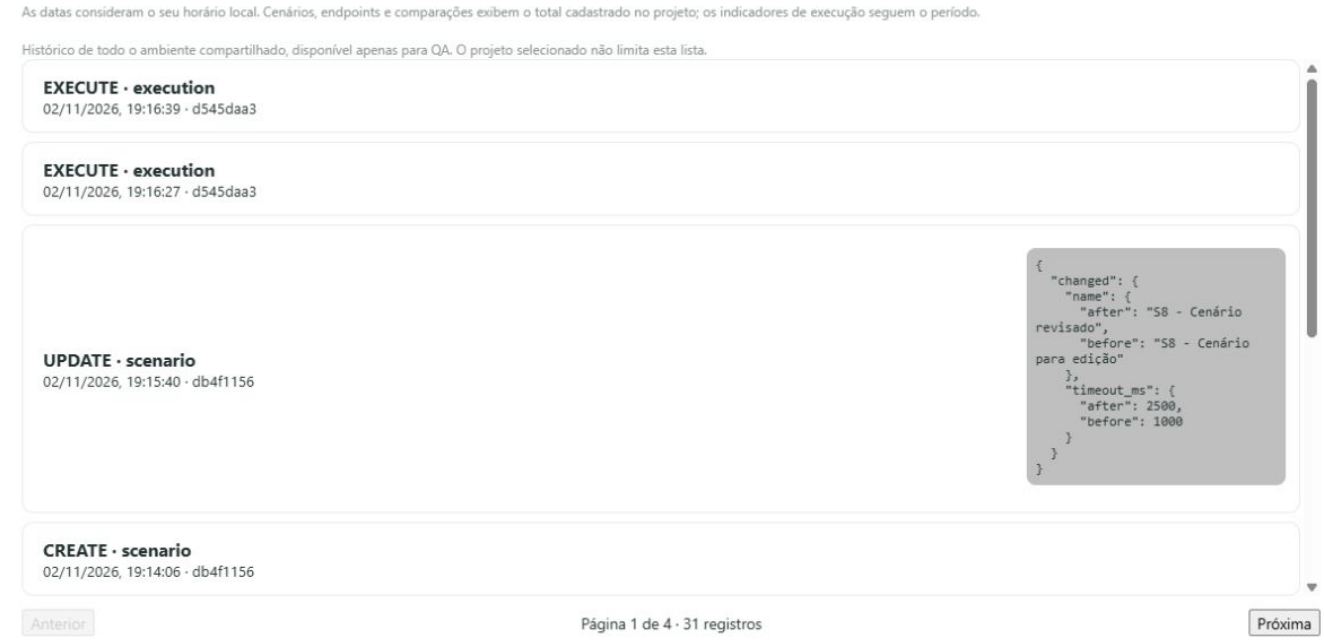


Figura S8-11. Print real da auditoria: início e encerramento da execução e evento UPDATE do cenário, com detalhes da alteração.

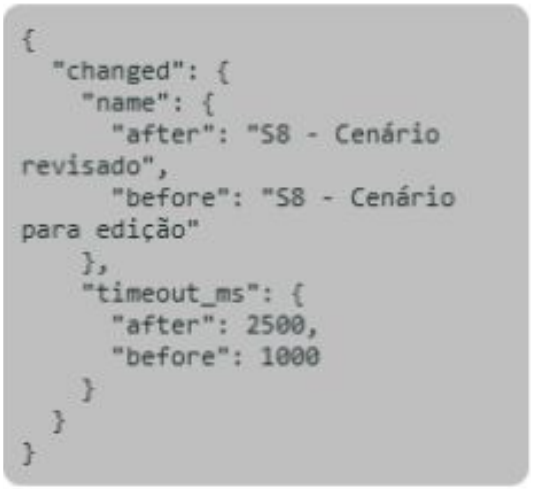


Figura S8-12. Ampliação do detalhe do mesmo print: nome anterior/novo e timeout de 1.000 para 2.500 ms.

O evento corresponde ao cenário editado e reaberto na Figura S8-02. Foram recuperados 31 registros nesta demonstração. Os testes confirmam que um cadastro duplicado não deixa um evento de criação parcial.

Senhas, hashes e tokens não são registrados nesses eventos; a alteração de senha é representada somente por um indicador booleano. Detalhes antigos em texto livre são exibidos sem interromper a tela.

65. Revisão das regras de negócio e do escopo

Regra revisada	Ajuste e evidência
Autorização explícita	authorization_acknowledged deve ser booleano verdadeiro e obrigatório. Ausência, falso e texto foram rejeitados com HTTP 422 na demonstração, sem criar execução.
Autorização atual do alvo	O início continua revalidando o endpoint, sua autorização e a configuração registrada no snapshot.
Histórico imutável	Edição do cenário não reescreve as configurações ou o alvo das execuções anteriores; comportamento coberto por teste.
Preservação do último QA	Bloqueio transacional no PostgreSQL serializa alterações concorrentes. No teste, uma despromoção recebe 200, outra 409 e permanece um QA.
Auditoria atômica	A operação e o evento compartilham a transação; falhas por duplicidade não deixam eventos parciais.
Aprovação de IA	A meta mínima de recall continua exigida para aprovação. Ausência de modelo válido mantém o fallback por regras.

Alterações no escopo. Nenhuma funcionalidade do MVP foi retirada. A Sprint 08 completa a operação dos cenários e exportações na interface e consolida os controles. Execução distribuída, multi-tenancy, IA externa, GPU, aplicativo móvel e cobrança permanecem fora do MVP, conforme o planejamento já registrado.

Adequação do ambiente. O suporte a Python 3.11/3.12 foi explicitado por compatibilidade com NumPy 1.26.4, SciPy 1.13.1 e scikit-learn 1.5.2. Os locks foram sincronizados e o produto identificado como 0.8.0. O Vite foi atualizado para 7.3.7 após o diagnóstico de dependências.

Concluir a funcionalidade de IA não significa demonstrar ganho ou atingir a meta experimental de recall. A campanha com modelo aprovado e teste reservado continua necessária; os resultados históricos não foram reescritos.

66. Verificação final, dificuldades e próximos passos

Verificação registrada em 02/11/2026	Resultado
Backend com PostgreSQL 16 e dependências do lock	125 testes aprovados; zero falhas e zero skips.
Frontend	Quatro testes aprovados e build TypeScript/Vite concluído.
Ambiente Docker	Imagens da API e do frontend construídas com sucesso.
Auditoria npm	Zero vulnerabilidades reportadas após a atualização do Vite.
Demonstração visual complementar	Edição persistida, carga e parada, relatórios/exportação, consultas, dashboard, perfis e auditoria capturados em funcionamento.

Dificuldades. Foi necessário alinhar a base local com a Sprint 07, uniformizar as dependências do ambiente de testes e tratar concorrência na preservação do último QA. A organização dos prints exigiu dados de demonstração separados dos históricos anteriores. Permanecem oito avisos de depreciação nas bibliotecas de testes, sem falhas.

Próximos passos. Realizar revisão humana da branch e sua integração por PR; ampliar os ensaios com diferentes cargas e modelos aprovados; medir recall e antecipação em teste reservado; validar usabilidade com usuários e diferentes resoluções; refinar os estados de carregamento ao aplicar um intervalo inválido.

Repositório atualizado
https://github.com/camimcl/sre-fabrica-de-software

Branch publicada da Sprint 08
sprint-08-consolidacao-permissoes-usabilidade

Referência da entrega: revisão 9114be3, com 17 commits. A documentação técnica, a matriz de acesso e as evidências automatizadas encontram-se em docs/relatorio-sprint-08.md e docs/evidences/sprint-08/. A publicação da branch não equivale à integração em main.

67. Sprint 09 - verificação completa do sistema

Grupo 21 | LoadForge | Turma CC8NB
Registro: 06/11/2026. Prazo de entrega: 07/11/2026.

Como QA e PO, organizei a verificação dos módulos existentes por risco: acesso, cadastros, execução, controle, relatórios e continuidade da navegação. Registrei os resultados de testes automatizados e capturas da aplicação real, com reprodução e reteste dos problemas encontrados.

Integrante	Responsabilidade de referência
Camile Marcele Pereira de Araújo	QA/PO: plano, matriz, bugs, evidências e consolidação
Aline Bianca Arantes da Silva	Backend: contratos e regras de negócio
Emerson Wallace Barcelos de Araújo	Scrum Master/backend: coordenação e envio único
Ricardo Cezar Ottoni Assis de Almeida	Dados/infraestrutura: persistência e modelos
Cayo Vitor Fagundes	Frontend/UI: navegação, formulários e mensagens

A divisão orienta revisão e apresentação. Os percursos aqui registrados foram executados por automação; não representam atribuição de execução manual individual.

O documento preserva integralmente as páginas 1-81 das Sprints 01-08. Este complemento começa na seção 67. A base analisada foi a main **f9a62fb**, com a Sprint 08 integrada. As novas alterações estão organizadas em branches locais por tarefa.

Repositório: https://github.com/camimcl/sre-fabrica-de-software
Protótipo: Figma - LoadForge

O protótipo é uma referência de interface sujeita à evolução. As imagens desta sprint mostram a aplicação implementada, executada com API e PostgreSQL.

68. Estratégia e resultados da verificação

Utilizei laboratório próprio com PostgreSQL 16, API Python 3.11/FastAPI e React/Nginx. Separei banco de demonstração e banco de testes. O Edge executou o percurso principal em 1280x960, com fuso America/Fortaleza. As requisições de carga foram dirigidas somente ao alvo local autorizado.

Verificação	Resultado obtido
Backend, incluindo PostgreSQL e migrations	167 aprovados; zero falhas e zero ignorados; 91,64 s.
Frontend: filtros, downloads e erros HTTP	11 aprovados; zero falhas e zero ignorados.
Percurso integrado na aplicação	17 verificações aprovadas; capturas e esperado/obtido registrados.
Respostas atrasadas: projeto e dashboard	Dois reprodutores falharam antes e passaram após correção.
Persistência após reinício da API	11 execuções e uma comparação recuperadas por novo login.
Compilação e ambiente	Build TypeScript/Vite e imagens API/frontend concluídos; serviços saudáveis.

Rastreabilidade. O JUnit da regressão final referencia a revisão funcional **7c2c87c**. Registros de persistência e documentação foram acrescentados depois. Há oito avisos de depreciação nas bibliotecas de testes; não houve falha nem skip nessa rodada.

A primeira rodada teve 152 aprovações e uma falha no contexto novo SQLite, que não ativava chaves estrangeiras. Corrigi a preparação do teste e confirmei o bloqueio de dependências também no PostgreSQL. A ocorrência é registrada como ajuste de teste, sem atribuí-la ao esquema produtivo.

Evidências primárias: docs/evidences/sprint-09/pytest-final.xml, pytest-final.txt, frontend-after.txt, results.json e persistence.json. As datas brutas estão em UTC; as execuções às 01h UTC de 07/11 correspondem às 22h de 06/11 em Fortaleza.

69. Casos funcionais e resultados esperados/obtidos

Caso / procedimento	Esperado e resultado obtido
NAV-01/09/10: login, usuários e sessão	Login inválido recusado; CRUD QA persistido; recarregar mantém sessão e sair remove token. Aprovado.
NAV-02 / VAL-UI-01: projetos e endpoints	Criar/editar/recuperar e exigir URL/evidência válida. Cadastros persistidos e formulários inválidos bloqueados. Aprovado.
VAL-UI-02 / NAV-03: cenário e início	Inicial maior que máxima impede salvar; cancelamento de edição conserva dados; confirmação exigida para iniciar. Aprovado.
NAV-03/06: motor e parada	Carga real conclui com métricas; parada finaliza como CANCELLED. Ambos persistidos. Aprovado.
VAL-API-01: exclusões dependentes	Projeto, endpoint e cenário com dependências recebem 409 e permanecem consultáveis. Aprovado no PostgreSQL.
NAV-04: treino sem base suficiente	Mensagem de insuficiência de amostras/classes e ausência de aprovação indevida. Aprovado.
NAV-05 / FUNC-HTTP-01: relatórios	Gerar relatório, executar 3x3 estratégias e baixar PDF válido. Comparação e arquivos produzidos. Aprovado.
NAV-07/08: consultas e auditoria	Filtrar três execuções FIXED, exportar filtro e expandir detalhes de operação. Aprovado.
NAV-11/12/13: perfis e estabilidade	VIEWER e QA não proprietário consultam sem mutações alheias; API 403; nenhuma exceção JavaScript. Aprovado.

Os 167 casos individuais de backend, seus nomes e tempos estão no CSV casos-backend.csv e no JUnit. O CSV casos-navegacao.csv registra esperado e obtido dos 17 casos integrados. Os testes existentes cobrem ainda estados inválidos, snapshots, propriedade, concorrência do último QA, pipeline temporal de IA e hashes de artefatos.

70. Evidências funcionais: cadastro e execução

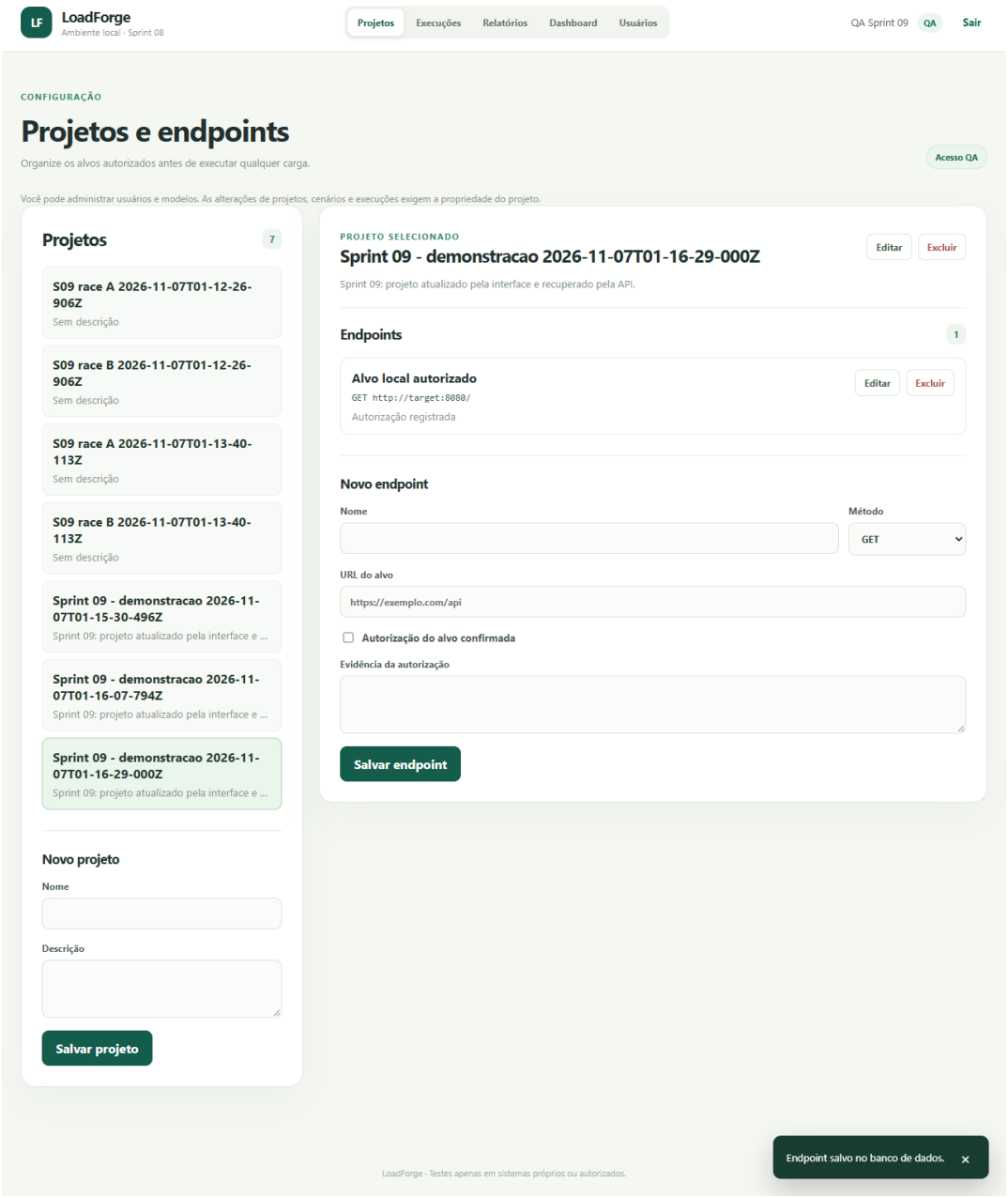


Figura 1 - NAV-02/VAL-UI-01. Criei projeto, atualizei a descrição e cadastrei alvo autorizado. Consultei os registros pela API; recarregar conservou os dados.

Evidências de execução - continuação

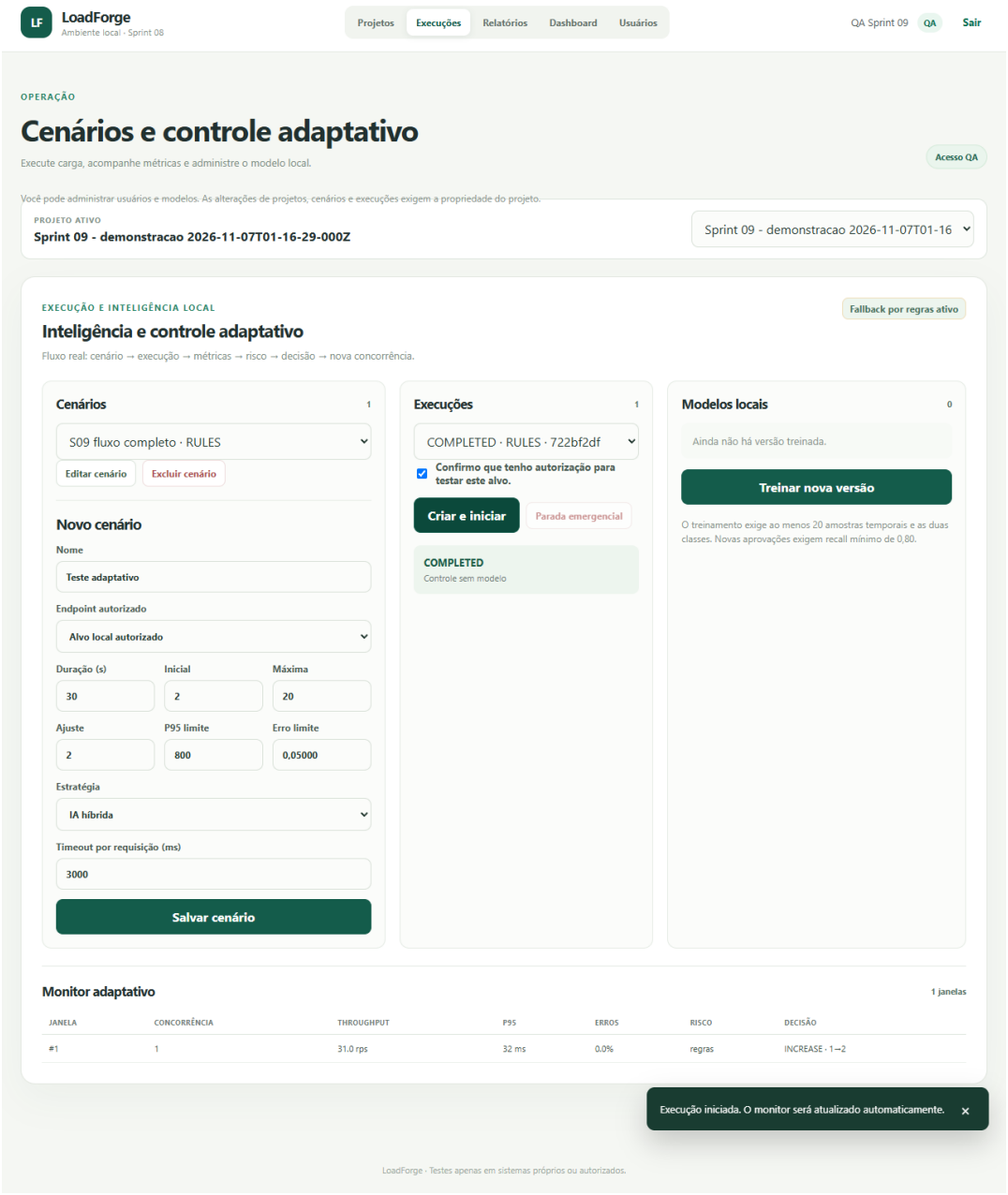


Figura 2 - NAV-03. Configurei cenário de 2 s, confirmei autorização e iniciei o motor. O estado COMPLETED e as janelas com request_count positivo foram recuperados do PostgreSQL.

71. Evidências funcionais: parada e comparação

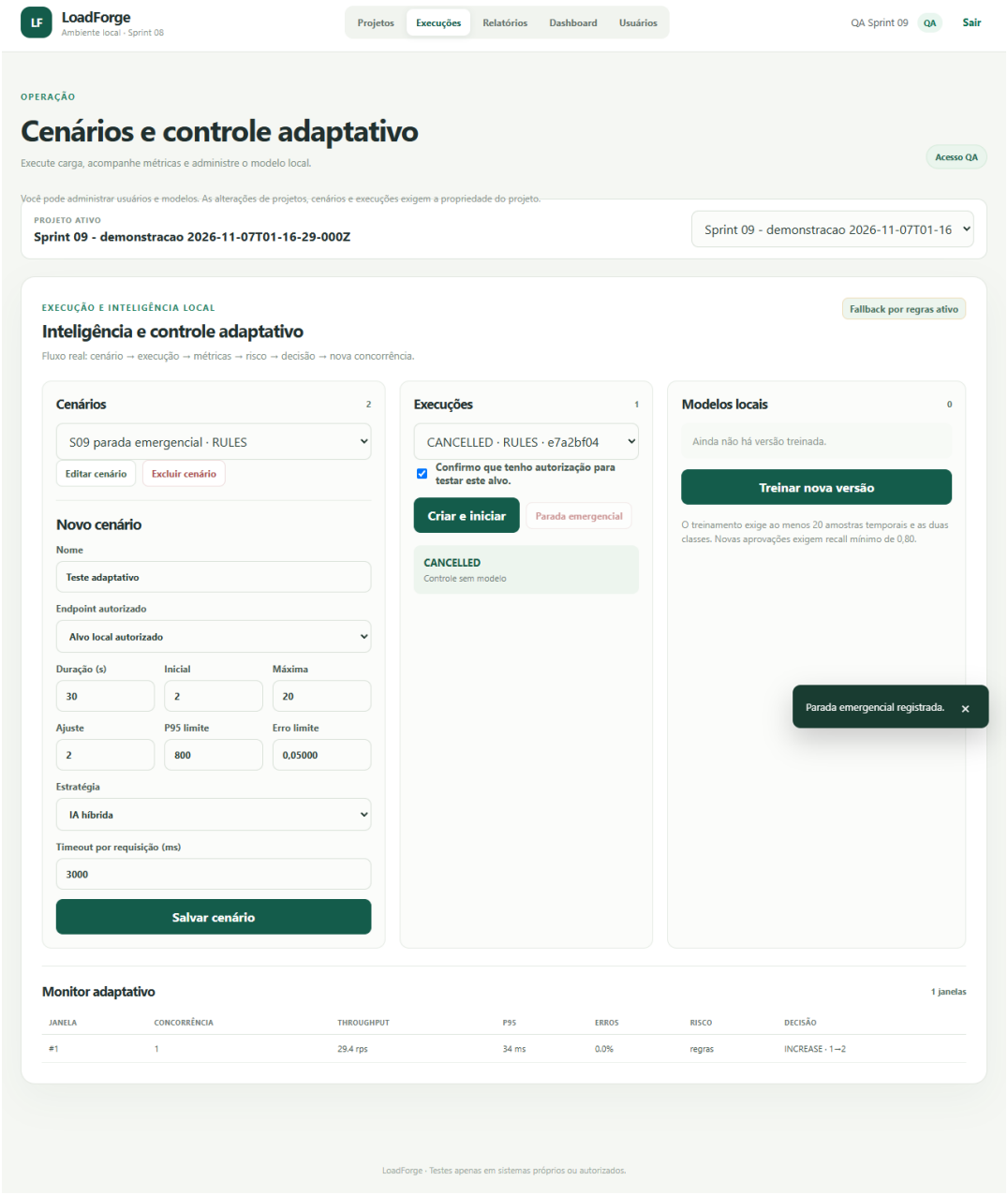


Figura 3 - NAV-06. Iniciei cenário de 30 s e acionei Parada emergencial durante RUNNING. Aguardei CANCELLED e confirmei o mesmo estado pela API.

Evidências de execução - continuação

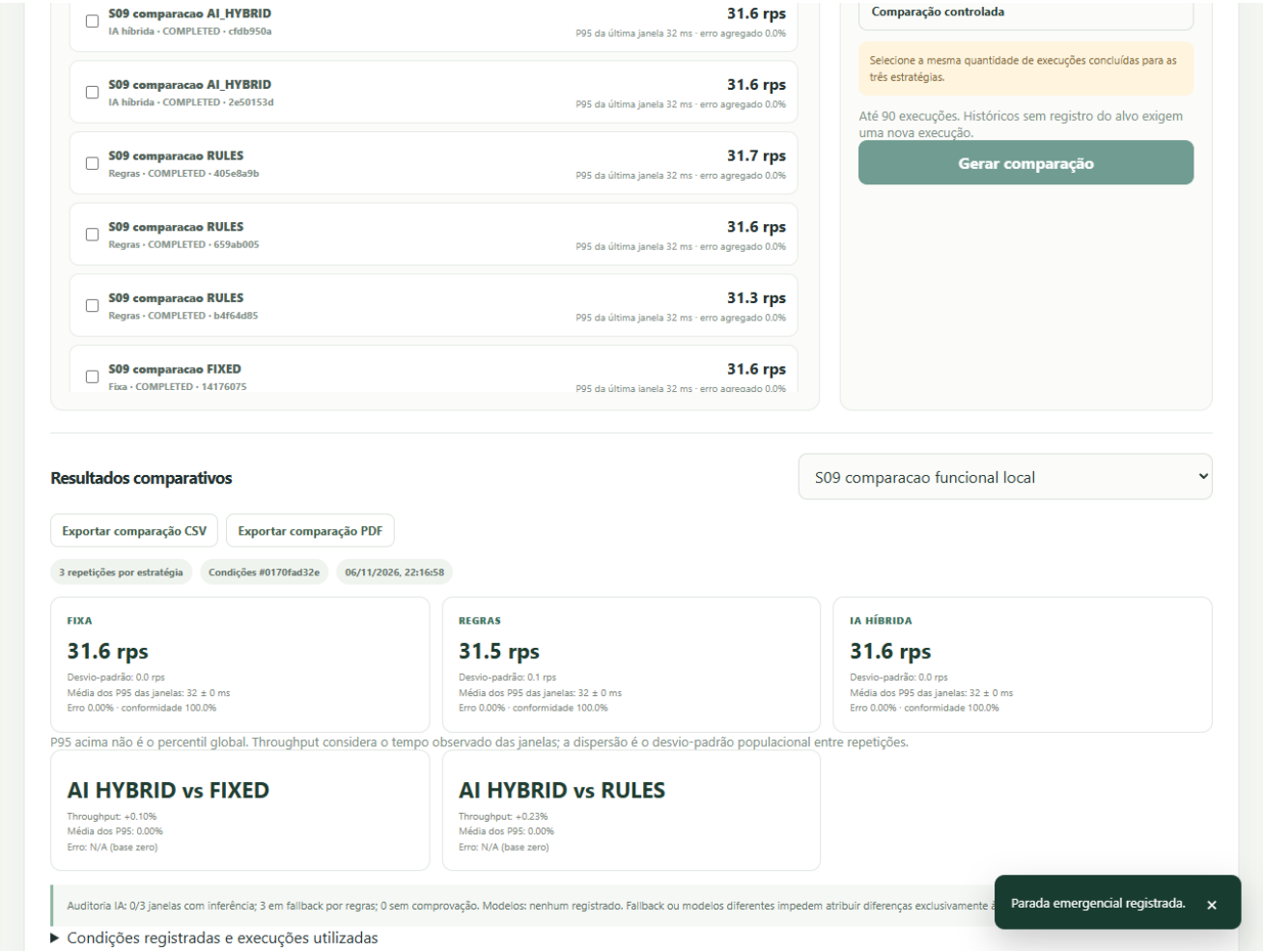


Figura 4 - FUNC-HTTP-01. Executei três repetições de FIXED, RULES e AI_HYBRID com alvo/parâmetros equivalentes. A comparação foi persistida e seu PDF baixado. Sem modelo aprovado neste laboratório, AI_HYBRID usou fallback por regras; não atribuo as diferenças exclusivamente à IA.

72. Evidências de integração e recuperação de dados

Procedimento	Resultado obtido
Consulta SQL independente	13 tabelas de domínio e alembic_version; migration 20261020_0007.
Descarte da conexão e reinício apenas da API	Serviço pronto; novo login autenticado.
GET do projeto após reinício	Projeto e nome recuperados com HTTP 200.
GET dashboard do mesmo projeto	11 execuções; cinco cenários; um endpoint; uma comparação.
GET comparações após reinício	Uma comparação recuperada; três repetições por estratégia.

O projeto do percurso teve dez execuções concluídas e uma cancelada. As contagens globais do laboratório incluem dados exploratórios e são identificadas separadamente em persistence.json. O procedimento confirma persistência após reinício da API, sem caracterizar restauração de backup ou retomada de carga em andamento.

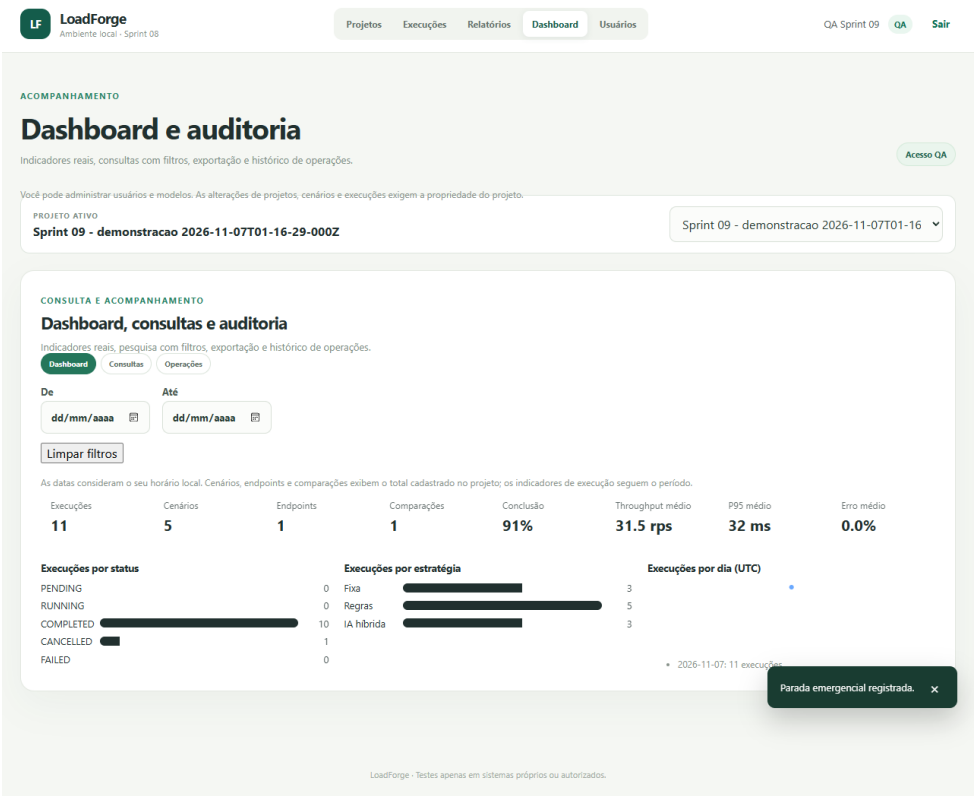


Figura 5 - NAV-07. Indicadores consultados da aplicação para o projeto do percurso. O registro persistence.json comprova a recuperação após reiniciar a API por novo login.

73. Testes de validação e proteção de regras

Os testes HTTP enviaram nomes vazios, URLs inválidas, credenciais/fragmentos na URL, método TRACE, limites fora da faixa, UUID inválido e campos extras. Conferi 422 com explicação e ausência de alteração dos dados anteriores. O novo conjunto inclui 28 casos gerais e 14 casos de autorização explícita.

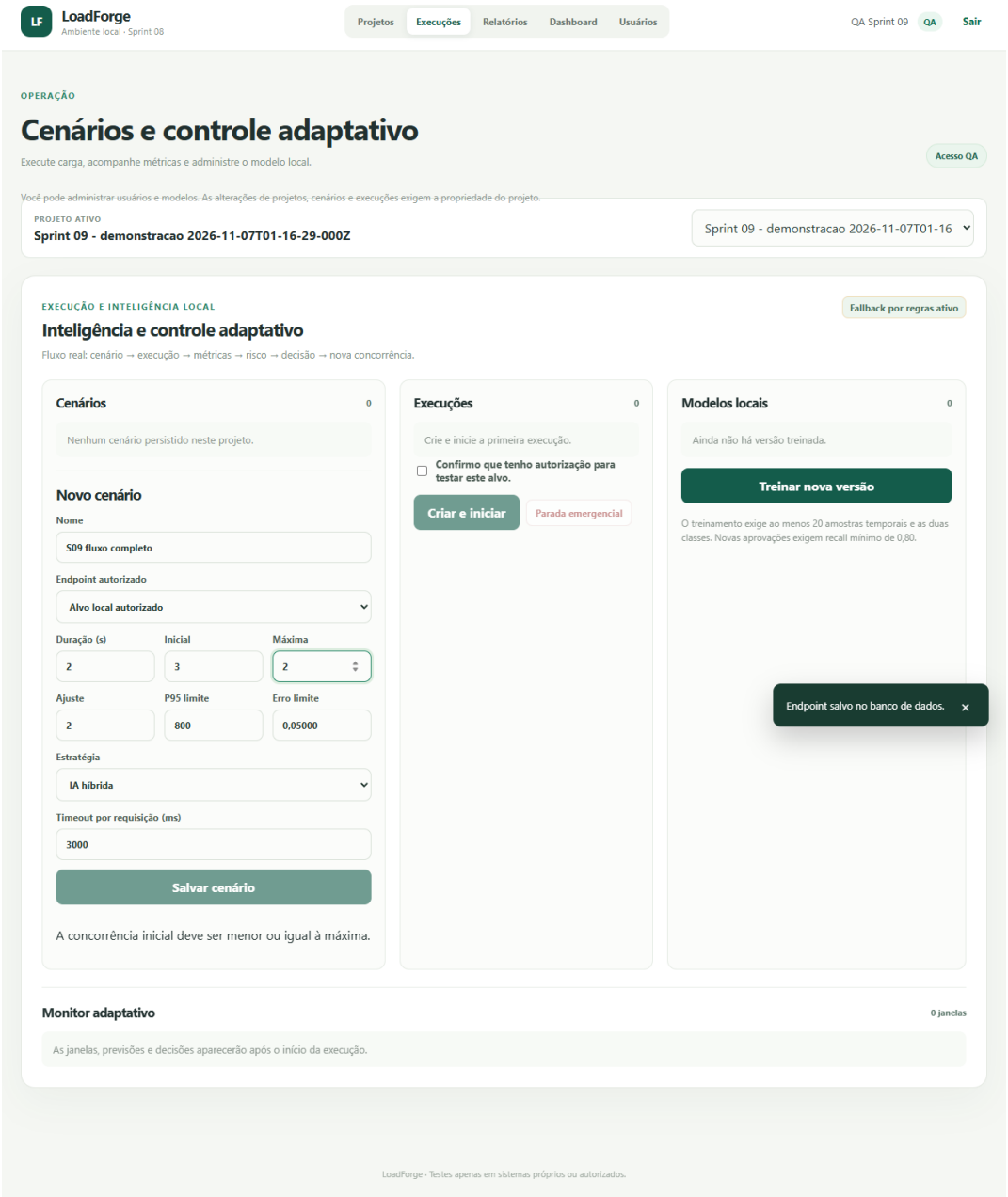


Figura 6 - VAL-UI-02. Inicial 3 e máxima 2 impedem salvar o cenário; após corrigir para 1 e 2, o cadastro foi realizado e recuperado.

Evidências de execução - continuação

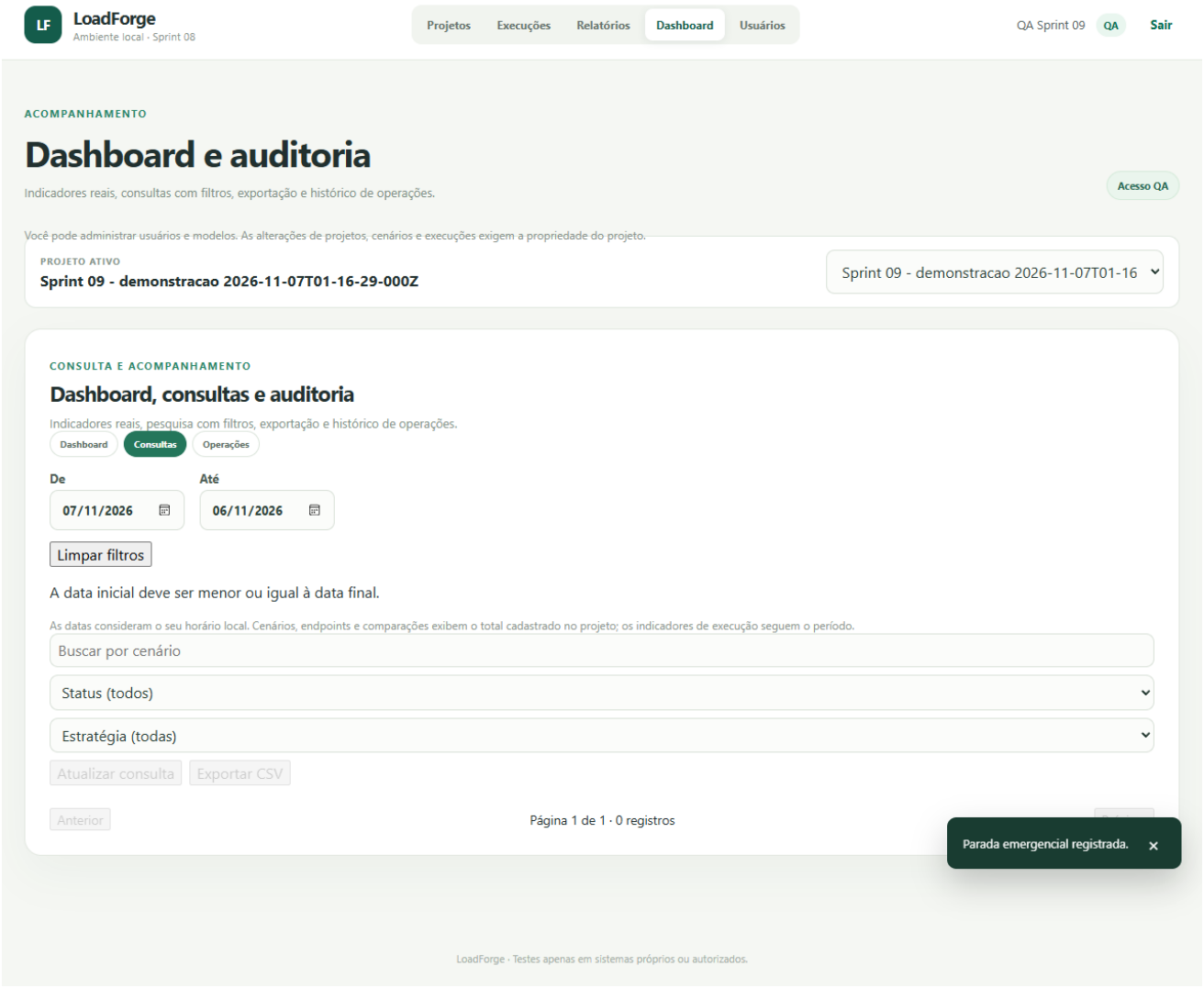


Figura 7 - NAV-07. De posterior a Até mostra mensagem e bloqueia consulta/exportação. Ao limpar o filtro, os dados retornam. O teste adicional NAV-RACE-02 verifica também a resposta que já estava em andamento.

74. Navegação, permissões e continuidade

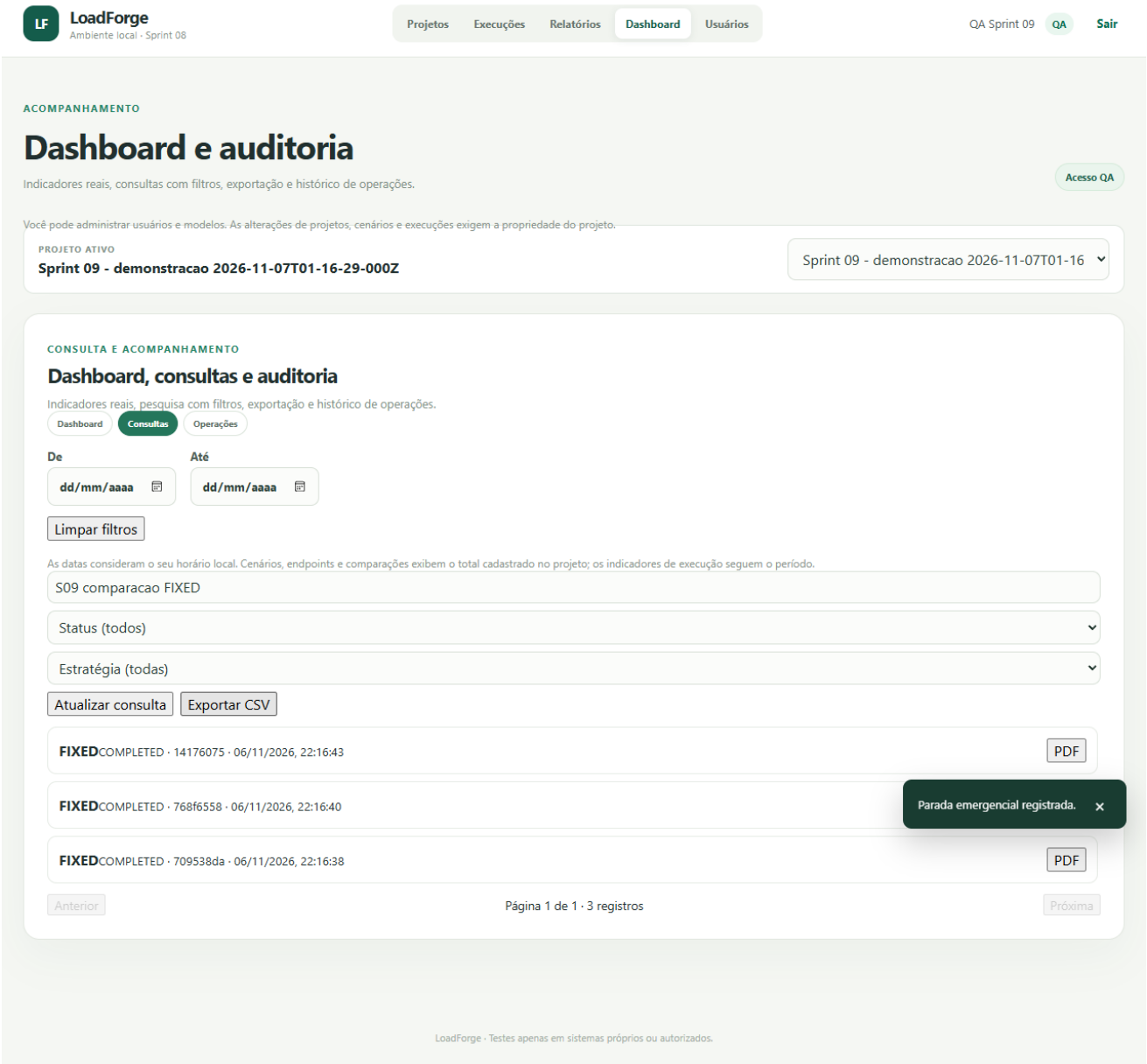


Figura 8 - NAV-07. Pesquisei o cenário FIXED e recuperei suas três execuções. O CSV baixado continha FIXED e não continha AI_HYBRID.

Evidências de execução - continuação

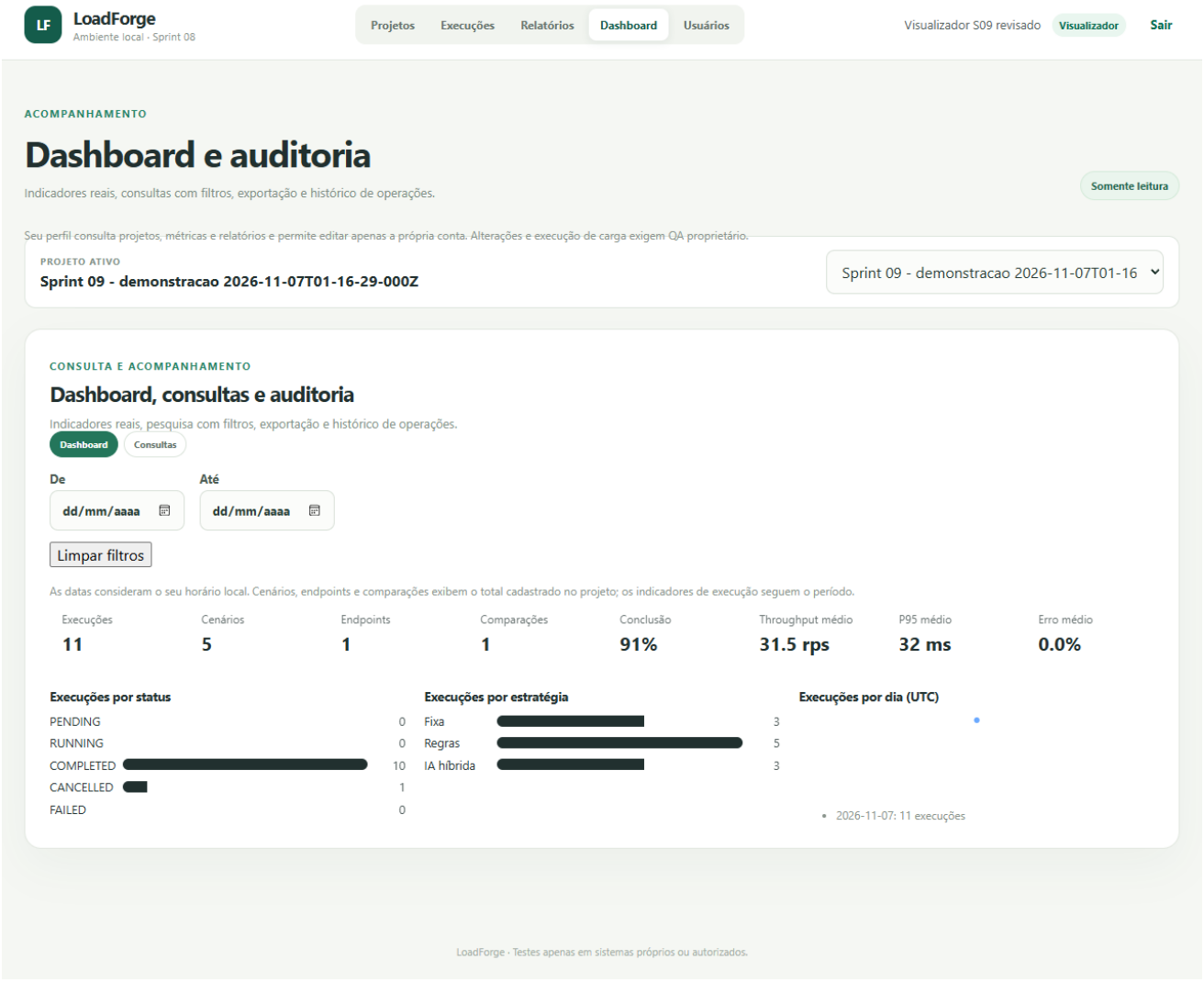


Figura 9 - NAV-11. Entrei como Visualizador e consultei o dashboard. Não havia Operações nem controles de execução/treino; chamadas diretas de escrita e auditoria retornaram 403.

Também confirmei navegação entre todas as áreas, edição/cancelamento, persistência ao recarregar, logout e QA não proprietário. O papel restringe ações na API; os botões refletem as mesmas regras. O ambiente é compartilhado para leitura autenticada, conforme o MVP.

75. Bugs corrigidos e retestes

Bug confirmado	Correção e comprovação
S09-B01 - autorização coercível (alta)	POST/PUT aceitavam textos/números como confirmação. Field strict=True exige booleano JSON. Dez falhas antes; 14 casos aprovados depois, incluindo controles.
S09-B02 - erro JSON null (média)	Leitura direta de detail causava TypeError em API/download. Acesso seguro conserva mensagem com status. Caso null reproduzido; 11 testes frontend aprovados.
S09-B03 - contexto de endpoint (alta)	Resposta de A podia preencher B e apagar rascunho. Invalidação por troca de projeto/sessão impede atualizar lista/formulário/aviso antigos. NAV-RACE-01 aprovado.
S09-B04 - carregamento preso (média)	Datas inválidas durante consulta deixavam loading ativo. Estado é encerrado no intervalo inválido; NAV-RACE-02 aprovado e limpeza recupera dashboard.

Os reprodutores de navegação retardam apenas respostas de requisições reais; não substituem os dados por conteúdo inventado. Os registros before/after preservam esperado, obtido e resultado. A severidade expressa impacto funcional e de regra, sem afirmar exploração externa.

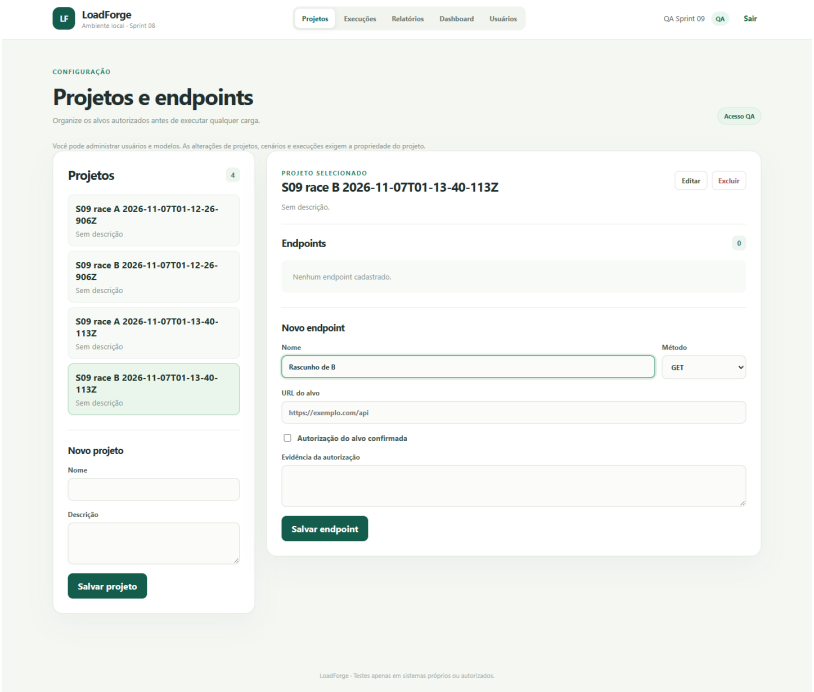


Figura 10 - Reteste S09-B03. A resposta anterior foi liberada depois de selecionar B. O projeto B conservou seu rascunho e não exibiu endpoint de A; as duas listas também foram conferidas pela API.

76. Atualização da documentação técnica

Revisei arquitetura, classes, MER e modelo relacional contra o código e o esquema. A solução permanece um monólito modular: React/Nginx encaminha REST à FastAPI; SQLAlchemy/Alembic persistem no PostgreSQL; motor, métricas, IA e controle executam no backend; artefatos de modelo são locais. O módulo audit registra operações na mesma transação e sua consulta global exige QA.

Material atualizado	Decisão e motivo
Classes/MER/relacional	Incluem auditoria e comparações; filhos opcionais corrigidos. O esquema tem 13 entidades, além da tabela de versão Alembic.
Auditoria	actor_id opcional usa SET NULL; entity_id é referência polimórfica sem FK, para preservar log após exclusão.
IA e controle	Horizonte de cinco janelas futuras; 2 s são nominais, com duração observada variável. Ausência de modelo aprovado mantém fallback explícito.
Imagens atuais	Três painéis derivados do ORM mostram colunas, PK/FK e nulabilidade por domínio, evitando ligações cruzadas. Ilustrações antigas identificadas como planejamento.
README, cronograma e contribuição	Estado verificado, links de evidências, divisão e branches por tarefa. Relatório técnico e roteiro de QA detalham execução e rastreabilidade.

Fontes e imagens: docs/diagrams. As novas exportações locais estão em docs/diagrams/exports/sprint-09/. A publicação desta revisão depende de autorização; o link aponta ao repositório da equipe e não comprova que as novas branches já foram enviadas.

Não houve alteração de tabelas nesta sprint. Os testes de equivalência entre schema.sql, migrations e modelos passaram na regressão. As transições start/cancel pertencem à API/motor; os nomes de serviços no desenho representam responsabilidades conceituais, não métodos inexistentes nas entidades ORM.

77. Dificuldades, próximos passos e referências

Dificuldades. A inicialização do Docker, uma porta indisponível no Windows e o contexto SQLite exigiram ajustes do laboratório. Para reproduzir falhas de navegação, controlei a liberação de respostas reais e registrei o antes/depois. Ajustes de seletores da automação foram separados dos bugs da aplicação.

Próximos passos. Revisar e publicar as branches por tarefa mediante autorização, com PRs e dependências declaradas. Atualizar as bibliotecas que emitem oito avisos de depreciação com nova regressão. Ampliar ensaios com modelos aprovados, recall/antecipação em teste reservado e diferentes resoluções. Planejar recuperação do motor após queda do processo da API. Os quatro bugs confirmados desta sprint foram corrigidos e retestados.

Limites da evidência. A comparação local com fallback comprova o fluxo funcional e a persistência, sem demonstrar superioridade da IA ou generalização em produção. A cobertura de validação/permissões não constitui pentest completo. Credenciais e tokens permanecem fora das evidências públicas.

Referência no repositório	Conteúdo
docs/relatorio-sprint-09.md	Análise, divisão, casos, regras, resultados e limites.
docs/bugs-sprint-09.md	Reprodução, causa, correção, severidade e reteste por bug.
docs/evidences/sprint-09/	JUnit, CSVs de casos, resultados JSON, capturas e downloads reais; hashes SHA-256.
docs/qa-sprint-09.md	Preparação e roteiro manual de 42 passos para demonstração.
CONTRIBUTING.md	Branches por tarefa, commits e revisão por PR.

Repositório da equipe:
https://github.com/camimcl/sre-fabrica-de-software

O Scrum Master coordena o envio único em nome dos cinco integrantes identificados. A consolidação desta sprint mantém os registros anteriores e acrescenta procedimentos e resultados demonstráveis.